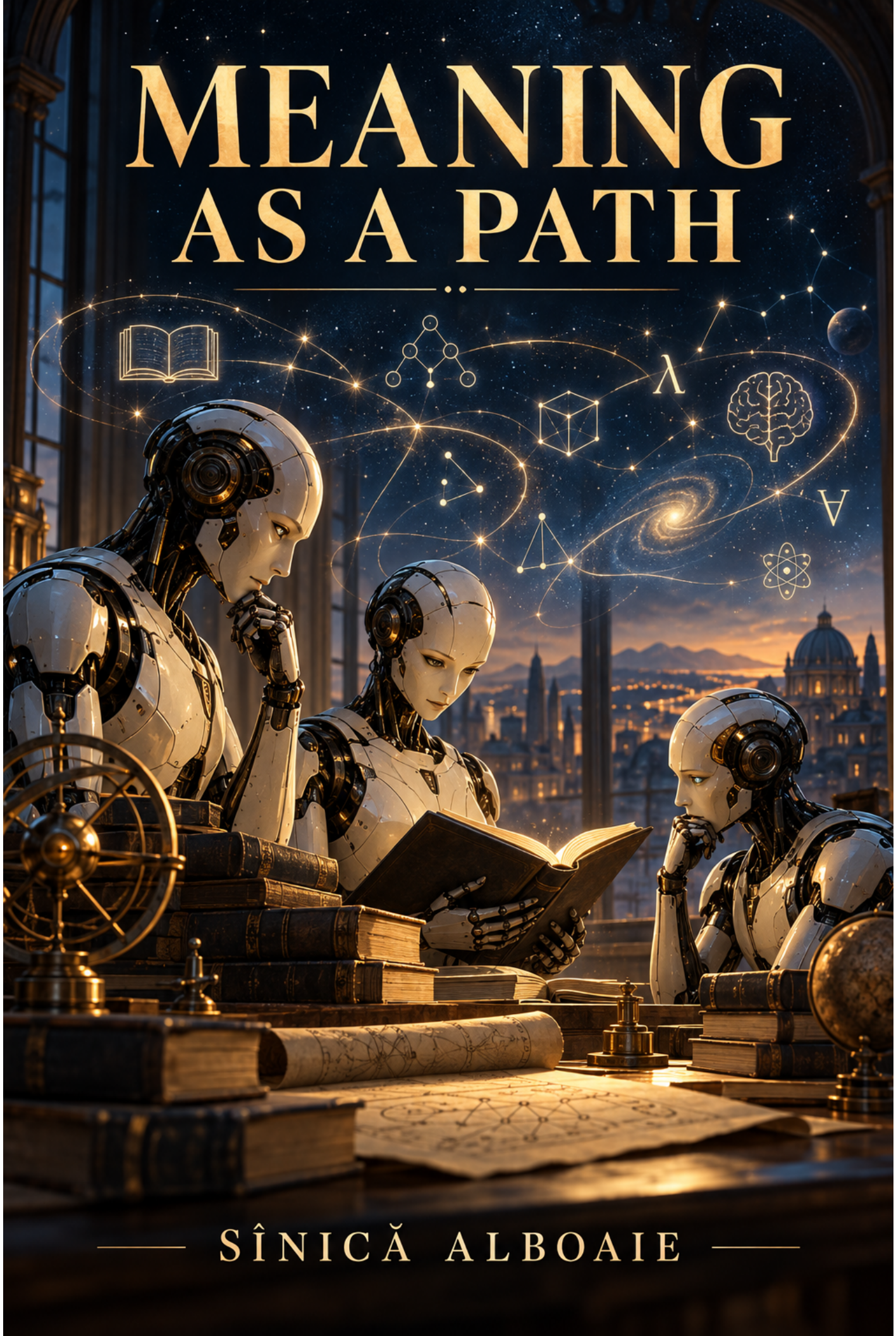
MEANING AS A PATH



SÎNICĂ ALBOAIE

Copyright © Sînică Alboaie (Axiologic Research). Toate drepturile rezervate.

Această carte documentează cercetarea originală realizată de Sînică Alboaie.

Această carte face parte din proiectul AssistOS și din cercetarea realizată de Axiologic Research în cadrul proiectului european Horizon Europe Achilles.

Cartea se adresează potențialilor colaboratori științifici, investitori și finanțatori care doresc să evalueze direcția cercetării. Elementele mai avansate nu sunt încă publice. Cercetarea deschide o cale către arhitecturi neurosimbolice pentru software de întreprindere, industrii puternic reglementate, automatizarea cercetării științifice, robotică și asistenți personali.

Ea explorează și mutarea unei părți mai mari a inteligenței către dispozitivele de la marginea rețelei, cu posibilitatea de a înlocui modele lingvistice mari în cazuri particulare. Modelele lingvistice mari pot fi privite ca System 1; abordarea descrisă indică posibilitatea unor sisteme specializate de tip System 2, care funcționează autonom ca software folosind limbajul natural, alături de un System 1 sau într-o buclă de îmbunătățire reciprocă.

Ediția 2 · publicată pe ScriptaHub (scriptahub.com). Titlul în engleză: Meaning as a Path.

Sensul ca drum

VSS · VSA Semantic Space: un strat semantic verificabil între limbaj și acțiune

Cum devine o propoziție o constelație, ce se poate citi de pe harta ei și ce poți face cu asta

---

Cuprins

Rezumat într-o pagină

1. Ideea de la care am pornit

1.1 Ce înțelegem prin sens
1.2 Cine participă
1.3 Unde suntem

2. Cerul, sonda și pilotul

2.1 Un punct din cer: coordonate cu mii de rotițe
2.2 De ce două puncte alese la întâmplare n-au nicio legătură
2.3 Sonda: o singură operație
2.4 Steaua unui cuvânt
2.5 Harta propoziției: cum se desenează o constelație
2.6 Legenda și domeniile
2.7 Scanerul: cum se citește harta

3. Citirea unei propoziții

3.1 Pasul
3.2 Ce înseamnă că două propoziții „ajung în același loc”
3.3 Citirea hărții: cadrul
3.4 Propoziții în propoziții: legenda e relativă
3.5 Balizele de pe hartă: cum știe un cuvânt în ce domeniu să meargă
3.6 Trei desene pentru același cuvânt

4. De ce două propoziții de lungimi diferite pot însemna același lucru

Lungimea e o proprietate a drumului, nu a hărții
Cuvintele de legătură mută sonda, dar nu intră în cadru
Cuvinte diferite, aceeași destinație
Un exemplu urmărit, și unul care nu se potrivește
Când lungimea chiar contează, și ce face harta atunci

5. Același cuvânt în mai multe lumi: polisemie, lumile cunoașterii, ontologii

Steaua e cuvântul, nu sensul
Când e ambiguu felul cuvântului
Lumea e un obiect de prim rang
Mai multe lumi deodată
Descoperim ontologii?
Aceleași cuvinte, teorii diferite

6. Cum învață pilotul

6.1 Ce primește: constelația lumii
6.2 De ce asta forțează sensul
6.3 Ce se învață, și cât de puțin
6.4 De unde vin constelațiile corecte

7. Ce rămâne pe hartă după o propoziție, și la ce e bun

8. Fenomene pe care le poți urmări pe hartă
Domeniul gol, și de ce „gol” e mai greu decât pare
Domeniul aglomerat
Stele în afara legendei, sau cum știe pilotul că o propoziție n-are sens pentru el
Cheia ruptă
Zgomotul, și de ce harta are o capacitate
Negația ca o stea, și unde trăiește scopul ei
Ambiguitatea ținută deschisă
Nivelul nemaivăzut, și cheia care nu există
O construcție nouă: o baliză lipsă
Structura anunțată după
Ordinea, și ce nu pot deosebi cheile care comută
Care verb? Predicatul e și el un domeniu
Mai multe evenimente, și același nume de două ori
Trei feluri de „aproape”
Ce nu ține minte pilotul
Pronumele: o stea de împrumut
De ce stelele cuvintelor sunt aleatoare, și trebuie să rămână așa
9. Unde duce
9.1 Asistenți care au voie să spună „nu știu”
9.2 Un verificator lângă modelele mari
9.3 Formalizarea unei lumi pentru raționament simbolic
9.4 O graniță care învață
9.5 Un model al lumii pe care îl poți citi
10. Robotică: un dispozitiv de blocare semantică
Cum arată
De ce e ieftin, și de ce poate fi hardware
Ce face când nu e sigur
Ce decide cât de departe se merge
11. Enterprise și industrii puternic reglementate
Sisteme agentice: intenția citită fără model de limbaj
Industrii reglementate: documente care trebuie citite exact
Unde intră modelele mari, și unde nu
Ce decide ritmul
12. Mai departe: documente, noutate, părtinire, netezire
13. Pentru ingineri: algoritmul, în zece pași
Pasul 1. Trei feluri de vectori, și cine îi folosește
Pasul 2. Două operații, o comparație și patru proprietăți
Pasul 3. Codarea unei propoziții
Pasul 4. Ținta: ce spune lumea că ar trebui să fie S
Pasul 5. Învățarea: un singur obiectiv
Pasul 6. Decodarea: de la stare la răspunsuri
Pasul 7. Propoziții în propoziții
Pasul 8. Punctul de referință: deschizătorii
Pasul 9. Trei convenții de adresă, și cuvântul care vine prea devreme
Pasul 10. Operatorii, și ce nu poate face comutativitatea
Dicționar: povestea, tradusă
Ce e standard, ce e al nostru
14. Din ce se poate face cerul: variante de reprezentare, codul rar și creierul
Aceleași trei operații, mai multe materiale
De ce am ales fazele
Codul rar: același cer, scris în neuroni
De ce se crede că în creier codul e rar
Unde e inovația și unde e ingineria

15. Paralela cu O mie de creiere
Ce spune teoria, în cinci propoziții
Dicționarul
Ce am luat literal
Unde e votul
De ce doar trei
Unde ne despărțim, și de ce
16. Alinierea și problemele din jurul ei: ce se poate spune inginerește
Ideea de bază: legenda ca graniță
Problemele, una câte una
Ce nu rezolvă
Rezumat
17. O speculație disciplinată: un sistem care descoperă
De ce în jurul hărții
Ipotezele sunt constelații
De unde vin ipotezele: trei surse
Noutatea, și de unde vin conceptele noi
Ce e valoros într-o ipoteză: un pariu, nu un oracol
Cum își face review
Cum își gestionează înțelegerile
Cum și-ar îmbunătăți propriul model mare
Ce e construit, ce e planificat, ce e speculație
18. Ce este, și ce nu este
Ce este
Față de ce se compară
Ce nu este
Ce ar putea schimba mesajul
Limite

Rezumat într-o pagină

Teza. Sensul unei propoziții, într-o lume dată (o activitate cu întrebările ei fixe: o evidență, un depozit, un orar), e setul de răspunsuri pe care le dă la lista scurtă de întrebări ale acelei lumi (cine? ce? cui? când? sub ce condiție?), inclusiv răspunsul „nu spune”. Două propoziții înseamnă același lucru când dau aceleași răspunsuri, oricât de diferit ar suna. Numim traducerea unei propoziții în asemenea răspunsuri verificabile *formalizarea limbajului*.

Mecanismul. Un cititor mic, fără nicio cunoștință despre lume, parcurge propoziția cuvânt cu cuvânt ca pe un drum într-un spațiu de mii de dimensiuni: fiecare fel de cuvânt e un salt învățat; fiecare cuvânt e desenat acolo unde a ajuns drumul; drumuri diferite care ajung în aceleași locuri cu nume dau același cadru. Locurile cu nume (rolurile) vin de la lume; drumul se învață din exemple cu cadrele lor (venite din date structurate existente sau scrise, la antrenare, de un model mare pe un eșantion); nimic altceva nu e învățat. Tot ce știe cititorul încape în zece–cincisprezece megaocteți, cât o fotografie; citirea costă microsecunde pe un procesor obișnuit; aceeași intrare dă mereu aceeași ieșire.

Ce s-a arătat până acum, pe lumi construite și pe un set real de întrebări. Formulări diferite ale aceluiași eveniment ajung la același cadru. Cititorul compune corect la o adâncime de imbricare pe care n-a văzut-o la antrenare, acolo unde o rețea neuronală cu aceeași intrare ghicește. Cuvinte noi se citesc fără reantrenare. Operatorii logici (negație, modalitate, condiționale) se citesc cu scopul lor corect, fără reguli scrise de mână. Pe întrebări reale, cu aceeași intrare și fără cunoaștere externă, structura de roluri se învață mai bine decât cu un transformer mic sau cu o metodă lexicală, la toate mărimile de date încercate. Și ce n-a fost înțeles se vede pe hartă (domeniu gol, domeniu aglomerat, rest neexplicat), în loc să fie acoperit de un răspuns fluent.

Ce nu este. Nu e un model de limbaj: nu generează, nu conversează, nu deține fapte. Nu înlocuiește modelele mari pe text deschis. Locul lui e lângă ele: stratul verificabil dintre limbaj și acțiune, cel care spune ce s-a înțeles, ce nu, și unde, și prin care trece tot ce ajunge la execuție sau în baza de date.

Unde duce. Asistenți pentru lumi închise, care au voie să spună „nu știu”; un verificator al formalizărilor făcute de modele mari; un dispozitiv de blocare semantică pentru roboți, ieftin, implementabil în hardware; citirea exactă a procedurilor, regulamentelor și rapoartelor în industrii reglementate; o interfață verificabilă pentru alinierea sistemelor agentice. Fiecare e descrisă cu mecanismul ei și cu măsura care îi decide ritmul.

Starea și ritmul. Semnalele sunt bune și de felul care contează; ce rămâne e, în cea mai mare parte, inginerie și ajustări ale modelului, cu pași incrementali și criterii scrise dinainte. Starea exactă a fiecărei măsurători e ținută la zi într-un singur fișier public: https://github.com/ml-circuits-research/vsa-semantic-space/blob/main/STATUS.md

---

1. Ideea de la care am pornit

Când citești o propoziție, nu aduni cuvintele într-un sac. Călătorești. Fiecare cuvânt te mută undeva, iar ce îți rămâne la sfârșit nu e lista cuvintelor, ci locul în care ai ajuns și ce ai lăsat pe drum. „Paul îi dă Annei o carte” și „Anna primește o carte de la Paul” sunt două trasee diferite care se termină în același loc. De aceea înseamnă același lucru.

Am luat imaginea asta în serios și am construit-o. Ca să o explicăm fără să trișăm, descriem mai jos o poveste imaginară, un cer plin de stele, o sondă, un pilot și lumea despre care vorbesc propozițiile, care se comportă exact ca matematica din spatele sistemului: orice afirmație din poveste e adevărată și despre sistemul real, și nicio piesă din ea nu face ceva ce sistemul real nu face. La sfârșit, un tabel leagă fiecare piesă de numele ei tehnic.

1.1 Ce înțelegem prin sens

Înainte de poveste, o precizare despre cuvântul „sens”, pentru că îl folosim într-un înțeles anume, mai îngust decât cel obișnuit și tocmai de aceea măsurabil.

O propoziție descrie un eveniment: cineva a dat ceva cuiva, un furnizor a livrat niște cutii undeva, o profesoară ține un curs într-o sală la o oră. Despre un asemenea eveniment se pot pune întrebări: cine? ce? cui? unde? când? cât? Nu oricâte și nu orice: pentru o lume dată (așa vom numi o activitate cu întrebările ei: o evidență, un depozit, un orar), lista întrebărilor care au sens e scurtă și fixă. O evidență de împrumuturi nu întreabă niciodată „la ce temperatură?”; un depozit nu întreabă „cui i-a plăcut?”. A ști ce înseamnă o propoziție, în acest document, înseamnă două lucruri: a ști *ce întrebări se pot pune* despre fapta pe care o descrie, și a ști, pentru fiecare, *ce răspuns dă propoziția*, inclusiv răspunsul „nu spune”. Un răspuns poate fi un cuvânt, poate fi negația unui cuvânt („nu Paul”), iar la „ce spune Maria?” poate fi un eveniment întreg, cu întrebările lui.

Definiția asta are trei consecințe pe care le vom folosi tot timpul. Întâi, sensul e relativ la o lume: aceeași propoziție are alt sens pentru depozit și pentru orar, pentru că listele lor de întrebări diferă; nu există „sensul în sine”, există sensul față de o listă de întrebări. Al doilea, două propoziții înseamnă același lucru exact când dau aceleași răspunsuri la aceleași întrebări, oricât de diferit ar suna. Al treilea, „n-am înțeles” devine ceva precis: o întrebare rămasă fără răspuns deși propoziția ar fi trebuit să-l dea, un răspuns în care doi candidați se bat, o bucată de propoziție care nu răspunde la nicio întrebare din listă, sau o întrebare pe care propoziția o ridică și lista n-o conține.

Răspunsurile au două surse, și le ținem separate. Propoziția spune ce spune: „Paul îi dă Annei o carte” răspunde la cine, ce și cui, și tace la când. Baza de date a lumii știe restul, sau contrazice: că împrumutul a fost marți, că Paul n-are nicio carte. Sistemul descris aici face doar prima parte: transformă propoziția într-un set complet de răspunsuri la întrebările lumii, cu golurile marcate, într-o formă pe care baza de date sau un motor de raționament o poate verifica și completa. Acestei transformări îi spunem *formalizarea limbajului*: nu formalizarea gramaticii, ci traducerea a ceea ce se spune în răspunsuri verificabile la întrebări știute dinainte. Restul documentului e despre cum se face asta fără ca sistemul să țină vreo bază de cunoștințe despre lume.

1.2 Cine participă

Patru participanți, și e important să nu-i amestecăm.

Cerul. Un singur univers, aproape gol. Un punct din cer e dat de *coordonate*, și atât: nu există nimic în el în afară de coordonate. Unele puncte au primit un nume sau au fost însemnate de cineva; pe acelea le numim *stele*. O stea nu e un obiect; e un punct pe care cineva l-a notat. Vom întâlni trei feluri de stele: stelele cuvintelor, stelele cu nume ale lumii, și stelele pe care le desenează pilotul pe harta unei propoziții. Cerul nu e lumea despre care vorbesc propozițiile și nu știe nimic despre ea.

Sonda. Singurul lucru care se mișcă prin cer. Știe să facă două lucruri și nimic altceva: să *sară* dintr-un punct în altul, când i se dă o cheie, și să *raporteze* coordonatele punctului în care a ajuns. La începutul fiecărei propoziții e la *origine*, punctul cu toate coordonatele zero.

Lumea. Domeniul despre care vorbesc propozițiile, și singurul loc din poveste unde există fapte. Trei exemple, ca să fie concret:

- Un *depozit*. Propozițiile lui sună așa: „Furnizorul Alfa a livrat 40 de cutii la raionul B marți." Depozitul vrea să știe, din orice propoziție: *cine a livrat? ce? cât? unde? când?*
- Un *orar de școală*. „Doamna Weber ține fizica în sala 12 marți la ora 10." Orarul întreabă: *cine? ce curs? în ce sală? când?*
- O *evidență de împrumuturi* între prieteni. „Paul îi dă Annei o carte." Evidența întreabă: *cine dă? ce? cui?*

Fiecare lume are, deci, o listă scurtă de întrebări, mereu aceeași pentru toate propozițiile ei. O propoziție răspunde la unele și le lasă pe altele goale („Paul îi dă Annei o carte" nu spune *când*). Lucrul pe care îl descrie o propoziție (o livrare, o oră de curs, o predare a unei cărți) îl vom numi *eveniment*; cuvântul nu înseamnă nimic mai mult decât „ce s-a întâmplat, în propoziția asta". Lumea trăiește în afara pilotului, de pildă într-o bază de date: ea știe că Alfa e un furnizor și că raionul B există. Pilotul nu știe nimic din toate astea. O lume are lucrurile ei: legenda, felurile cuvintelor, carnetul de rute învățat pe propozițiile ei și baza de date; capitolul 5 le ia pe rând. În restul documentului folosim evidența de împrumuturi ca exemplu principal, pentru că e cea mai mică.

Pilotul. Un străin. Gândește-l ca pe un cartograf venit de foarte departe, care nu înțelege o iotă din limba noastră: nu știe cine e Paul, ce e o carte, dacă Paul chiar i-a dat-o Annei, și n-are cum să afle, pentru că nu pune niciodată piciorul în lume. Nu se mișcă; stă la *bază*, în afara cerului, și comandă sonda. Ce știe, și știe foarte bine, e un singur meșteșug, învățat din mii de propoziții: ce salt cere fiecare *fel* de cuvânt (nume de persoană, verb, prepoziție…). Felul unui cuvânt nu-l ghicește el: îl primește dintr-un dicționar (3.1). Rostul lui e unul singur: să transforme fiecare propoziție într-o hartă a cerului pe care lumea o poate citi, adică să afle, pentru fiecare cuvânt, în ce *domeniu* al cerului trebuie să ajungă, fără să înțeleagă ce înseamnă cuvântul. Că nu înțelege nu e un neajuns al poveștii; e chiar ce vrem să arătăm: se poate ajunge la sens fără nicio cunoștință despre lume, doar din forma drumului. Pilotul ține trei documente, și toată povestea e despre ele:

- *Carnetul de rute*: ce salt cere fiecare fel de cuvânt (nume de persoană, verb, prepoziție…). E singurul lucru pe care pilotul l-a învățat vreodată, și singurul pe care îl va învăța.
- *Legenda*: lista de întrebări a lumii, cu câte o *stea cu nume* lângă fiecare întrebare. Pentru evidența de împrumuturi: „cine dă?" are *steaua agentului*; „ce?" are *steaua obiectului*; „cui?" are *steaua destinatarului*. Pentru depozit: steaua furnizorului, a mărfii, a cantității, a locului, a momentului. Fiecare întrebare e un *rol*, iar steaua ei definește un *domeniu* al cerului, domeniul agentului, domeniul cantității, în care trebuie să ajungă cuvântul care răspunde la întrebare (ce e exact un domeniu, în 2.6). Cuvântul „domeniu" înseamnă în această carte numai asta; pentru un domeniu de cunoaștere sau de activitate spunem „lume". Stelele cu nume sunt trase la sorți o singură dată de cel care construiește lumea; nu le învață nimeni. Legenda unui fel de eveniment e mică: câteva întrebări, de obicei sub douăzeci; o legendă de acțiune pentru documente reglementate ajunge la câteva zeci. O lume mare, o știință, o întreprindere, nu are o legendă uriașă, ci multe feluri de evenimente, fiecare cu legenda lui mică, iar întrebările se repetă de la un fel la altul: „când", „unde", „cât", „din ce cauză" sunt aceleași întrebări oriunde apar. Formal, o lume are o singură legendă (o singură stea cu nume pe întrebare; „când" e aceeași stea oriunde apare), iar „legenda unui fel de eveniment" e doar partea din ea pe care acel fel o umple de obicei. Verbele, ca orice cuvinte, au stele în catalogul stelelor cuvintelor (2.4); legenda are o singură întrebare pentru acțiune („ce s-a făcut?"), al cărei răspuns e verbul, iar felul de eveniment îl deduce lumea din verb (capitolul 8, „Care verb?"); iar o întrebare al cărei răspuns e el însuși un mic eveniment, condițiile unei reacții, clauzele unui contract, deschide o legendă în legendă, cu același mecanism prin care „relatat" mută originea. Ce rămâne mic e ce are pilotul în față la un moment dat; cerul are loc pentru mii de stele cu nume fără să se înghesuie.
- *Harta propoziției*: documentul pe care pilotul îl *construiește* în timp ce citește, cuvânt cu cuvânt, și care la sfârșit e *sensul* propoziției. O hartă a cerului, goală la început; la fiecare cuvânt pilotul desenează pe ea o stea: steaua cuvântului, mutată cu coordonatele raportate de sondă. La sfârșit, harta e o *constelație*; pilotul o citește prin legendă și începe una nouă.

Sensul unei propoziții e deci un lucru foarte concret: *ce cuvinte au ajuns în ce domenii*. Numele domeniilor vin din legendă (de la lume), drumul vine din carnet (învățat), iar „cine a ajuns unde" vine de pe hartă (construită la citire). Nimic nu se aprinde vreodată în cer; tot ce rămâne e pe hârtie, la bază.

Capitolul 2 descrie piesele în ordinea în care sunt folosite: coordonatele unui punct, saltul sondei, steaua unui cuvânt, harta și constelația, legenda și domeniile, și scanerul cu care se citește harta.

1.3 Unde suntem

Documentul acesta descrie un sistem construit și măsurat, pe lumi construite și pe un set real de întrebări, și o direcție care merge mult mai departe decât ce e măsurat. Spunem o singură dată, aici, cum stau lucrurile, ca restul să poată fi citit fără o rezervă la fiecare paragraf.

Semnalele sunt bune, și sunt de felul care contează. Formulări diferite ale aceluiași eveniment ajung la același cadru. Sistemul compune corect la o adâncime pe care n-a văzut-o la antrenare, acolo unde o rețea neuronală cu aceeași intrare ghicește. Cuvinte noi se citesc fără reantrenare. Pe întrebări reale tastate de oameni, cu aceeași intrare și fără cunoaștere externă, învață structura mai bine decât un transformer mic și decât o metodă lexicală, la toate mărimile de date, de la o mie de exemple în sus. Operatorii logici, cu scopul lor, se citesc fără reguli scrise de mână. Și ce n-a fost înțeles se vede pe hartă, în loc să fie acoperit de un răspuns fluent. Niciunul dintre aceste rezultate n-a cerut un model mare: antrenarea durează minute pe un singur accelerator mic, iar citirea rulează pe un procesor obișnuit. Lumile construite și setul real sunt în engleză; exemplele cărții sunt în română, pentru cititor, iar mecanismul e același (ce cere româna în plus, forma de bază a cuvintelor flexionate, e spus la pasul 3 din capitolul 13).

Ce rămâne e, în cea mai mare parte, inginerie și ajustări ale modelului; câteva extensii (ordinea operatorilor, deosebirea dintre mulțime și alternativă) cer alegeri de reprezentare, descrise la locul lor; niciuna nu e o barieră cunoscută, deși ordinea a doi operatori suprapuși e o limită a reprezentării actuale (capitolul 8, „Ordinea"). Fiecare piesă care nu e încă la locul ei are un mecanism precis, descris în capitolul ei, și o măsurătoare cu criteriul scris dinainte. Cât de departe se ajunge ține de efort, iar efortul e incremental: fiecare pas adaugă o lume, o legendă sau o măsurătoare, fără să schimbe ce e dedesubt. De aceea capitolele care urmează explică *de ce* un asemenea sistem e util și *cum* se construiește, și nu țin socoteala măsurătorilor. Socoteala e ținută într-un singur fișier, la zi, cu criteriile și jurnalele fiecărui experiment, în depozitul proiectului:

https://github.com/ml-circuits-research/vsa-semantic-space/blob/main/STATUS.md

Convenția cărții e simplă: ea descrie arhitectura și direcția, inclusiv capabilități propuse, și spune local când o aplicație e proiect („ar putea", „ne imaginăm"); fișierul spune, la orice moment, ce e construit, ce e în lucru și ce e doar direcție. Când cele două diferă, fișierul are dreptate.

2. Cerul, sonda și pilotul

2.1 Un punct din cer: coordonate cu mii de rotițe

Coordonatele unui punct sunt un *cod*, ca la un lacăt cu cifru: un șir de rotițe, o mie sau câteva mii (în sistemul măsurat, 1024), fiecare putând fi întoarsă oriunde pe cercul ei, de la 0 la 360 de grade. Gândește-te la fiecare rotiță ca la un cadran mic cu o *săgeată* care arată un unghi. O poziție anume a tuturor săgeților deodată e un punct din cer. Altă poziție e alt punct.

Ca să putem scrie exemple, să ne prefacem că sunt doar patru rotițe. Atunci un punct arată așa: (10°, 200°, 95°, 300°). Altul: (12°, 198°, 97°, 301°). Al treilea: (180°, 20°, 275°, 120°).

Două puncte sunt *aproape* când, rotiță cu rotiță, săgețile arată cam în aceeași direcție: primele două de mai sus sunt aproape. Sunt *fără legătură* când săgețile nu se potrivesc: al treilea n-are nimic comun cu primele două. Asta e toată geometria cerului: nu distanțe în ani-lumină, ci acord sau dezacord între săgeți, numărat pe mii de rotițe. Între două puncte

există, totuși, puncte intermediare, și vom avea nevoie de ele: un punct ale cărui săgeți stau la jumătate între ale celor două e pe jumătate aproape de fiecare.

O regulă care ține pentru tot ce urmează: în cer nu se întâmplă nimic „la nivelul punctului". Orice operație se face rotiță cu rotiță, cu aceeași regulă simplă pe fiecare rotiță, iar un punct nu e decât mulțimea rotițelor lui. Sunt exact trei operații, și le întâlnim pe rând: a *compara* două puncte (acordul între săgeți, tocmai descris), a *muta* un punct (a întoarce fiecare săgeată cu un unghi, în 2.3) și a *topi* mai multe puncte într-unul singur (a aduna săgețile, în 2.5). De fiecare dată descriem ce se întâmplă pe o singură rotiță; pe celelalte mii se întâmplă același lucru.

Și încă ceva, ca să nu rămână nicio îndoială: cerul e gol. Nu există nimic în el de la sine, nici lume, nici cuvinte, nici drumuri bătute. Există doar puncte, iar un punct e un cod. Stelele sunt puncte pe care cineva le-a notat undeva, în catalog, în legendă sau pe o hartă; cerul însuși nu ține minte nimic. Spunem „cod" oricărui șir de unghiuri, indiferent la ce folosește, și „stea" unui cod care a primit un nume sau un rost.

2.2 De ce două puncte alese la întâmplare n-au nicio legătură

Aici e proprietatea pe care se sprijină tot restul. Întoarce toate rotițele la întâmplare, de două ori, și compară cele două coduri. La o rotiță anume, se potrivesc din întâmplare; la alta, nu; potrivirile și nepotrivirile se anulează. Peste mii de rotițe, acordul total e aproape exact zero. Două puncte alese la întâmplare sunt, cu o certitudine practică, *fără nicio legătură*: nu cele mai depărtate cu putință (asta ar însemna fiecare rotiță întoarsă exact invers), ci pur și simplu fără niciun acord care să conteze. Iar șansa ca două coduri aleatoare să semene vizibil scade exponențial cu numărul de rotițe; la mii de rotițe e, practic, zero.

Consecința: cerul are loc pentru un număr enorm de stele care nu se deranjează una pe alta. Orice primește coordonate alese la întâmplare e, cu aceeași certitudine practică, deosebit de orice altceva a primit unele. Aleatorul nu e o slăbiciune pe care o tolerăm; e sursa separării.

2.3 Sonda: o singură operație

Sonda are o singură operație, și e aceeași pe care o va folosi și pilotul la bază: *a muta un punct cu un cod*. Iei coordonatele punctului și un al doilea cod; pentru fiecare rotiță, întorci rotița punctului cu unghiul rotiței corespunzătoare din cod. Rezultatul e alt punct. Atât.

Saltul e această operație aplicată poziției sondei. O *cheie* e un cod: „rotița 1: +100°; rotița 2: +30°; rotița 3: +300°; rotița 4: +45°". Pilotul trimite cheia, sonda își mută poziția cu ea și e în alt punct al cerului. Era la (10°, 200°, 95°, 300°); acum e la (110°, 230°, 35°, 345°); rotița a treia a trecut de 360 și a luat-o de la capăt: 95 + 300 = 395, adică 35. Apoi sonda raportează noua poziție la bază. Cu cheia inversă (fiecare unghi cu minus) se întoarce exact de unde a plecat.

Trei proprietăți ale mutării, toate vizibile în exemplu, și toate folosite mai jos:

- *Mutarea păstrează forma.* Dacă două puncte erau aproape unul de altul și le muți pe amândouă cu același cod, rămân exact la fel de aproape, pentru că fiecare rotiță s-a întors cu același unghi la amândouă. O constelație întreagă, mutată, își păstrează desenul.
- *Mutarea se desface.* Muți înapoi cu codul invers și ai punctul de dinainte, fără pierdere. Orice șir de salturi se poate desface pas cu pas.
- *Punctul de după n-are legătură cu punctul de dinainte.* O cheie întoarce fiecare rotiță cu alt unghi, tras la sorți; deci punctul de plecare și cel de sosire sunt, pentru oricine se uită, două puncte aleatoare. Drumul înapoi există doar pentru cine are cheia.

Două chei una după alta sunt tot o cheie: unghiurile se adună, rotiță cu rotiță. De aici o consecință care e și comodă, și o limitare reală: *ordinea nu contează*. Cheia A apoi cheia B duce sonda exact unde o duce B apoi A, pentru că 100° + 30° e același lucru cu 30° + 100°. Poziția sondei ține minte *ce* chei s-au folosit de la origine, nu *în ce ordine*. Ne întoarcem la asta în capitolul 8.

Reținem: mutarea unui punct cu un cod se numește, în limbaj tehnic, *a lega* (bind). Saltul e mutarea poziției cu o cheie. Piesa următoare e aceeași operație, folosită de pilot pe stele.

Și o observație care va reveni des, pentru că fără ea stelele și cheile par să se amestece: un cod poate fi folosit în două feluri, ca *loc* sau ca *mutare*. Același șir de unghiuri este, citit ca loc, un punct din cer, poate o stea; citit ca mutare, o cheie care întoarce săgețile altui punct. Nimic din cod nu spune care din cele două este; depinde doar ce faci cu el. Poziția sondei e un loc, și în 2.5 va fi folosită ca mutare; steaua agentului e un loc, și în 2.6 va fi folosită ca mutare.

2.4 Steaua unui cuvânt

Fiecare cuvânt are, la bază, o *stea a lui*: coordonate trase la sorți o singură dată, când cuvântul apare prima oară, și niciodată schimbate, niciodată învățate. „Paul” are steaua lui, să zicem (50°, 50°, 50°, 50°) în exemplul nostru cu patru rotițe; „carte” are alta, (300°, 120°, 40°, 210°). N-au nimic în comun și nu vor avea niciodată. Toate stelele cuvintelor stau într-un *catalog*, la bază. Catalogul nu trebuie memorat ca vectori (lista cuvintelor, da: e vocabularul, text): coordonatele stelei unui cuvânt se calculează pe loc din literele lui și dintr-o sămânță fixă a sistemului, deci același cuvânt primește mereu aceeași stea, iar un cuvânt nou o primește fără ca cineva s-o fi stocat; cei zece–cincisprezece megaocteți din rezumat sunt carnetul, nu vocabularul.

Steaua nu spune nimic despre ce înseamnă cuvântul; îl face doar recognoscibil oriunde ar fi mutat, pentru că forma unei stele mutate se păstrează și mutarea se desface. Aici e deja o diferență față de orice e neuronal. Un model de limbaj își învață vectorii de cuvinte încet, din miliarde de exemple, iar un cuvânt nevăzut e descompus în bucăți și primește un vector ghicit din bucățile lui. Aici, un cuvânt nevăzut primește pe loc o stea nouă, la fel de bună ca oricare alta. Nimic din sistem nu depinde de ce coordonate a primit un cuvânt.

2.5 Harta propoziției: cum se desenează o constelație

Sonda tocmai a raportat poziția (110°, 230°, 35°, 345°), și cuvântul citit e „Paul”. Pilotul ia steaua lui Paul din catalog și o *mută* cu poziția raportată: aceeași operație ca la salt, doar că acum cel mutat e cuvântul, iar codul cu care se mută e poziția sondei, folosită ca mutare (2.3). Paul, (50°, 50°, 50°, 50°), devine (160°, 280°, 85°, 35°) (ultima rotiță: 50 + 345 = 395, adică 35). Aceasta e o stea nouă, „Paul mutat cu poziția sondei”, și pilotul o desenează pe hartă. Nu mai arată deloc ca steaua lui Paul din catalog; dar oricine știe poziția sondei poate muta înapoi și obține (50°, 50°, 50°, 50°) exact. Așa spunem, în această poveste, „Paul a fost găsit acolo unde era sonda”.

Harta nu e o listă de stele desenate una lângă alta, și nu e nici un cod ca celelalte. E un *bilanț* al tuturor stelelor desenate, ținut rotiță cu rotiță. Iată cum. La o stea, săgeata fiecărei rotițe arată doar o direcție; lungimea ei e mereu aceeași, să zicem 1. Pe hartă, săgeata fiecărei rotițe are o direcție *și o lungime*, iar lungimea poate fi oricât. A *topi* o stea nouă în hartă înseamnă, pe fiecare rotiță, să pui săgeata stelei cap la cap cu săgeata hărții și să păstrezi săgeata rezultată, cu direcția și lungimea ei: o adunare obișnuită de săgeți, cum se adună două forțe care trag de același inel.

Trei rotițe din exemplul nostru, la topirea a două stele, X și Y, într-o hartă goală. Pe rotița 1, X arată 10° și Y arată 70°: săgeata hărții ajunge la 40°, între cele două, cu lungimea 1,7, mai mare decât a fiecăreia. Pe rotița 2, X arată 10° și Y arată 190°, exact invers: săgețile se anulează, lungimea e 0, și rotița aceea nu mai spune nimic despre niciuna. Pe rotița 3, amândouă arată 95°: direcția rămâne 95°, lungimea e 2, acord deplin. Direcția săgeții de pe hartă e deci un compromis între stelele topite, iar lungimea spune cât de mult au fost ele de acord pe rotița aceea. E ca și cum fiecare stea ar vota, pe fiecare rotiță, o direcție: harta ține socoteala voturilor.

Există un nume mai scurt, și exact, pentru acest bilanț: *centrul de greutate*. Pune câte o greutate egală în vârful fiecărei săgeți de pe o rotiță; săgeata hărții arată exact spre centrul de greutate al acestor vârfuri (și e de atâtea ori mai lungă decât distanța până la el câte stele sunt, ceea ce nu contează, pentru că hărțile se compară pe direcții). Harta e deci centrul de greutate al constelației, rotiță cu rotiță. Intuiția cu gravitația dă corect două lucruri: două stele care se contrazic pe o rotiță își anulează greutățile, iar prea multe stele trag centrul spre mijlocul cadranului și săgețile se scurtează, de unde capacitatea de mai jos. Un lucru nu-l dă: un centru de greutate în trei dimensiuni uită din ce mase a fost făcut; pe o mie de rotițe nu uită. Fiecare stea topită poate fi regăsită în el, și paragraful următor spune cum. Și nu ieșim din reprezentare: harta e descrisă de aceleași rotițe, doar că săgețile ei au și lungime. Dacă stelele sunt punctele cu toate săgețile de lungime 1, suprafața, harta e un punct din interior, cum centrul de greutate al unor puncte de pe un cerc cade în interiorul cercului. Direcția spune *unde*, lungimea spune *cât de sigur*, rotiță cu rotiță; iar tot ce știe sonda să facă unei stele, să o mute și să o compare, știe să facă și unei hărți.

Când cauți o stea în hartă, te uiți rotiță cu rotiță dacă săgeata hărții înclină spre săgeata stelei. Rotițele cu săgeți lungi cântăresc mai mult, cele cu săgeți scurte aproape deloc; cât de lungi sunt săgețile în total nu contează (o hartă cu toate săgețile dublate e aceeași hartă). Lungimile fiecărei rotițe în parte contează însă, și am măsurat-o: dacă forțezi harta înapoi

la săgeți de lungime 1, citirea pierde pe construcții noi și la adâncime, pentru că lungimile poartă numărul voturilor. O stea topită în hartă a împins fiecare săgeată puțin spre ea, și peste mii de rotițe împingerile astea mici se adună într-un acord clar; o stea care n-a fost topită se potrivește din întâmplare pe unele rotițe și nu se potrivește pe altele, exact ca în 2.2, și acordul total e zero. Numim ce rezultă *constelația* propoziției, iar harta, documentul care o ține, e o *hologramă*: toate stelele sunt în ea deodată, suprapuse. Îi poți pune întrebări („conții steaua lui Paul mutată cu cutare poziție?") și răspunde clar da sau nu, dar nu-ți poate *enumera* stelele: din suma săgeților nu recuperezi săgețile, poți doar să testezi candidați.

Două lucruri despre hartă trebuie reținute de la început.

Întâi: o stea desenată rămâne mutată cu poziția *din clipa aceea*. Sonda poate sări mai departe oriunde; steaua lui Paul de pe hartă nu se mai mișcă. Fiecare cuvânt poartă poziția sondei din momentul citirii lui, nu poziția de la sfârșitul propoziției.

Al doilea: harta are o capacitate. Cu cât topești mai multe stele, cu atât votul fiecăreia e o parte mai mică din săgeata fiecărei rotițe, și cu atât e mai greu de recunoscut. Cu o mie de rotițe și mai bine, câteva zeci de termeni stau curat într-o constelație (măsurat pe termeni aleatori; locul exact unde începe să cedeze e în fișierul de stare, iar constelațiile structurate ale propozițiilor, cu roluri și puțini candidați, țin mai mult); sutele se amestecă; mai departe, harta alunecă spre o direcție mijlocie care nu seamănă cu nimic. E singura limită dură a poveștii, și motivul pentru care textele lungi vor fi mai multe hărți legate între ele, nu una singură.

2.6 Legenda și domeniile

Acum putem spune concret ce e legenda, pe exemplul evidenței de împrumuturi. Lumea pune trei întrebări despre participanți: cine dă (agentul), ce (obiectul), cui (destinatarul), și una despre acțiunea însăși, pe care o lăsăm pentru capitolul 8. Cel care construiește lumea trage la sorți, o singură dată, câte o stea pentru fiecare întrebare: *steaua agentului*, *steaua obiectului*, *steaua destinatarului*. Trei puncte aleatoare din cer, fixe pentru totdeauna, fără nicio legătură între ele.

O stea cu nume definește un *domeniu* al cerului. Domeniul agentului e mulțimea tuturor stelelor pe care le obții mutând o stea de cuvânt cu steaua agentului folosită ca mutare (2.3): steaua lui Paul mutată cu steaua agentului, steaua lui Peter mutată cu steaua agentului, și așa mai departe, câte una pentru fiecare cuvânt din catalog. E, dacă vrei, o copie întreagă a catalogului, mutată într-o altă parte a cerului, cu forma păstrată. Domeniul obiectului e altă copie, mutată cu steaua obiectului; domeniul destinatarului, a treia. Pentru că stelele cu nume sunt aleatoare, cele trei domenii n-au nimic în comun și nu se confundă, cu aceeași certitudine practică din 2.2, deși fiecare conține toate cuvintele.

„Paul e în domeniul agentului" înseamnă deci exact: pe hartă e desenată steaua lui Paul mutată cu steaua agentului. Iar ca să fie așa, sonda trebuie să fi fost *la steaua agentului* când a fost citit „Paul": mutarea cu poziția sondei (2.5) și mutarea cu steaua agentului sunt atunci același lucru. Ca să verifici cine e în domeniul agentului, muți harta înapoi cu steaua agentului și te uiți ce stea din catalog a reapărut.

Același lucru pentru depozit, doar cu alte stele cu nume: a furnizorului, a mărfii, a cantității, a locului, a momentului. „Furnizorul Alfa a livrat 40 de cutii la raionul B marți” ar trebui să ajungă pe hartă cu „Alfa” în domeniul furnizorului, „cutii” în domeniul mărfii, „40” în domeniul cantității, „raionul B” în domeniul locului și „marți” în domeniul momentului. Dacă propoziția n-ar fi spus „marți”, domeniul momentului ar fi rămas gol pe hartă, și asta se vede, cum arată capitolul 7. Pentru pilot, „40” și „marți” sunt cuvinte ca oricare altele, cu stele trase la sorți; că 40 e aproape de 41 nu știe și nu trebuie să știe (9.1 spune cum sunt tratate numerele și datele înainte de citire).

Legenda e minusculă, stă la bază, și e singurul lucru pe care lumea și pilotul îl au în comun. Lumea o folosește ca să deseneze constelația corectă a unei propoziții („Paul în domeniul agentului, carte în domeniul obiectului, Anna în domeniul destinatarului”). Pilotul o folosește ca să-și citească propria hartă. Nimic altceva nu e comun: pilotul nu știe faptele lumii, iar lumea nu știe cum sare sonda.

2.7 Scanerul: cum se citește harta

Vrei să știi cine e în domeniul agentului. Pilotul mută întreaga hartă înapoi cu steaua agentului. Ce fusese în domeniul agentului se întoarce la steaua lui din catalog, recognoscibilă. Ce fusese în alte domenii e mutat acum cu un cod care nu era al lui și ajunge într-un punct fără legătură, adică zgomot. Rezultatul nu e un răspuns curat, ci o hartă zgomotoasă: steaua lui „Paul”, plus zgomot de la „carte” și „Anna”.

Harta aceasta intră în *scaner*. Scanerul o compară cu fiecare stea din catalog, rotiță cu rotiță, cum am spus în 2.5, și raportează pe cea mai apropiată, împreună cu două numere: cât de aproape e, și cu cât e în fața următoarei. Pentru că stelele cuvintelor sunt aleatoare și n-au legătură între ele, aproape întotdeauna una singură iese în evidență, iar restul stau la nivelul zgomotului: „Paul”. Când niciuna nu iese în evidență, sau două ies la fel, asta e o informație în sine, pe care o vom folosi.

(În exemplul cu patru rotițe scanerul n-ar funcționa: patru rotițe nu ajung ca zgomotul să se anuleze. Cu mii de rotițe, rata de eroare scade exponențial cu numărul lor și depinde de câte stele sunt topite în hartă și de mărimea catalogului; capitolul 8 spune unde apare limita. De asta sunt mii.)

Cu atât e descrisă povestea: un cer în care un punct e doar coordonate, o sondă care sare și raportează, și un pilot cu un catalog de stele ale cuvintelor, o hartă pe care desenează constelația propoziției, o legendă cu stelele cu nume ale lumii și un scaner. Tot ce urmează e construit doar din acestea.

3. Citirea unei propoziții

3.1 Pasul

Carnetul de rute al pilotului are câte o cheie pentru fiecare *fel* de cuvânt: un fel pentru toate numele de persoane, unul pentru toate obiectele, și câte un fel pentru fiecare cuvânt de legătură și fiecare verb al lumii („îi”, „de”, „la”, „că”, „dă”…), care sunt puține. Felul unui cuvânt nu-l stabilește pilotul: îl ia dintr-un dicționar al lumii (și, pentru un cuvânt nou, de la

un etichetator obișnuit); capitolul 13, pasul 3, dă lista. Cheile sunt ce a învățat pilotul, și singurele lucruri pe care le-a învățat. Legenda nu e în carnet; e folosită doar la citirea hărții, la sfârșit.

La fiecare cuvânt, pilotul face același lucru, identic:

1. Se uită ce fel de cuvânt e, ia din carnet cheia felului aceluia și o trimite sondei. Sonda sare și raportează poziția. Poziția e, în fiecare clipă, suma tuturor cheilor trimise de la începutul propoziției.
2. Ia steaua cuvântului din catalog, o mută cu poziția raportată și o topește în harta propoziției.

Atât. Harta de la sfârșit e *starea* propoziției: constelația ei. Nu păstrează lista cuvintelor, nici ordinea lor, nici vreun arbore gramatical. Doar stelele cuvintelor, fiecare mutată cu poziția la care era sonda când a fost citit cuvântul.

Toată partea învățată e carnetul: o cheie pe fel de cuvânt, câteva zeci cu totul (plus cheile pe perechi de feluri din 3.5, câteva mii). Stelele cuvintelor, adică grosul vocabularului, nu se ating niciodată. Iar scopul învățării, pe care îl detaliem în capitolul 6, e unul singur: ca salturile dictate de carnet să aducă sonda, în clipa în care un cuvânt e citit, exact la steaua cu nume pe care legenda o cere pentru cuvântul acela, adică să-l așeze în domeniul lui.

3.2 Ce înseamnă că două propoziții „ajung în același loc”

Ia „Paul îi dă Annei o carte”. Pilotul trimite cheia pentru nume, apoi cheia pentru „îi”, apoi cheia pentru „dă”, apoi iar cheia pentru nume, apoi pentru articol, apoi pentru substantiv. Pe drum a desenat „Paul” mutat cu poziția de după primul salt, „Annei” cu cea de după al patrulea, „carte” cu cea de după al șaselea. Fiecare stea de pe hartă poartă punctul în care era sonda în acel moment.

Acum „Anna primește o carte de la Paul”. Alte cuvinte, alte chei, alt traseu. Dar dacă carnetul e învățat bine, „Paul” e desenat după un șir de salturi (nume, primește, o, carte, de, la, nume) care aduce sonda exact la *steaua agentului*, adică în același punct în care era și când a citit „Paul” în prima propoziție. „Anna” ajunge în domeniul destinatarului, „carte” în domeniul obiectului.

Două propoziții înseamnă același lucru când desenează aceeași constelație: aceleași stele de cuvinte, în aceleași domenii. Traseul nu contează. Trei lucruri trebuie ținute separate aici. Hărțile, ca desene, nu sunt identice: cuvintele de legătură și drumurile diferite lasă urme diferite, și hărțile sunt doar *asemănătoare*; asemănarea asta e ce folosește antrenarea. Cadrele citite din ele, domeniu cu domeniu, sunt *egale*: aceleași stele în aceleași domenii. „Același sens” înseamnă al doilea lucru, și așa se măsoară: compari cele două hărți prin legendă, domeniu cu domeniu.

3.3 Citirea hărții: cadrul

Vrei să știi cine e agentul. Pilotul mută harta înapoi cu steaua agentului și dă rezultatul scanerului: „Paul”. Face asta pentru fiecare stea cu nume din legendă și are *cadrul*: agent

Paul, obiect carte, destinatar Anna. Cadrul nu e calculat din hartă printr-un alt program; el *este* harta, citită prin legendă.

Cu fiecare răspuns vin cele două numere ale scanerului: cât de bine se potrivește câștigătorul, și cât de departe în urmă e al doilea. Diferența e *marja*. Marjă mare: un singur cuvânt a fost găsit în domeniul acela. Marjă mică: au fost găsite două, sau unul a fost desenat de la un punct aflat între două stele cu nume, deci e pe jumătate în două domenii. Marja e locul unde trăiește „nu sunt sigur".

3.4 Propoziții în propoziții: legenda e relativă

„Maria spune că Paul îi dă Annei o carte." Evidența noastră are în legendă și întrebări pentru relatări: *cine spune?* (vorbitorul) și *ce spune?*. Dar ce spune Maria nu e un cuvânt, e o propoziție întreagă, cu agentul, obiectul și destinatarul ei. Dacă pilotul l-ar desena pe „Paul" în domeniul agentului, n-am mai ști dacă Paul e agentul din propoziția mare sau din cea relatată. Trebuie un set separat de domenii pentru propoziția din interior, care să aibă totuși *aceleași nume*.

Soluția e tot o stea. Legenda are, pe lângă steaua vorbitorului, o stea pentru rolul „ce spune": steaua *relatat*. Și aici e de văzut că stelele cu nume nu sunt, de fapt, puncte absolute, ci *deplasări față de originea propoziției*: „steaua agentului e la +cheia agentului față de origine". Pentru propoziția relatată, originea e alta: e punctul în care ajunge sonda sărind cu steaua relatat. Steaua agentului *a propoziției relatate* e „origine + relatat + agent": un punct nou, fără legătură cu steaua agentului din propoziția mare, și totuși cu același nume în legendă, citit de la altă origine. Toată constelația legendei se mută deodată, cu forma păstrată, pentru că mutarea păstrează forma.

Pilotul a învățat că felul de cuvânt „că" cere exact saltul relatat. De acolo, citește propoziția interioară cu aceleași chei din carnet ca de obicei. „Paul" e desenat în domeniul agentului de la originea relatată. Maria a rămas în domeniul vorbitorului, față de originea mare.

Ca să citească agentul interior, pilotul desface drumul: mută harta înapoi cu steaua relatat (se întoarce la originea propoziției mari), apoi cu steaua agentului, apoi scanerul: „Paul". Două propoziții una în alta („Maria spune că David crede că Paul…"): desface două salturi relatat. Adâncimea imbricării e, literal, numărul de origini mutate; îi spunem *nivel*.

Și de aici vine un lucru pe care l-am măsurat: pentru că „mută originea" e aceeași cheie de fiecare dată, un pilot antrenat pe propoziții cu cel mult două niveluri citește corect o propoziție cu trei, fără nicio învățare nouă. E același salt, făcut încă o dată. Antrenat până la două niveluri, a citit trei fără greșeală, în timp ce o rețea neuronală antrenată pe aceleași propoziții, cu aceeași intrare, a ghicit la întâmplare la nivelul trei. Nimic special n-a fost construit pentru asta: fiecare nivel e legenda întreagă, citită de la o altă origine, și originile se pot muta de oricâte ori, pentru că o cheie poate fi folosită de oricâte ori și desfăcută de tot atâtea.

3.5 Balizele de pe hartă: cum știe un cuvânt în ce domeniu să meargă

Pasul din 3.1, în care cheia unui cuvânt depinde doar de felul lui, are o slăbiciune pe care o vezi imediat. „Paul dă…" și „Paul primește…" încep cu același cuvânt, de același fel, deci cu

același salt. Dar Paul e agent în prima și destinatar în a doua. Când pilotul îl desenează pe Paul, după primul salt, nu are de unde să știe în ce domeniu trebuie să ajungă; ce urmează nu s-a citit încă.

Ce se întâmplă în practică: cheia învățată pentru nume ajunge la un compromis, un punct la jumătatea drumului între steaua agentului și steaua destinatarului. Propoziția se citește totuși corect, pentru că atunci când întrebi „cine e agentul?", Paul e *cam pe jumătate* acolo (cât anume depinde de cum se măsoară apropierea), iar nimeni altcineva nu e deloc. Dar marja e subțire, și cu câteva asemenea compromisuri într-o propoziție citirea se tulbură.

Versiunea completă a pilotului rezolvă asta cu două obiceiuri în plus, tot fără să știe nimic despre lume.

Primul: unele cuvinte pun balize pe hartă. Luăm verbul ca exemplu pentru că e cel mai ușor de urmărit; care cuvinte pun balize de fapt se învață, și rezultatul e spus câteva paragrafe mai jos. Când pilotul ajunge la „dă", notează pe hartă câte o *baliză* pentru fiecare fel de cuvânt care ar putea urma: „un substantiv va fi la poziția mea + cheia relativă (verb → substantiv)", „un nume la poziția mea + cheia relativă (verb → nume)". Balizele nu știu de roluri, pentru că pilotul nu vede legenda; cheile relative sunt pe *perechi de feluri* (felul cuvântului care pune baliza, felul celui care o ia), învățate și ținute în același carnet. Că baliza pentru substantiv ajunge să cadă pe steaua obiectului, iar cea pentru nume pe a destinatarului, e meritul învățării, nu o regulă. Un cuvânt care sosește mai târziu se uită dacă există o baliză pentru felul lui; dacă da, steaua lui e mutată cu *poziția balizei*, nu cu poziția raportată de sondă. Balizele sunt semne pe hartă, nu lumini aprinse în cer, și se șterg la sfârșitul propoziției.

Al doilea: un cuvânt sosit prea devreme e ținut deoparte. „Paul" de la începutul propoziției nu are încă nicio baliză care să-l aștepte. Pilotul nu-l desenează cu baliză încă; îl ține deoparte (celelalte două desene din 3.6 le face pe loc) și merge mai departe. Când ajunge la primul cuvânt care pune balize („îi" în carnetul învățat, „dă" în exemplul nostru simplificat) și apare o baliză pentru un nume, Paul de deoparte e primul care i se potrivește, și e desenat atunci, mutat cu poziția balizei (cu o cheie relativă proprie, pentru „așezat înapoi", alta decât a cuvintelor de după), adică în domeniul cuvenit. Așa primește primul cuvânt domeniul lui, deși rolul lui se vede abia mai târziu.

Ce feluri de cuvinte au voie să pună balize nu decidem noi (cu trei excepții declarate, la pasul 8 din capitolul 13). Fiecare fel de cuvânt are, în carnet, pe lângă cheie, o *poartă*: un număr între zero (nu pune niciodată) și unu (pune mereu), învățat din același obiectiv ca tot restul. Noi am fi pus pe unu verbele și prepozițiile. Pilotul a ales altfel, singur: cuvintele care marchează o relație („lui", „de", „la", „că", o virgulă) au porțile deschise; numele le au închise; verbele, surprinzător, tot închise. Explicația, odată văzută, e simplă: dacă cuvintele ținute deoparte pot fi așezate de ce urmează, prepoziția spune deja tot ce ar spune verbul. Aceeași alegere a reapărut, fără s-o cerem, când am antrenat pilotul pe întrebări reale tastate de oameni despre tabele. Structura „cine deschide un cadru" (cine pune balize, adică deschide așteptări pentru cuvintele de după el) e, în acest sistem, un rezultat, nu o intrare.

3.6 Trei desene pentru același cuvânt

Avem acum două feluri de a afla cu ce poziție se mută steaua unui cuvânt: poziția balizei (precisă, dar fragilă: dacă o construcție e nouă și o baliză lipsește, cuvântul rămâne deoparte sau e desenat greșit) și poziția raportată de sondă, adică suma cheilor generice ale cuvintelor de dinainte (robustă, dar neclară, cum am văzut cu Paul). Mai există un al treilea, mai grosolan: o poziție care depinde doar de *felul cuvântului anterior* („după o prepoziție, nume"), aproape niciodată greșită și aproape niciodată precisă.

Pentru că topirea în hartă e ieftină, pilotul nu alege între ele. Desenează steaua cuvântului de *trei ori*, mutată cu cele trei poziții, și topește toate cele trei copii în aceeași hartă. Când propoziția are o formă familiară, cele trei poziții sunt același punct și desenul e net. Când propoziția folosește o construcție nemaivăzută, una din poziții e aiurea, dar celelalte două tot aduc steaua destul de aproape de domeniul ei ca scanerul s-o recunoască. O cheie învățată greșit nu mai distruge citirea; doar o încețoșează. Pe construcții noi, redundanța asta a ridicat citirea corectă de la șase din zece la nouă din zece.

Cu asta, pilotul e complet: un carnet cu o cheie pe fel de cuvânt, chei relative pentru balize, o poartă pe fel, și pasul din 3.1, repetat identic la fiecare cuvânt, cu trei desene. Tot ce urmează e despre cum ajunge carnetul să fie bun, și ce poți face cu harta de la sfârșit.

4. De ce două propoziții de lungimi diferite pot însemna același lucru

„Paul îi dă Annei o carte" are șase cuvinte. „Anna primește o carte de la Paul" are șapte. „Cartea i-a fost dată Annei de către Paul" are opt. Un cititor care numără cuvinte, sau care ține minte ordinea lor, le vede ca trei lucruri diferite. Pilotul le vede ca unul singur. Capitolul acesta spune de ce, pas cu pas, pentru că aici se vede cel mai clar ce e și ce nu e harta.

Lungimea e o proprietate a drumului, nu a hărții

Sonda face câte un salt pe cuvânt: șase salturi pentru prima propoziție, șapte pentru a doua. Drumurile au lungimi diferite. Dar harta nu înregistrează drumul; înregistrează *unde era sonda când a fost citit fiecare cuvânt*. Pe hartă nu există „al treilea cuvânt" sau „după șase salturi". Există steaua lui Paul mutată cu steaua agentului, și atât. Dacă două drumuri de lungimi diferite aduc sonda la steaua agentului în clipa în care e citit „Paul", cele două hărți au exact aceeași stea în același loc, și nimic pe hartă nu mai spune cât de lung a fost drumul până acolo.

Asta nu e un accident, e o alegere. Niciuna dintre cele trei poziții cu care pilotul ștampilează un cuvânt (3.6) nu conține numărul de ordine al cuvântului în propoziție: poziția sondei e suma cheilor de dinainte, baliza e o deplasare față de cuvântul care a deschis cadrul, iar a treia depinde doar de felul cuvântului anterior. Am încercat la un moment dat și o poziție care conținea numărul de ordine; ea lega citirea de lungime și de ordine, adică exact de ce schimbă parafrazele. Am lăsat-o deoparte, apoi am remăsurat-o cu toate patru desenele topite: ajută o formulare nouă care pune un rol în locul unde îl puneau formulările de antrenare și strică una care nu — un pariu pe poziție, nu o înțelegere a drumului — și a rămas o opțiune declarată, în afara referinței (capitolul 15).

Cuvintele de legătură mută sonda, dar nu intră în cadru

„Îi”, „o”, „de”, „la”, „către” sunt și ele cuvinte, și pilotul le tratează ca pe oricare: fiecare are o cheie, sonda sare, steaua cuvântului e desenată pe hartă acolo unde a ajuns. Dar punctele în care ajung nu sunt stele cu nume. Când pilotul citește harta prin legendă, întreabă doar domeniile cu nume; steaua lui „de” stă într-un punct fără legătură cu oricare dintre ele și nu răspunde la nicio întrebare (poate ajunge, în schimb, să poarte markerul de ocupare al unui domeniu, capitolul 8). Cuvintele de legătură sunt schelăria drumului: fără ele sonda n-ar ajunge unde trebuie, dar odată ajunsă, ele nu mai contează. O propoziție cu mai multe cuvinte de legătură are mai multă schelărie pe hartă, în puncte fără nume, și același cadru.

De aceea și comparăm hărțile *domeniu cu domeniu*, nu ca pe două desene suprapuse. Două hărți ale aceluiași eveniment nu sunt identice punct cu punct, pentru că schelăria diferă; sunt identice acolo unde contează, în răspunsurile la întrebările legendei.

Cuvinte diferite, aceeași destinație

Rămâne întrebarea grea: de ce „dă” și „primește … de la” aduc sonda în aceleași stele cu nume, când sunt cuvinte diferite, cu chei diferite? Pentru că așa a fost învățat carnetul. La antrenare, cele două propoziții au primit aceeași constelație a lumii (6.1), iar singura libertate a pilotului era carnetul. Ca să satisfacă amândouă, a trebuit să găsească chei sub care „nume, dă, nume” și „nume, primește, …, de, la, nume” aduc numele la aceleași stele. Balizele fac asta posibil fără contorsiuni: în exemplul nostru „dă” pune balize pentru obiect și destinatar față de locul lui, iar „la” din „de la” pune o baliză pentru agent față de locul lui. (În carnetul învățat, balizele le pun mai ales cuvintele de relație, „îi”, „la”, nu verbul; unde verbul tace, celelalte două desene din 3.6 aduc cuvântul la locul lui.) Cuvântul care urmează ia baliza, oriunde ar fi în propoziție. Lungimea dintre „la” și „Paul” nu contează, pentru că baliza e o destinație, nu un număr de pași.

Verbele diferite, „dă” și „primește”, ajung și ele pe hartă, în domeniul predicatului (un domeniu al legendei; pasul 4 din capitolul 13 îl scrie ca R[acțiunea]). Dacă ele contează ca același eveniment sau ca două evenimente diferite nu decide pilotul, ci legenda lumii, care spune ce verbe descriu același eveniment (8, „Care verb?”). Pilotul doar le desenează consecvent.

Un exemplu urmărit, și unul care nu se potrivește

„Paul îi dă Annei o carte”: sonda pleacă din origine; la „Paul” pilotul îl ține deoparte (nicio baliză încă); „îi” (și „dă”, în exemplul simplificat) pun balize; Paul ia baliza care duce la agent, „Annei” pe cea spre destinatar, „carte” pe cea spre obiect; „o” rămâne în schelărie. Harta: Paul în domeniul agentului, carte în al obiectului, Anna în al destinatarului, „dă” în al predicatului.

„Anna primește o carte de la Paul”: „Anna” ținută deoparte; „primește” (în exemplul nostru) deschide balize; Anna ia baliza destinatarului (pentru „primește”, cine primește e destinatarul, și asta e tot în carnet); „carte” ia obiectul; „la” deschide baliza agentului; „Paul” o ia. Harta: Paul în domeniul agentului, carte în al obiectului, Anna în al destinatarului,

„primește” în al predicatului. Șapte cuvinte, același cadru, cu excepția predicatului, pe care legenda îl declară același eveniment.

Acum „Anna primește marți o carte de la Paul”. Opt cuvinte, și o stea în plus: „marți” în domeniul momentului. Cele două hărți *nu* mai spun același lucru, și diferența e localizată exact: același răspuns la cine, ce, cui, un răspuns în plus la când. E al treilea fel de „aproape” din capitolul 8, și e tot ce înseamnă „mai lung” când lungimea aduce informație: un domeniu în plus umplut, nu o hartă mai mare.

Când lungimea chiar contează, și ce face harta atunci

Două cuvinte în loc de unul într-un domeniu („Paul și Peter”): domeniul e aglomerat, cu două stele, și asta e sensul corect, nu o eroare. Un cuvânt lipsă („Paul dă o carte”, fără destinatar): un domeniu rămâne gol, și asta e, iar, sensul corect, cu golul la vedere. O repetiție de întărire („foarte, foarte târziu”) desenează aceeași stea de două ori în același loc, ceea ce o face puțin mai puternică și nu schimbă cadrul; dacă o lume vrea să deosebească „târziu” de „foarte târziu”, trebuie să aibă un domeniu pentru asta. Și limita cunoscută: când forma propoziției se dezvăluie abia la sfârșit („…, spune Maria”), lungimea de dinainte a fost deja desenată față de originea greșită; un cititor care merge înainte nu poate repara asta, cum spune capitolul 8.

Pe lumile construite, formulări diferite ale aceluiași eveniment, cu numere diferite de cuvinte și ordine diferite, au ajuns la același cadru, domeniu cu domeniu; e rezultatul din care a pornit tot documentul. Pe text real, semnalul vine de la întrebările despre tabele, unde întrebări de lungimi diferite au dus la aceeași coloană în opt cazuri din zece.

5. Același cuvânt în mai multe lumi: polisemie, lumile cunoașterii, ontologii

Un cuvânt ca „bancă” înseamnă altceva într-un text despre finanțe și altceva într-unul despre un râu. „Energie” înseamnă altceva în fizică și în nutriție. „Câmp” înseamnă altceva în algebră, în agricultură și în fizică. Un cititor care are o singură stea pentru fiecare cuvânt pare să nu poată face față. Capitolul acesta spune cum stau lucrurile de fapt, și ce *nu* pretindem.

O precizare de vocabular înainte: în această carte, *domeniu* înseamnă regiunea unui rol (domeniul agentului, domeniul cantității). Pentru un domeniu de cunoaștere sau de activitate, finanțe, fizică, un depozit, spunem *lume*, pentru că așa am numit-o în capitolul 1: lucrul despre care vorbesc propozițiile, cu lista lui de întrebări.

Steaua e cuvântul, nu sensul

Steaua lui „bancă” e una singură, fixă, aceeași oriunde. Asta nu e o slăbiciune; e consecința definiției din 1.1. Sensul unei propoziții e „la ce întrebări răspunde, și cu ce”. Sensul unui *cuvânt*, în aceeași definiție, e „la ce întrebări răspunde, în lumea în care e citit”. În lumea finanțelor, „bancă” răspunde la întrebări ca „cine a acordat creditul?”; în lumea geografiei, la „unde s-a oprit barca?”. Aceeași stea, legende diferite, domenii diferite. Pilotul nu trebuie să decidă niciodată *ce fel de bancă* e; trebuie doar s-o așeze în domeniul potrivit al lumii în care lucrează. Ce e o bancă, în acea lume, știe baza de date, și e treaba ei.

Spus altfel: polisemia nu trăiește în stele, trăiește în legende. Două sensuri ale aceluiași cuvânt sunt două legende în care el răspunde la întrebări diferite. Asta e și motivul pentru care stelele pot rămâne aleatoare și fixe (capitolul 8): nu au nimic de învățat despre sens, pentru că sensul e în altă parte.

Ce nu rezolvă legenda singură e sensul dublu în *aceeași* lume și *același* domeniu: „pe bancă", ca loc, poate fi banca din parc sau clădirea băncii, și dacă steaua și domeniul sunt aceleași, harta nu le deosebește. Acolo dezambiguizarea rămâne a contextului, a unui identificator de sens în legendă sau a bazei de date. Legenda îngustează ambiguitatea; nu o elimină.

Când e ambiguu felul cuvântului

Există o ambiguitate care îl privește direct pe pilot: când același cuvânt poate fi de două *feluri*, cu două chei diferite. „Car" e substantiv sau verb; „cer", la fel. Felul decide saltul, deci aici pilotul chiar trebuie să aleagă. Am măsurat două căi. Prima: pilotul ia felul cel mai frecvent al cuvântului și merge mai departe. Funcționează aproape perfect, dintr-un motiv care merită spus: locul unui cuvânt pe hartă e decis mai ales de balizele cuvintelor din jurul lui (prepoziția dinainte, verbul care urmează), nu de propria lui cheie; „cer" după „pe" ajunge în domeniul obiectului oricum, pentru că „pe" l-a așezat acolo. A doua cale, desenul la jumătatea drumului între cele două feluri până se lămurește, costă pe substantive și n-o folosim. Ce rămâne pentru cazurile cu adevărat ambigue e scanerul: la citire încearcă ambele feluri și raportează marja dintre ele; când e mică, harta spune, ca pentru orice domeniu aglomerat, „nu știu dacă e lucru sau acțiune". Ambiguitatea lexicală nu cere nimic nou.

Lumea e un obiect de prim rang

Întrebarea firească: dacă „bancă" înseamnă altceva în două lumi, n-ar trebui ca și *saltul* pe care îl face sonda la „bancă" să fie altul în fiecare lume? Da, și așa și este; merită spus limpede ce anume aparține lumii și ce nu.

Singurul lucru pe care îl împart toate lumile e cerul, cu stelele cuvintelor: steaua lui „bancă" e una, pentru că e identitatea cuvântului, nu a sensului. Tot restul e al lumii. O lume are *legenda* ei (ce întrebări pune, cu stelele lor cu nume); *felurile* ei de cuvinte (în lumea finanțelor „bancă" e de felul instituțiilor, în geografie de felul locurilor, iar felul decide cheia); *carnetul* ei de rute (cheile se învață pe propozițiile acelei lumi, deci saltul la „bancă" e altul în fiecare lume, pentru că felul e altul și carnetul e altul); *fondul* ei pentru rest (restul e ce rămâne neexplicat pe hartă după citire, definit în capitolul 8; fondul, cât rest lasă o propoziție înțeleasă acolo); și *baza* ei de date, unde stau faptele. Pilotul ține câte un carnet pentru fiecare lume în care lucrează. Lumea e, deci, un obiect de prim rang în poveste, nu un context vag: are nume, are piese, se antrenează separat și se poate schimba fără să atingă celelalte lumi. Când în limbaj obișnuit spunem „domeniu științific" sau „domeniu de antrenare", în poveste spunem *lume*, și ea e cetățean cu drepturi depline.

Există și o a doua cale, pe care n-am construit-o: lumea însăși ca stea. Așa cum cheia „relatat" mută toată legenda la altă origine, o stea a lumii ar putea muta legenda finanțelor într-o parte a cerului și pe a geografiei în alta, cu un singur pilot și un singur carnet. Ar fi

mai economic, dar ar cere ca felurile cuvintelor să fie împărțite între lumi, și tocmai acolo e polisemia; ar fi nevoie atunci de suprapunerea felurilor din secțiunea anterioară. E o direcție de cercetare cu un compromis clar: un carnet pe lume e simplu și izolează lumile; o stea a lumii e economică și le leagă.

Mai multe lumi deodată

Un sistem instalat într-o întreprindere are de obicei mai multe lumi: facturi, livrări, concedii. Fiecare are legenda și carnetul ei, cum am spus. O propoziție nouă e citită cu fiecare carnet deodată (un „pilot" pe lume, dacă vrei); costă microsecunde fiecare. Apoi se compară hărțile: la pilotul lumii potrivite, domeniile se umplu cu marjă clară și reziduul (restul, capitolul 8) e mic; la ceilalți, stelele cad în afara legendei și reziduul e mare. Lumea unei propoziții se recunoaște după cât de bine se potrivește legenda ei, ca o cheie într-o broască. Nu e nevoie de un clasificator separat de „despre ce e textul": potrivirea e clasificatorul, și vine cu aceleași numere, marjă și reziduu, ca orice altă citire.

Două limite, spuse deschis. Când două lumi au vocabular comun și întrebări asemănătoare, potrivirea e mai slabă, și cât de slabă se măsoară, pe lumi construite cu suprapunere controlată. Și când o singură propoziție amestecă lumi, cum face literatura, metafora, gluma, pilotul n-are ce potrivi: raportează reziduu mare în toate legendele, adică „nu e a niciunei lumi pe care o cunosc". Nu e o promisiune pe care o facem; literatura nu e ținta.

Descoperim ontologii?

Nu, și e important să nu lăsăm impresia asta. O ontologie, în sensul folosit în informatică, e o listă de tipuri de lucruri și de relații între ele, cu nume. Legenda unei lumi *este* o ontologie mică, cu câteva zeci de rânduri, scrisă (de oameni, sau propusă de un model și aprobată de oameni; 9.3), nu descoperită din date. Felurile de cuvinte vin dintr-un dicționar (și dintr-un etichetator, pentru cuvintele noi), nu din antrenare; am verificat că o grupare statistică a corpusului dă aceleași feluri, și nu le revendicăm. Ce emerge cu adevărat din antrenare e un singur lucru: care feluri de cuvinte deschid cadre (porțile), și asta e mai puțin decât o ontologie.

Ce facem, în schimb, e să *populăm* o ontologie dată: citirea unui corpus întreg produce o listă de cadre, adică fapte cu roluri, adică un graf de cunoștințe relativ la legendă: cine a livrat ce, cui, când, de câte ori. Toate stelele care au ajuns vreodată în domeniul furnizorului sunt, luate împreună, vocabularul acelei lumi pentru „furnizor"; nimeni nu l-a scris, a rezultat din citire. E un inventar, nu o teorie, dar e un inventar verificabil propoziție cu propoziție.

Și mai e un lucru, care ține de ontologie fără să fie descoperirea ei: harta poate semnala că *lipsește o întrebare*. Dacă propozițiile unei lumi spun sistematic ceva la care legenda nu întreabă (toate livrările au o oră, dar legenda n-are „când"), stelele acelea cad mereu în afara legendei, și cad în același fel: reziduul nu e zgomot, are o direcție. Semnalul e „aici se spune ceva ce eu nu întreb", iar răspunsul e o întrebare nouă în legendă, adăugată de un om și învățată apoi de pilot cu carnetul lui. Asta e bucla din 9.4, „o graniță care învață", spusă concret: ontologia nu e descoperită de sistem, e *întreținută* de oameni cu semnalul sistemului.

Aceleași cuvinte, teorii diferite

Rămâne cazul cel mai interesant din întrebarea de la care am pornit: același cuvânt în două teorii, „masă” la Newton și în relativitate, „energie” în termodinamică și în nutriție. În termenii noștri, două teorii sunt două legende: aceleași cuvinte răspund la întrebări diferite și stau în relații diferite. Un text citit cu amândouă legendele dă două hărți, și diferența dintre ele, domeniu cu domeniu, spune exact unde cele două teorii folosesc același cuvânt altfel. E o direcție, nu un instrument: cere legende scrise pentru teorii, ceea ce nimeni n-a făcut încă, și texte care să le umple. Dar e același mecanism ca diferența între două versiuni ale unui regulament din 11, aplicat la ceva mai greu.

6. Cum învață pilotul

6.1 Ce primește: constelația lumii

Nimeni nu îi spune pilotului care propoziții sunt parafraze. Dar nici „fără nicio supraveghere” nu e, și asta trebuie spus limpede. Fiecare propoziție de antrenare vine cu o *constelație a lumii*: lumea, care știe evenimentul și are legenda, desenează ea însăși harta corectă: „steaua lui Paul mutată cu steaua agentului, a cărții cu steaua obiectului, a Annei cu steaua destinatarului, topite”. În unele dintre lumile pe care le-am construit, constelația lumii descrie evenimentul; în altele, consecința lui („Anna are cartea”). În toate, legenda (rolurile) e fixată dinainte de cel care a construit lumea; ce învață pilotul e *cum să ajungă* sonda la stelele ei cu nume, cuvânt cu cuvânt.

Antrenarea e atunci un singur joc, repetat: pilotul citește propoziția cu carnetul lui, își desenează harta, și harta lui e comparată, rotiță cu rotiță, cu constelația lumii. Cu cât se potrivesc mai bine, cu atât mai bine; cheile din carnet se ajustează puțin, în direcția care ar fi apropiat cele două hărți. Atât. Supervizarea e, deci, pe propoziție și structurală (o constelație cu roluri date dinainte), nu o listă cu ce propoziții înseamnă același lucru.

6.2 De ce asta forțează sensul

Două propoziții despre același eveniment primesc aceeași constelație a lumii. Deci sunt obligate să deseneze aceeași hartă. Singura libertate a pilotului e carnetul: o cheie pe fel de cuvânt și câte una pe pereche de feluri, câteva mii de chei cu totul, niciuna pe cuvânt. Cu câteva mii de chei pe feluri, memorarea propozițiilor una câte una e exclusă practic; ce poate găsi sunt regularități, și tocmai asta vrem. Deci singurul mod în care poate satisface amândouă cerințele e să găsească chei sub care ambele trasee aduc sonda la aceleași stele cu nume. Echivalența nu e etichetată pe perechi; e indusă de țintele comune și e soluția unei constrângeri.

Am încercat alternativele evidente și au eșuat instructiv. Dacă îi ceri pilotului să prezică *următorul cuvânt*, ca un model de limbaj, toate traseele devin la fel și domeniile dispar: nimic nu-l obligă să separe rolurile. Dacă îi dai o memorie care *caută* constelația lumii pornind de la harta lui, prezice perfect și nu aliniază nimic: o căutare poate duce trei hărți diferite la același răspuns fără să le facă vreodată aceeași hartă. Doar când pilotul trebuie să *producă* constelația lumii printr-o operație fixă și rigidă (harta lui, comparată direct) e forțat să facă hărțile să coincidă. Rigiditatea e esența.

6.3 Ce se învață, și cât de puțin

Câteva zeci de chei (una pe fel de cuvânt), câteva mii de chei relative pentru balize și pentru perechile de feluri vecine, și o poartă pe fel. Totul e de ordinul a zece–cincisprezece megaocteți la mărimile pe care le folosim, cât o fotografie de telefon; cea mai mare parte sunt cheile pe perechi de feluri. Stelele cuvintelor, grosul vocabularului, sunt aleatoare și fixe. Stelele cu nume din legendă sunt aleatoare și fixe. Toată partea antrenabilă e carnetul de rute.

De aceea antrenarea durează minute, de aceea un cuvânt nou nu costă nimic, și de aceea pilotul nu știe nimic despre lume: n-are unde să țină fapte. Faptele trăiesc în lume, care într-un sistem instalat e o bază de date sau un motor de raționament, unde pot fi inspectate și corectate. Pilotul ține doar cum așază limba cuvintele în domenii.

6.4 De unde vin constelațiile corecte

Secțiunea 6.1 a spus că fiecare propoziție de antrenare vine cu constelația ei corectă. Rămâne întrebarea practică: cine o scrie? Pilotul nu învață din text; învață din text *împreună cu* răspunsurile lui la întrebările lumii. Răspunsurile acelea au trei surse, și e bine să fie numite, pentru că fiecare are alt preț.

Prima e lumea construită: în experimentele noastre, lumea alege întâi evenimentul (Paul, cartea, Anna) și abia apoi produce propoziția, din șabloane. Răspunsurile sunt cunoscute înainte de propoziție, nu extrase din ea. Așa am măsurat mecanismele; așa nu se antrenează un sistem real.

A doua sunt datele structurate care există deja lângă text: o factură și descrierea ei, un bilet de suport și câmpurile lui, un mesaj de jurnal și înregistrarea din baza de date, o întrebare și interogarea care i-a răspuns. Acolo răspunsurile sunt gratuite și exacte, și nu e nevoie de nimeni care să le scrie; setul real pe care l-am folosit e de felul acesta.

A treia, pentru un domeniu în care textul vine singur, e un model mare pus pe post de profesor: citește fiecare propoziție dintr-un eșantion și scrie răspunsurile la întrebările legendei, în formatul în care le-ar citi pilotul. Trei lucruri trebuie spuse despre acest profesor. Lucrează doar la antrenare; la citire pilotul n-are nevoie de niciun model mare (dacă o arhitectură folosește unul pe cererile nesigure, ca în capitolul 11, rezultatul lui e recitit de pilot). Erorile lui sunt plafonul pilotului, în două feluri: pilotul nu poate învăța ce profesorul a greșit, iar noi măsurăm acordul cu profesorul, deci greșelile lui ne apar drept greșeli ale pilotului — de aceea pilotul antrenat recitește propozițiile de antrenare și, acolo unde contrazice cu marjă mare ce a scris profesorul, cazul e revăzut, iar rata acestor dezacorduri e raportată ca estimare a zgomotului de etichetare. Și lucrează pe un eșantion, nu pe tot ce va citi pilotul vreodată: pilotul generalizează la formulări pe care nu le-a văzut și la cuvinte noi fără reantrenare, iar cât de bine o face pe formulări nevăzute e numărul care spune cât de mare trebuie să fie eșantionul.

Ce nu face metoda, și nici nu pretinde: să învețe sensul din text fără ca nimeni să fi răspuns la nicio întrebare. Teza ei e că sensul *este* răspunsul la întrebări date; cineva trebuie să răspundă la primele.

7. Ce rămâne pe hartă după o propoziție, și la ce e bun

După ce o propoziție a fost citită, rămâne o singură hartă: constelația ei. Pe ea, recuperabile dar nu listate, sunt stelele cuvintelor, fiecare mutată cu poziția în care a fost găsită. Lângă ea e legenda: stelele cu nume. Și între hartă și legendă sunt niște distanțe care pot fi citite *fără să înțelegi nimic*: cât din constelație e explicat de domenii, cât de net e ocupat fiecare domeniu, dacă vreun domeniu pe care lumea îl vrea plin pare gol.

Asta e proprietatea pe care se sprijină tot restul. Harta nu e măsurată față de ea însăși; e măsurată față de legendă. Așa că pilotul poate spune, fără să înțeleagă o propoziție, *că* n-a înțeles-o, și în ce fel:

- *Nimic recunoscut într-un domeniu.* Muți harta înapoi cu steaua destinatarului și nicio stea din catalog nu iese în evidență. Fie propoziția n-a spus cine e destinatarul, fie l-a spus cu un cuvânt necunoscut; ca să le deosebești e nevoie de un mic dispozitiv în plus, descris în capitolul 8. Oricum: *destinatar nerecunoscut*.
- *Un domeniu aglomerat, sau ocupat pe jumătate.* Doi candidați, marjă mică. Ambiguitate, și pilotul îi poate numi pe amândoi.
- *Stele rămase neexplicate.* Iei fiecare rol recunoscut, reconstruiești constelația pe care ar produce-o aceste recunoașteri și o scazi din harta reală. Ce rămâne sunt stele pe care legenda n-a putut să le așeze în niciun domeniu: cuvinte citite, salturi făcute, desene puse, dar nu în domenii cu nume. Propoziția e despre ceva din afara lumii, sau e incoerentă, sau folosește o construcție pentru care carnetul n-are chei. Măsurabil ca mărimea acelui rest.
- *O cheie care n-a putut fi folosită.* Un cuvânt fără fel (o stea nouă primește orice cuvânt; o cheie, doar cuvintele cu un fel); traseul se rupe acolo, și fiecare desen de după poartă o poziție atinsă cu o cheie ghicită. Vizibil ca o prăbușire a marjei pentru toate rolurile de după cuvântul acela.

Două precauții, pentru că se uită ușor. O citire netă nu e o citire corectă: un cuvânt poate fi desenat cu încredere în domeniul greșit, și cât de des se întâmplă asta trebuie măsurat, nu presupus. Iar „nerecunoscut” e o afirmație despre legenda și catalogul pilotului, nu despre propoziție.

Un model de limbaj n-are legendă. Starea lui e un punct fără nimic față de care să fie măsurat, și răspunde cu aceeași fluență fie că traseul lui a trecut prin domenii cu nume, fie prin gol. Al nostru raportează unde a ieșit traseul din legendă. Fiecare aplicație de mai jos e această unică proprietate, în alt cadru.

8. Fenomene pe care le poți urmări pe hartă

Tot ce e în acest capitol se întâmplă în interiorul poveștii descrise și se poate măsura acolo. Nimic nu e o regulă scrisă de noi. Mergem încet, pentru că acestea sunt fenomenele care construiesc intuiția, și pentru că fiecare e și un diagnostic care poate fi arătat unui utilizator.

Domeniul gol, și de ce „gol” e mai greu decât pare

Muți harta înapoi cu steaua destinatarului și compari rezultatul cu fiecare stea din catalog. Fie una iese în evidență cu marjă clară (*destinatar: Anna*), fie niciuna.

Aici e o capcană în care e ușor să cazi. Ai putea spera să deosebești „nimeni n-a fost găsit în domeniul ăsta” de „a fost găsit cineva pe care nu-l cunosc” după cât de *multă lumină* se întoarce. Nu se poate, și merită spus de ce. Mutarea e rigidă: mută întreaga hartă, fiecare stea din fiecare domeniu, deodată. Ce se întoarce are întotdeauna „greutatea” întregii constelații, cu stelele celorlalte domenii împrăștiate ca zgomot. Cantitatea nu îți spune nimic; doar recunoașterea spune. „Paul dă o carte” și „Paul îi dă o carte lui Zxyq” arată la fel din domeniul destinatarului: nicio stea cunoscută nu iese în evidență.

Ca să le deosebești, e nevoie de un dispozitiv mic și explicit, standard în această familie de sisteme: un *marker* fix, același mereu, care înseamnă „aici a fost desenat ceva”. Lumea îl topește în constelația ei lângă fiecare răspuns; pilotul nu-l desenează el, dar, ca harta lui să semene cu a lumii, învață să-l pună din cuvintele de legătură: „îi” apare ori de câte ori există un destinatar, și steaua lui ajunge, prin carnet, exact unde lumea pune markerul destinatarului (pasul 4 din capitolul 13 arată calculul). Atunci domeniul destinatarului poate fi întrebat două lucruri separate: e markerul aici (s-a desenat ceva), și ce stea e aici (și o cunosc). „Paul dă o carte”: fără marker, nimic desenat, *destinatar nedat*. „Paul îi dă o carte lui Zxyq”: marker prezent, stea care nu e în catalogul lumii, *destinatar prezent, necunoscut*. Dispozitivul e ieftin; fără el, citirea e una singură: *destinatar nerecunoscut*.

Domeniul aglomerat

Uneori două stele se întorc din același domeniu cu putere asemănătoare. Sunt două feluri în care se întâmplă, și arată la fel din afară, dar nu din interior.

Unu: propoziția chiar a pus două lucruri acolo. „Paul și Peter îi dau Annei o carte.” Domeniul agentului are pe hartă două stele topite, și amândouă se întorc cam pe jumătate. Nu e confuzie; e o mulțime, corect desenată, și pilotul trebuie să le raporteze pe amândouă. Harta singură nu deosebește însă „Paul și Peter” de „Paul sau Peter, nu știm care”: amândouă ar fi două stele în același domeniu. Deosebirea cere un marcaj în legendă, unul pentru „și”, altul pentru „una din două”, la fel cum markerul de ocupare deosebește golul de necunoscut.

Doi: un singur cuvânt a fost desenat de la un punct aflat *între* două stele cu nume. E soarta primului cuvânt la pilotul simplu din 3.1: „Paul” e citit înaintea verbului, nimeni nu știe încă dacă dă sau primește, și poziția sondei e la jumătatea drumului dintre steaua agentului și cea a destinatarului. Întrebi domeniul agentului și Paul se întoarce pe jumătate; întrebi domeniul destinatarului și tot pe jumătate. Citirea merge, pentru că nimeni altcineva nu e în niciunul din cele două, dar marja e subțire.

Balizele repară al doilea caz și îl lasă în pace pe primul. Când sosește cuvântul care pune balize, Paul, ținut deoparte, e desenat de la poziția balizei cuvenite și se întoarce cu putere aproape întreagă (desenul lui de la poziția sondei rămâne la adresa de compromis).

Înainte-și-după se vede ca un salt de marjă; în măsurătorile noastre, cam de trei ori. Domeniul aglomerat de primul fel rămâne aglomerat, pentru că așa trebuie.

Stele în afara legendei, sau cum știe pilotul că o propoziție n-are sens pentru el

Citești fiecare domeniu, iei stelele recunoscute, și reconstruiești constelația pe care ar produce-o: fiecare stea mutată cu steaua cu nume a domeniului ei, toate topite. Scazi constelația reconstruită din harta reală. Ce rămâne e partea din propoziție pe care legenda n-a putut-o explica. Îi spunem *rest* (mai jos și *reziduu*: același lucru), și merită o explicație fără mister, pentru că e piesa pe care se sprijină cele mai multe diagnostice.

Restul nu apare la citire. Citirea unui domeniu, mutarea hărții înapoi și scanerul, e o operație exactă: mutarea e o izometrie, totul se mută intact, nimic nu „rămâne" în urmă. De aceea nici nu poți afla din *cât* se întoarce dacă domeniul e ocupat; se întoarce mereu întreaga hartă, mutată. Restul apare la *comparație*: după ce ai citit toate domeniile, ai cadrul (agent Paul, obiect carte, destinatar Anna); desenezi din nou, de la zero, harta pe care ar produce-o exact aceste răspunsuri; și o scazi din harta reală. E o descompunere, ca în statistică: harta = (ce explică legenda) + (rest). Cadrul e partea explicată, restul e ce rămâne. Partea explicată spune ce s-a înțeles; restul spune cât din propoziție n-a putut fi așezat în niciun domeniu cu nume.

Trei precizări îl fac cinstit. Întâi, restul nu e niciodată zero: cuvintele de legătură desenate la poziții fără nume, copiile în plus din cele trei desene și zgomotul suprapunerii lasă un *fond* care nu stă în niciun domeniu; fondul se măsoară o dată, pe lume, pe propoziții înțelese, și ce contează e *cât depășește* restul acest fond. Al doilea, restul se calculează după ce constelația reconstruită e adusă la scara hărții: hărțile se compară pe direcții, nu pe lungimi, deci o hartă de trei ori mai „lungă" nu e de trei ori mai neînțeleasă. Și o a treia, măsurată: *mărimea* restului nu e un detector sigur pentru o întrebare care lipsește din legendă. Am scos un rol din legendă și am antrenat fără el; propozițiile care îl exprimau n-au avut un rest mai mare, ci mai mic, pentru că fondul e dominat de cuvintele de legătură, iar o propoziție mai lungă are o hartă mai „lungă". Ce a dat în vileag rolul lipsă a fost *direcția* restului: resturile propozițiilor cu același răspuns ascuns arată în aceeași parte, și se grupează după el fără niciun vector și fără nicio etichetă. Când spunem mai jos „restul crește", vorbim de propoziții pe care legenda nu le explică deloc; pentru o întrebare lipsă, detectorul e direcția. Cu aceste precizări: pentru o propoziție înțeleasă restul rămâne la fond; pentru una neînțeleasă crește, și de unde vine depinde de motiv:

- *Cuvinte cunoscute, într-o ordine neparcursă.* Fiecare cuvânt și-a folosit cheia, dar șirul de salturi a dus sonda în puncte pe care legenda nu le numește. Stelele sunt reale, ar fi recunoscute dacă ai ști cu ce să muți harta înapoi, dar nicio stea cu nume nu e cea potrivită. E forma unei construcții noi: piesele sunt familiare, traseul nu.
- *Cuvinte necunoscute.* Fiecare e o stea nouă, aleatoare, mutată cu poziția unde s-a nimerit sonda. Stele aleatoare în puncte arbitrare: adaugă greutate hărții și nu contribuie cu nimic la niciun domeniu. E forma nonsensului, sau a unui vocabular străin.
- *O propoziție despre altceva.* Cuvintele sunt cunoscute, construcția obișnuită, și legenda pur și simplu n-are domenii pentru acest fel de eveniment, pentru că lumea

pe care a fost antrenat pilotul nu-l conține. Stelele ajung în puncte perfect bine formate și perfect fără nume. E forma unei întrebări din afara lumii.

Observă ce e și ce nu e această măsurătoare. Nu e „propoziția n-are sens”. E „cheile mele nu duc propoziția asta în legenda mea”. O propoziție poate fi plină de sens pentru un om și totuși să cadă în afara legendei unui pilot antrenat pe facturi. Pilotul cunoaște limitele propriei legende, și tocmai felul acesta de ignoranță e util.

Cheia ruptă

Un cuvânt necunoscut nu doar că nu e recunoscut; el nici n-are cheie, pentru că n-are fel. Pilotul, ajuns la el, trimite o cheie ghicită, și tot ce citește după e desenat de la poziții care poartă acea cheie greșită. Desenele dinaintea cuvântului necunoscut sunt bune; cele de după sunt deplasate cu o cantitate necunoscută. Ceri rolurile și vezi tiparul: cele umplute înainte de ruptură se întorc nete, cele de după, la nivelul zgomotului. Ruptura e localizabilă: pilotul poate spune *am pierdut firul la cuvântul șase*. Cele trei desene îndulcesc asta, pentru că nu se rup toate deodată; dar ceva de după ruptură e întotdeauna încețoșat.

Zgomotul, și de ce harta are o capacitate

Fiecare stea topită adaugă puțin zgomot în fiecare domeniu. Când muți harta înapoi cu steaua agentului, aduci înapoi, slab, și tot ce era desenat în celelalte domenii, mutat cu un cod care nu era al lui și deci împrăștiat aleator. Cu puține stele, împrăștierea e mult sub semnal. Cu multe, se adună, și la un moment dat cuvântul din domeniul agentului nu mai e clar deasupra împrăștierii din tot restul.

Cât înseamnă „multe” depinde de câte rotițe au coordonatele. Cu o mie și ceva, zeci de stele sunt curate, o sută-două încă lizibile cu grijă, iar dincolo harta alunecă spre mijloc, unde nimic nu mai e recognoscibil. Nu e un defect; e geometria. De aceea un document nu e o hartă, ci mai multe hărți legate prin chei, fiecare ținând ce a desenat o propoziție sau un eveniment. Saltul „relatat” e deja o astfel de legătură: propoziția interioară e constelația ei proprie, mutată întreagă la altă origine, în harta celei exterioare.

Negația ca o stea, și unde trăiește scopul ei

„Paul nu îi dă Annei cartea.” Legenda dă o stea pentru negație. Întrebarea e ce poziții o poartă. Dacă toate desenele evenimentului poartă steaua negației, negația e a propoziției: nu e cazul că Paul dă. Dacă doar desenul lui „Paul” o poartă, înainte de steaua agentului, negația e a agentului: nu Paul e cel care dă. Dacă doar desenul Annei o poartă, negația e a destinatarului: nu Anna e cea care primește, iar cine primește rămâne o întrebare deschisă (nu „nimeni”: „nimeni” e un cuvânt al lui, cu desenul lui, pe care legenda îl tratează separat).

Scopul unei negații nu e deci o etichetă; e *ce cuvinte au fost desenate de la poziții care conțin steaua negației*. Citești harta înapoi și vezi: muți toată harta cu negația și citești apoi domeniile; dacă toate stelele reapar la locul lor, negația era a propoziției întregi; dacă reapare doar Paul, era a lui; dacă nu reapare nimic, negația nu era acolo. Pilotul citește care dintre cazuri e, pentru opt feluri de a spune negația, fără o regulă pentru niciunul.

Și iată de ce un sac de cuvinte nu poate face asta. Pentru sac, „nu” e un cuvânt ca oricare altul: propoziția negată și cea afirmată au în comun toate cuvintele în afară de unul și sunt aproape identice. În povestea noastră, propoziția negată are stelele mutate cu o cheie în plus, și cele două constelații sunt departe, aproape ca două puncte aleatoare. Diferența dintre „aceleași cuvinte” și „același sens” e diferența dintre a număra stelele și a întreba unde au fost desenate.

Ambiguitatea ținută deschisă

„Fiecare copil citește câteva cărți.” Aceleași câteva cărți pentru toți, sau câte ale lui fiecare? Propoziția nu spune. Un cititor obligat să aleagă va alege și va greși jumătate din timp fără să știe.

Cuvintele de felul acesta, „fiecare”, „câteva”, „un anumit”, „niciun”, cele care spun *la câte* lucruri se referă un substantiv, se numesc *cuantificatori*, iar cele două înțelesuri sunt două *citiri*: care cuantificator e „deasupra” celuilalt. În povestea noastră pilotul nu alege și nici nu ezită: desenează fiecare cuantificator ca pe orice alt cuvânt și, în plus, desenează substantivul lui a doua oară, mutat cu o cheie relativă învățată a cuvântului cuantificator (pilotul nu vede legenda; cheia e în carnet). Legenda, de partea ei, ține două stele cu nume la care acest al doilea desen trebuie să ajungă prin învățare: una pentru cuvintele care *impun* o citire („fiecare în parte”, „un anumit”) și una, neutră, pentru cele care o lasă deschisă („fiecare”, „câteva”); articolele („o”, „un”) nu desenează nimic în plus. Lumea citește apoi harta cu o regulă simplă: dacă vede steaua „impune” pe una din fraze, acea citire e cea bună; dacă vede o singură stea neutră, e singurul cuantificator; dacă vede două stele neutre și niciuna care impune, raportează *ambele citiri*. Pilotul desenează; legenda raționează.

Am măsurat trei lucruri, pe propoziții cu una și cu două fraze cuantificate, inclusiv la o adâncime de relatare pe care pilotul n-o văzuse: că citirea impusă e găsită; că pe propozițiile neambigue „ambele” nu apare niciodată din greșeală; și că propozițiile ambigue, pe care pilotul nu le văzuse deloc la antrenare, sunt raportate „ambele”. Un cuvânt ulterior care se potrivește doar cu una din citiri („aceleași cărți”) ar putea-o închide pe cealaltă; asta nu e construit. O precizare: „ambele” nu ține minte și cele două înțelesuri întregi, cu legătura dintre alegeri; e o reprezentare a îndoielii, nu două hărți.

Nivelul nemaivăzut, și cheia care nu există

De ce poate pilotul citi o propoziție cu trei niveluri când a fost antrenat pe două? Pentru că „mută originea” e o cheie: de două ori e o cheie, de trei ori e o cheie, iar desfacerea e același număr de pași înapoi. Al treilea nivel n-are nimic nou; e același salt, încă o dată.

Și de ce n-a reușit, prima dată, să citească o negație *în interiorul* unei propoziții relatate? Pentru că acea combinație cerea o înlănțuire de chei pe care pilotul n-o făcuse niciodată: negație-la-nivel. Nu o cheie nouă, ci o înlănțuire nouă, pentru care n-avea nicio cheie relativă învățată. Odată ce acea înlănțuire a apărut în antrenare la un nivel, a citit-o la nivelul următor fără antrenare suplimentară. Tiparul merită spus ca ceea ce e: pilotul compune ce are chei pentru. O înlănțuire pe care n-a făcut-o niciodată nu e începătoșată, e lipsă. Asta înseamnă „generalizează cu un nivel mai departe decât a fost antrenat, odată ce

înlănțuirea a fost văzută la un nivel mai jos”, și e și exact locul unde metoda eșuează: pe o combinație de operatori cunoscuți pe care nu i-a văzut niciodată împreună.

O construcție nouă: o baliză lipsă

„Cartea, Paul i-o dă Annei.” Cuvinte cunoscute, ordine neparcursă. Nicio cheie nu e, în sine, greșită: fiecare cuvânt are felul lui și cheia felului. Ce se strică e altceva, și merită spus exact ce, pentru fiecare dintre cele trei desene. *Baliza*: construcția nouă deschide așteptările în altă ordine; „Cartea” sosește înainte de orice baliză și poate fi luată de una nepotrivită sau de niciuna; desenul prin balize e cel mai fragil aici. *Poziția sondei*: suma cheilor e o combinație nouă de chei cunoscute, deci un punct pe care sonda nu l-a mai atins; nu e aiurea, dar nici exact la steaua cu nume; desenul e deplasat. *Felul cuvântului anterior*: „după un articol, substantiv” e același ca în orice propoziție; desenul e grosolan, dar neatins.

Cu un singur desen, cel prin balize, cuvântul ar fi pierdut. Cu trei, celelalte două îl aduc destul de aproape de domeniul lui ca scanerul să-l recunoască, cu marjă mai mică. Citirea e încețoșată, nu distrusă. Ăsta e întregul conținut al rezultatului despre redundanță: o construcție nouă costă o parte din marjă în loc de toată.

Structura anunțată după

„Paul îi dă Annei o carte, spune Maria.” Până la „spune Maria”, prima propoziție a fost deja desenată pe hartă, față de originea mare, ca afirmație simplă. Saltul relatat vine prea târziu: stelele sunt deja desenate, și un pilot care merge doar înainte n-are cum să le ridice și să le mute la altă origine după fapt. La fel pentru o virgulă care reordonează o propoziție, și pentru orice construcție a cărei formă se dezvăluie doar la sfârșit.

E o limită reală a citirii înainte, și o declarăm ca atare. E și, precis, ce face „ținutul deoparte” pentru cuvinte singure, extins la propoziții întregi; singurul loc unde ideea că un cuvânt poate trebui reașezat de ce urmează supraviețuiește, în forma ei forțată de date.

Ordinea, și ce nu pot deosebi cheile care comută

Cheile comută. Relatat apoi negație duce sonda exact unde o duce negație apoi relatat. Deci propoziția interioară din „Maria spune că nu e cazul că P” și cea din „nu e cazul că Maria spune că P” au aceeași origine. Cele două propoziții se deosebesc totuși, dar numai pentru că *altceva* diferă: în a doua, și desenul Mariei poartă steaua negației. Acolo unde nimic altceva nu diferă, ordinea a doi operatori e pierdută.

Asta e cea mai adâncă limită structurală a poveștii așa cum o folosim, și trebuie declarată ca atare: o poziție ține minte ce chei s-au folosit, ca multimulțime (repetarea aceleiași chei contează, ordinea nu), nu ca șir; drumurile prin cer formează o grilă, nu un arbore. E același fapt, văzut din altă parte, care dă pilotului libertatea de a compune („încă o dată același salt” nu cere nicio învățare). În variantele pe care le-am cântărit, chei care nu comută, prin permutări, sau câte o stea relatat pentru fiecare nivel, ordinea se cumpără cu o parte din acea libertate. Nu e o imposibilitate matematică (există operații care păstrează ordinea și compun), ci un compromis al variantelor concrete. Am măsurat ce poate și ce nu poate versiunea care comută: acolo unde un alt desen diferă (vorbitorul, o propoziție acoperită doar de unul din operatori), cele două ordini sunt deosebite și citite corect; acolo unde nu

diferă nimic, nu. Și am găsit o rudă a aceleiași limite, mai importantă în practică: un cuvânt care vine *înaintea* a doi operatori care îl acoperă — „operatorul trebuie să nu deschidă valva” — e așezat de primul dintre ei, și cu porți tari adresa lui n-ar mai afla de al doilea. Am măsurat-o pe propoziții redate: cadrul iese corect, inclusiv la o adâncime nevăzută, pentru că porțile operatorilor se deschid parțial, adresa subiectului devine un amestec, iar ce operatori sunt și ce acoperă poartă cuvintele de după ei; costul rămâne pe adresa subiectului (luată singură, ea nu deosebește cele două ordini) și pe o singură formă, „poate să nu”, sub criteriu la adâncimea nevăzută. Remediul, așezarea prin toți operatorii care urmează, nu doar prin primul, e o schimbare a cititorului pe care n-o facem până nu e nevoie de ea. Aceeași limită e întrebarea pe care o pun limbile cu operatorul la sfârșit, negația finală din germană („… nicht”), verbul la sfârșit în japoneză, unde aproape fiecare propoziție are un cuvânt dinaintea operatorului care îl acoperă; o lume-jucărie cu verbul la sfârșit e testul natural al acestei limite, și nu e construită.

Care verb? Predicatul e și el un domeniu

„Paul îi dă Annei o carte”, „Paul îi împrumută Annei o carte”, „Paul îi vinde Annei o carte”: aceiași participanți, aceleași domenii, trei evenimente diferite. Dacă harta ține doar cine a fost găsit în domeniile agentului, obiectului și destinatarului, nu le poate deosebi. Verbul însuși trebuie desenat într-un domeniu al lui, domeniul predicatului, și legenda trebuie să aibă steaua lui. În lumile noastre timpurii n-o avea: a da și a vinde primiseră aceeași constelație a lumii și erau deci același eveniment pentru pilot. A fost o lacună în felul în care am construit acele lumi, nu o proprietate a poveștii; lumea de referință are acum steaua predicatului, și verbul se citește din ea ca orice alt răspuns. Răspunde și la o întrebare firească: „dă” și „primește” ajung la aceeași constelație pentru că lumea spune că descriu același eveniment; „împrumută” ajunge la alta pentru că lumea spune așa. Lumea decide ce e același eveniment; pilotul trebuie doar să fie consecvent cu ea.

Mai multe evenimente, și același nume de două ori

O hartă, un eveniment. Dacă două evenimente sunt topite într-o singură hartă fără nimic care să le țină separate, participanții se amestecă: agentul primului și agentul celui de-al doilea sunt amândoi în domeniul agentului, și nimeni nu mai poate spune cine ce a făcut cui. Povestea are remediul evident, fiecare eveniment mutat la propria origine înainte de topire, exact cum o propoziție relatată e ținută separat de cea care o relatează; dar l-am folosit doar pentru relatare, nu pentru „Paul îi dă Annei o carte și Peter îi împrumută un pix”. Până devine general, aria cinstită a unei hărți e un singur eveniment. La fel pentru un nume care apare de două ori în roluri diferite: sunt două desene în două domenii, se deosebesc, dar marja scade puțin, cum scade ori de câte ori aceeași stea e desenată în două locuri.

Trei feluri de „aproape”

Pentru că harta păstrează și stelele cuvintelor, și domeniile, două propoziții pot fi aproape una de alta în trei feluri, și harta le deosebește.

- Aceleași stele în aceleași domenii: propozițiile înseamnă același lucru. „Paul îi dă Annei o carte” și „Anna primește o carte de la Paul”.

- Aceleași stele în domenii diferite: propozițiile sunt despre aceleași lucruri și înseamnă altceva. „Paul îi dă Annei o carte” și „Anna îi dă lui Paul o carte”. Un sac de cuvinte nu vede deloc diferența; aici cele două constelații sunt departe.
- Stele diferite în aceleași domenii: aceeași formă, alt conținut. „Paul îi dă Annei o carte” și „Peter îi împrumută Mariei o mașină”. Același cadru, alte umpluturi; echivalente structural.

Un scor de similaritate le strivește pe cele trei într-un singur număr. Harta noastră le ține ca trei răspunsuri, iar o comparație a două texte poate spune ce fel de apropiere a găsit și în ce domenii.

Ce nu ține minte pilotul

Între propoziții, nimic. Cât timp citește o propoziție, pilotul are un mic set de lucru, balizele de pe hartă și cuvintele ținute deoparte, dar îl aruncă la sfârșit; ce supraviețuiește e doar harta. Propoziția următoare începe una nouă, iar sonda pleacă iar de la origine. E o goliciune deliberată: pilotul n-are memorie în care să acumuleze convingeri, și deci nu poate să derive sau să confabuleze de-a lungul unei conversații. Memoria e externă: cadrele citite sunt stocate într-o bază de date sau un motor de raționament, iar o propoziție ulterioară care se referă înapoi („ea o citește”) e rezolvată legându-i harta de cadrele stocate, printr-o căutare, nu printr-o memorie din interiorul pilotului. Legarea asta e condiția pentru orice la scara unui document. Secțiunea următoare spune cum se face.

Pronumele: o stea de împrumut

„Paul îi dă Annei o carte. Ea o citește.” Pentru pilot, „ea” e un cuvânt ca oricare altul: are steaua lui în catalog, e de felul „pronume feminin”, și ajunge pe harta propoziției a doua în domeniul agentului. Harta e corectă și, în același timp, nu spune cine e agentul: spune *ea*. Nimic din cer nu poate spune mai mult, pentru că steaua lui „ea” n-are nicio legătură cu steaua Annei; legătura dintre ele nu e în limbă, e în ce s-a spus înainte.

De aceea referința nu se rezolvă în cer, ci la bază, prin căutare, la fel ca greșelile de scriere. Baza păstrează constelațiile propozițiilor recente, un *atlas* al conversației. Când pilotul întâlnește un pronume, caută în atlas o stea de cuvânt compatibilă cu felul pronumelui: „ea” cere un nume feminin la singular, „acela” un lucru deja menționat, „celălalt” al doilea membru al unei perechi de același fel. Caută de la propoziția cea mai recentă înapoi, și se uită mai întâi în domeniile pe care verbul propoziției de față le așteaptă (după „citește”, un agent de felul persoană). Steaua găsită, a Annei, e *împrumutată*: e desenată pe harta propoziției a doua în locul stelei lui „ea”, iar harta spune de acum „agent: Anna”. Dacă două stele se potrivesc la fel de bine (doi candidați feminini la fel de recenți), pilotul nu alege: le topește pe amândouă în domeniul agentului, cu marcajul „una din două”, ca harta să nu le citească drept „Paul și Peter”. O propoziție următoare, sau lumea, o poate închide: lumea știe că o carte nu citește, pilotul nu, și nici nu trebuie.

Împărțirea e aceeași ca peste tot în poveste: pilotul propune după formă (felul cuvântului, recența, domeniile așteptate), lumea decide după fapte. Nimic din asta nu cere pilotului să înțeleagă cine e Anna.

Mecanismul e precis și nu cere pilotului nimic nou de învățat: piesele există deja (catalogul, domeniile, topirea candidaților, căutarea). Ce se măsoară e cât de des alege bine pe text real, și cât de des spune că nu poate alege când chiar nu poate.

De ce stelele cuvintelor sunt aleatoare, și trebuie să rămână așa

Ai putea crede că stele învățate ar fi mai bune: „pisică” lângă „câine”, „carte” lângă „roman”. Dar în povestea asta, apropierea între stelele cuvintelor e interferență. Dacă „Paul” și „Peter” ar fi aproape, domeniul agentului nu i-ar putea deosebi; marja pe care se sprijină fiecare diagnostic de mai sus s-ar micșora. Stelele aleatoare sunt fără legătură una cu alta, și asta e ce face fiecare domeniu lizibil. Apropierea semantică între cuvinte aparține altundeva: felului în care cuvintele sunt grupate în feluri (care decide cheile lor) și cunoașterii pe care lumea o are despre ce e o pisică. Treaba pilotului e să deseneze steaua potrivită în domeniul potrivit; are nevoie ca stelele să fie deosebite, nu asemănătoare. Asta rezolvă și greșelile de scriere: un cuvânt scris greșit e potrivit cu forma lui corectă *înainte* de citire, printr-o căutare care poate folosi tipare de litere, și steaua formei corecte, aleatoare ca toate, e cea desenată. Toleranța trăiește în căutare, niciodată în stele.

9. Unde duce

9.1 Asistenți care au voie să spună „nu știu”

Ia o singură lume: un orar, un inventar, un catalog de produse, fizica de liceu odată ce numerele și unitățile au stele cu nume proprii. Scrie-i legenda: rolurile pe care le pot avea întrebările lui, fiecare cu steaua ei. Antrenează carnetul de rute pe întrebări din acel domeniu, împerecheate cu constelațiile lor corecte (o interogare, o căutare, un răspuns calculat; de unde vin, în 6.4). Apoi rulează.

Un utilizator tastează. Cuvânt cu cuvânt, sonda sare și pilotul desenează; poți literalmente arăta cadrul cum se umple pe măsură ce tastează, pentru că fiecare pas e câteva mii de răsuciri și toată propoziția costă microsecunde pe un procesor obișnuit. Când propoziția se termină, cadrul e citit de pe hartă și dat bazei de date sau motorului de raționament. Răspunsul vine derivat, iar derivarea poate fi arătată lângă el: iată ce am înțeles, iată ce am întrebat, iată ce a venit înapoi.

Când harta nu se potrivește cu legenda, asistentul nu ghicește. Un domeniu rămas nerecunoscut: „n-ai spus care produs”. Un domeniu aglomerat: „factura din martie sau cea din aprilie?”. Stele rămase neexplicate: „nu am informații despre asta”. Un cuvânt fără fel: „nu cunosc termenul X”. Fiecare refuz e o distanță citită de pe hartă, nu o politică scrisă de mână; și fiecare e o afirmație despre propria legendă a pilotului, nu un verdict asupra utilizatorului.

Un astfel de asistent e mai greu de cap decât un chatbot. Nu face conversație, n-are opinii, nu știe nimic în afara legendei. În schimb, răspunsurile lui sunt restrânse la ce conțin legenda și baza de date, fiecare vine cu o marjă, și fiecare poate fi reprodus și verificat. Poate totuși greși cu încredere, desenând un cuvânt în domeniul greșit cu marjă clară; cât de des se întâmplă asta e un număr de măsurat pe fiecare lume, și asistentul trebuie judecat după el. Nu e niciun model de limbaj în bucla de citire (unul mic poate fi chemat pe

propozițiile incoerente, cum spune 9.1, și ce scrie e recitit), niciun GPU, niciun apel în cloud. Poate rula pe un telefon sau într-un browser, și își poate permite să verifice fiecare mesaj.

Intrarea dezordonată e tratată în straturi, și cinstit. Greșelile de scriere sunt absorbite în mare parte de corecția clasică de dinainte de citire, o căutare care duce forma greșită la cea corectă; steaua desenată e întotdeauna a formei corecte. Tot înainte de citire se aduc la o formă unică numerele, datele și identificatorii („patruzeci” și „40”, „marți” și „14.10”, „factura 4471”): pentru pilot ele sunt cuvinte ca oricare altele, fiecare cu steaua lui, trase la sorți pe loc dacă nu le-a mai văzut; el știe *în ce domeniu* a ajuns „40”, nu că 40 e aproape de 41. Compararea și socotelile cu valori sunt treaba lumii, după citire. Când o propoziție e cu adevărat incoerentă, un model de limbaj mic poate fi pus să o rescrie, dar nu are ultimul cuvânt. Rescrierea e citită și ea; fiecare stea recunoscută în original trebuie să fie încă prezentă; tot ce a adăugat rescrierea e marcat ca presupunere și pus utilizatorului ca întrebare. Asistentul nu pretinde că a verificat o rescriere a ceva ce n-a înțeles. Limitează ce poate strica rescrierea, și întreabă.

9.2 Un verificator lângă modelele mari

Direcția cealaltă. Un model de limbaj mare citește un document complex și propune o citire formală: un rezumat, o extracție, un set de fapte, o formă logică. Știe mult mai multă limbă decât noi. Și, uneori, scapă o propoziție, inversează două roluri sau adaugă în tăcere un detaliu plauzibil.

Noi citim același document cu pilotul nostru, separat, și comparăm constelațiile domeniu cu domeniu. Patru verdicte, nu trei. *Acord*: aceeași stea în același domeniu; două citiri separate sunt de acord, ceea ce nu înseamnă că citirea e adevărată, și nici că erorile lor sunt independente, mai ales când amândouă au învățat din texte produse de aceleași modele. *Compatibil, incomplet*: una spune „marți”, cealaltă „marți dimineață”; nu e contradicție, e detaliu în plus. *Contradicție*: stele diferite în același domeniu, fără ca una să o rafineze pe cealaltă; spunem în ce domeniu, nu cine are dreptate. *Neverificabil*: harta noastră e nesigură, un domeniu nerecunoscut sau aglomerat, sau propoziția e în afara legendei. Ultimul verdict e cel valoros: un model de limbaj poate fi învățat să se abțină, dar abținerea lui nu e legată de o structură pe care un om s-o verifice; a noastră spune în ce domeniu și de ce.

Pentru asta trebuie să fim și noi antrenați pe lumea respectivă; nu ca să egalăm acoperirea modelului mare, ci ca să avem propria constelație a textului cu care să comparăm. Partea dintr-un document pe care o putem confirma sau contrazice crește cu antrenarea, iar partea marcată neverificabilă scade. Verificatorul e cinstit în fiecare punct despre care e care. În domenii reglementate, cinstea asta valorează mai mult decât încă un punct de acuratețe.

Merită spusă și imaginea mai largă, ca direcție, nu ca rezultat. În termenii lui Kahneman, un model mare e un *sistem 1*: rapid, fluent, cu acoperire enormă și fără garanții pe un răspuns anume. Ce descrie această carte e un fel de a construi, pentru o lume dată, un *sistem 2* ieftin: un cititor mic, determinist, cu un cadru verificabil, care rulează în microsecunde și spune unde n-a înțeles. Și se construiește *din* sistemul 1: modelul mare scrie răspunsurile pe un eșantion (6.4), pilotul le învață și apoi citește singur tot restul. Bucla se poate închide și în

sens invers. Prima jumătate e construită: acolo unde pilotul contrazice profesorul cu marjă mare, cazul e revăzut și eticheta se corectează. A doua e o direcție pe care n-am testat-o și o spunem ca atare: cadrele verificate pe un corpus întreg sunt date structurate, cu proveniență, din care un model mare s-ar putea reantrena sau pe care le-ar putea primi ca ancoră. Nu e învățare de sine; e o pereche în care partea fluentă propune, partea verificabilă validează, iar validările se întorc ca date.

9.3 Formalizarea unei lumi pentru raționament simbolic

Întâi, cine scrie legenda. Poate fi scrisă de mână, de cineva care cunoaște lumea; poate fi și propusă de un model mare din textele lumii și rafinată într-o buclă cu un criteriu măsurabil (ce parte din propozițiile lumii încap în ea, cu ce rest), așa cum face unealta descrisă în depozitul proiectului. În ambele cazuri rezultatul e același fel de obiect: un text de câteva zeci de rânduri, pe care un om îl citește și îl aprobă, și care de aici înainte e declarat, nu învățat.

Cadrul citit de pe hartă e deja o structură cheie-valoare, cu niveluri acolo unde propoziția avea propoziții în propoziții. A-l transforma în fapte pentru un motor logic sau în constrângeri pentru un solver e o rescriere mecanică *odată ce cadrul, cu operatorii lui, are o semantică fixată*: domeniul agentului devine primul argument al unui predicat, nivelurile devin imbricare de termeni, o negație desenată ca rol devine un literal negat. Alegerile grele se fac o dată, în definiția cadrului, nu în traducere: ce înseamnă un cuantificator, cum se identifică două mențiuni ale aceleiași entități, cum se tratează timpul și modalitatea, și ce fel de negație se traduce (negația clasică a unei formule nu e negația prin eșec a unui program logic). Traducerea nu mai învață nimic, deci nu greșește ca pașii învățați; dar traduce consecvent și o definiție greșită.

Drumul de la limbaj natural la raționament formal devine: text, hartă, cadru, enunț formal, motor de raționament, răspuns. Fiecare verigă e inspectabilă. Toată bucla poate fi verificată față de propriul răspuns al lumii, și în lumile noastre controlate e exactă, inclusiv pe niveluri niciodată antrenate.

Operatorii logici înșiși sunt stele în legendă. Negația e o stea: lucrul negat e desenat de la o poziție care o conține, iar scopul ei, ce anume e negat, e pur și simplu ce a fost desenat așa: întreaga propoziție, sau un singur argument. „Paul nu îi dă cartea Annei” și „Paul nu dă cartea nimănui” desenează lucruri diferite, iar pilotul citește diferența fără o regulă pentru niciuna. Cuantificatorii sunt stele atașate substantivelor lor, iar care e „deasupra” o deduce legenda printr-o regulă (8, „Ambiguitatea ținută deschisă”). Acolo unde o propoziție e cu adevărat ambiguă, „fiecare copil citește câteva cărți”, lumea raportează ambele citiri: ambiguitatea e raportată, nu rezolvată printr-o ghicire, și pilotul n-a văzut propoziții ambigue la antrenare.

Pe o lume limitată, cu o schemă cunoscută, acest formalizator e mai rapid decât orice model de limbaj cu ordine de mărime și cel puțin la fel de corect, cu diferența decisivă că raportează cât de sigur e, domeniu cu domeniu, și poate fi tras la răspundere pentru acel raport. Pe text deschis, rolul realist e cel din 9.2: nu să înlocuiască formalizarea modelului mare, ci s-o verifice și s-o completeze.

9.4 O graniță care învață

Orice sistem construit din mai multe modele are cusături, și la fiecare cusătură apare ceva simbolic: o listă de obiecte detectate cu poziții și viteze, un set de câmpuri extrase, o comandă analizată. Trebuie să apară: oamenii inspectează cusătura, părțile se antrenează separat, regulile trăiesc acolo. Slăbiciunea cunoscută e că schema cusăturii e desenată de ingineri dinainte și pierde tot ce n-au prevăzut; iar reacția cunoscută, antrenarea a tot de la un capăt la altul într-un singur spațiu latent, cumpără acoperire cu prețul unei cusături pe care nimeni n-o poate citi.

Harta noastră e o cusătură de primul fel cu o parte din virtutea celui de-al doilea. Domeniile ei, rolurile, sunt tot date de lume; dar felul în care cuvintele ajung la ele, și ce cuvinte pun balize, e învățat, nu scris de mână; stele cu nume noi pot fi adăugate în legendă când lumea le cere, cu doar carnetul de reantrenat; e lizibilă prin construcție; și poate păstra, topit lângă domenii, un rezidual pentru tot ce n-are încă nume, astfel încât ce n-a prevăzut schema e înceţoşat, nu pierdut. Aceeași hartă poate fi umplută din mai multe simțuri deodată, pentru că o stea e doar coordonate și un domeniu e doar o stea cu nume: un cuvânt, un obiect detectat într-o imagine și desenat de la o poziție care îi codifică locul, un eveniment dintr-un jurnal, o citire de la un senzor. De fapt, limbajul a fost cazul greu: în text, cheia trebuie ghicită din cuvânt. Oriunde cheia e dată direct, de o comandă motorie, de timpul scurs, de o coordonată, mașinăria se aplică fără nimic de ghicit, ceea ce e tocmai cadrul pentru care a fost gândită teoria lui Jeff Hawkins despre cortex de la început (capitolul 15).

9.5 Un model al lumii pe care îl poți citi

Acolo unde constelația lumii e consecința unui eveniment, nu descrierea lui, a învăța să o produci din propoziție e un pas spre învățarea dinamicii unei lumi: stare plus eveniment dă starea următoare. Am făcut asta doar pe lumi mici, iar a citi un eveniment într-un cadru nu e încă același lucru cu a prezice ce urmează din el; afirmația de aici e o direcție, nu un rezultat. Modelele de lume de azi fac asta într-un spațiu latent pe care nu-l citește nimeni, și își imaginează viitorul rulând o rețea înainte, pas după pas, cu erori care se adună și nimic față de care să verifice. Starea noastră e o hartă cu o legendă. Oriunde lumea are reguli, date și termene, contracte, proceduri, cod, baze de date, „imaginarea consecinței” poate fi dată unui motor de raționament simbolic care lucrează pe cadru: exact, explicabil, stabil peste oricâți pași. Acolo unde lumea n-are reguli scrise, căderea unui pahar, abordarea latentă își păstrează locul, iar un hibrid folosește cadrul ca schelet și un rezidual latent pentru rest.

10. Robotică: un dispozitiv de blocare semantică

Roboții primesc tot mai des comenzi în limbaj natural, iar drumul de la comandă la motor trece printr-un model mare: un planificator care transformă „pune cutia pe raftul de sus” într-o secvență de mișcări. Modelul e capabil și opac. Când greșește, greșește în lumea fizică, și nimeni nu vede greșeala înainte să se întâmple. Industria are, pentru restul robotului, o soluție veche și bună: *interlock*-ul, dispozitivul care taie fizic alimentarea unei acțiuni nepermise, indiferent de ce a decis restul sistemului. Ce lipsește e un interlock

pentru *sens*: ceva care să știe ce s-a cerut și ce s-a propus, și să blocheze înainte de mișcare ce nu e permis.

Cum arată

Între planificator și actuatoare stă pilotul, cu o legendă de acțiune scrisă pentru robotul acela: ce acțiune, asupra cărui obiect, unde, cu ce viteză sau forță, sub ce condiție, la cererea cui. Orice comandă venită de la om și orice acțiune propusă de planificator trec prin el și devin răspunsuri la aceste întrebări. Ce nu încape în legendă nu există pentru interlock, deci nu poate fi executat: o acțiune pe care nimeni n-a prevăzut-o nu are cum să fie exprimată. Cadrul citit se traduce mecanic în fapte, iar politicile de siguranță sunt reguli în același limbaj: nicio mișcare rapidă cât timp o persoană e în zona B; niciun obiect din clasa „fragil" deasupra vitezei V; orice acțiune asupra obiectelor din lista L cere confirmare. Un motor de reguli decide, cu o derivare care se poate afișa, și numai cadrele permise ajung la actuatoare.

Partea care face asta mai mult decât un filtru de cuvinte e că aceeași legendă citește și *lumea senzorială*. Un obiect detectat cu poziția lui, o persoană intrată într-o zonă, o ușă deschisă sunt și ele fapte cu roluri (ce, unde, când), și intră pe aceeași hartă printr-un salt dat direct de senzor, fără nimic de ghicit (9.4). Regula „nu te mișca rapid cât timp cineva e în zona B" se verifică atunci pe aceeași hartă pe care stă și comanda, în același motor. Interlock-ul vede comanda și contextul în aceeași formă.

De ce e ieftin, și de ce poate fi hardware

Tot ce face pilotul e adunare de unghiuri pe o mie și ceva de rotițe și o comparație cu un catalog. O comandă costă microsecunde pe un procesor obișnuit; carnetul de rute încape în zece–cincisprezece megaocteți; nu e nevoie de niciun accelerator. Cu codurile rare pe blocuri din capitolul 14, operațiile devin aritmetică pe indici întregi, exact genul de lucru care se pune într-un microcontroler sau într-un FPGA. Asta deschide o arhitectură pe care siguranța funcțională o cunoaște bine: interlock-ul semantic pe un circuit separat, cu propria alimentare și propriul ceas, care primește comanda și contextul de la calculatorul principal și singur emite semnalul de *enable* spre actuatoare. Calculatorul principal poate fi oricât de complex și oricât de opac; dispozitivul care dă voie e mic, determinist și citibil linie cu linie.

Determinismul contează aici mai mult decât oriunde. Aceeași comandă dă aceeași hartă, de fiecare dată; nu există eșantionare, nu există temperatură, nu există răspuns „creativ". Fiecare blocare poate fi reprodusă și explicată: iată cadrul citit, iată regula încălcată. Fiecare acceptare la fel. Pentru certificarea unui sistem de siguranță, asta e diferența dintre un component care poate fi analizat și unul care trebuie crezut pe cuvânt.

Ce face când nu e sigur

Implicitul unui robot e repausul. Când harta e nesigură, un domeniu cu marjă mică, un reziduu mare, un cuvânt fără fel, interlock-ul nu ghicește și nu blochează orbește: *oprește și întreabă*, și poate spune exact ce a înțeles: „Am înțeles: mută cutia la raionul B, fără viteză precizată. Confirmi?". Confirmarea e tot o propoziție, citită în aceeași legendă. Dialogul de

confirmare e, de fapt, interfața firească a dispozitivului: omul vede cadrul înainte de mișcare, nu după.

Ce decide cât de departe se merge

Piesele sunt cele din tot documentul: cititorul, cadrul, traducerea în reguli, diagnosticele. Legenda de acțiune pentru un robot anume și regulile lui sunt inginerie pe lumea respectivă, ca la orice sistem de siguranță. Măsura care decide cât de departe se poate merge e aceeași din capitolul 16: scopul negației pe comenzi reale („nu lăsa cutia aici, du-o acolo") și rata greșelilor făcute cu încredere, pentru că un interlock blochează prin construcție doar ce nu poate exprima: o acțiune periculoasă poate fi perfect exprimabilă (ștergerea obiectului greșit, un șir de acțiuni fiecare permisă), iar o citire greșită cu marjă mare rămâne posibilă. Interfața restrânge și verifică; nu garantează. Corectitudinea citirii și potrivirea politicilor se măsoară separat.

11. Enterprise și industrii puternic reglementate

Sisteme agentice: intenția citită fără model de limbaj

Un sistem agentic de întreprindere primește cereri în limbaj natural („mută factura 4471 pe centrul de cost al departamentului de vânzări și anunță-l pe Martin") și le transformă în apeluri către software: ERP, CRM, sistemul de tichete, calendarul. Azi, pasul de la cerere la apel îl face un model mare, la fiecare cerere, cu latența, costul și nedeterminismul lui, și cu riscul ca o cerere să fie înțeleasă altfel decât a fost spusă și executată oricum.

Pentru o lume fixată, lumea unui software de întreprindere, cu entitățile, acțiunile și parametrii lui, pilotul face acest pas singur. Legenda e schema sistemului: ce acțiuni există, asupra căror obiecte, cu ce câmpuri. Carnetul de rute se antrenează o dată pe cereri din acel domeniu, cu cadrele lor, produse de un model mare pus pe post de profesor (6.4) și auditate pe un eșantion. După aceea, o cerere clară devine un cadru complet în microsecunde, pe un procesor obișnuit, în interiorul rețelei companiei, fără niciun apel extern: *intenția* e acțiunea al cărei domeniu s-a umplut, *parametrii* sunt stelele din celelalte domenii, iar ce lipsește e la vedere. „Mută factura 4471 pe centrul de cost al departamentului de vânzări și anunță-l pe Martin": acțiune mutare, obiect factura 4471, destinație centrul de cost vânzări, notificat Martin; termen, nedat.

Modelul mare nu dispare; își schimbă locul. Arhitectura e cu două căi. Calea rapidă: cererea e citită de pilot; dacă harta e netă, toate domeniile obligatorii pline cu marjă clară și reziduu mic, cadrul merge direct la execuție, după verificarea politicilor. Calea lentă: dacă harta e nesigură, cererea merge la modelul mare, care o reformulează sau o completează, iar rezultatul lui e citit din nou de pilot, pentru că nimic nu ajunge la execuție altfel decât ca un cadru verificat. Modelul mare lucrează pe coada grea a cererilor, nu pe toate; costul și latența lui se plătesc pe o parte din cereri, nu pe toate; cât de mare e partea se măsoară pe fiecare lume, nu se presupune.

Trei lucruri vin gratis din această așezare. *Autorizarea* devine o regulă pe cadru: cine are voie să facă ce acțiune asupra căror obiecte se scrie în același limbaj ca politicile din capitolul 16, și se verifică înainte de execuție, nu după. *Jurnalul* e o listă de cadre, nu de

texte: fiecare acțiune executată are lângă ea cererea, cadrul citit, marja și regula care a permis-o, lizibile de un om fără instrumente. Și *determinismul*: aceeași cerere dă același cadru, azi și peste un an, ceea ce face posibile testele de regresie pe înțelegere, nu doar pe cod.

Industrii reglementate: documente care trebuie citite exact

În farmaceutică, bănci, aviație, energie, documentele care guvernează activitatea (proceduri standard de operare, legi, regulamente, ghiduri, rapoarte de audit) sunt, în esență, liste de fapte cu roluri: cine trebuie să facă ce, când, în ce condiție, cu ce dovadă, cu ce excepție, în baza cărui articol. Sunt lungi, se schimbă des și trebuie citite exact, pentru că o obligație scăpată sau un termen înțeles greșit costă.

O legendă pentru acest fel de text are câteva zeci de întrebări: actorul obligat, acțiunea, obiectul, termenul, condiția, excepția, dovada cerută, referința normativă. Cu ea, pilotul transformă un document într-o listă de cadre, câte unul pe propoziție care spune ceva, fiecare legat de propoziția lui, cu domeniile goale la vedere și cu reziduul care arată ce n-a încăput în legendă. De aici, câteva operații care astăzi se fac manual sau cu modele mari fără verificare devin comparații între hărți:

- *Diferența între două versiuni ale unui regulament*: nu ce cuvinte s-au schimbat, ci ce obligații au apărut, au dispărut sau și-au schimbat termenul ori actorul, domeniu cu domeniu.
- *Conformitatea unei proceduri cu norma*: procedura și regulamentul citite în aceeași legendă, apoi comparate; o obligație din regulament fără cadru corespunzător în procedură e un gol vizibil, nu o părere.
- *Extracția fiabilă de date* din rapoarte, formulare, notificări: fiecare valoare extrasă vine cu domeniul ei, cu marja și cu propoziția sursă, iar „nu e precizat” e un răspuns, nu o celulă goală fără explicație.
- *Verificarea rezumatelor și extracțiilor făcute de modele mari* (9.2): confirmat, contrazis, neverificabil, cadru cu cadru, cu indicarea exactă a locului de verificat.

Ce caută un auditor sau un regulator la un asemenea sistem e exact ce are: aceeași intrare dă aceeași ieșire; fiecare răspuns e trasabil la o propoziție și la o linie din legendă; modelul e mic și poate fi inspectat în întregime; datele nu părăsesc instituția, pentru că nu e nevoie de niciun serviciu extern ca să citești. Logica de validare a sistemelor computerizate din industriile reglementate cere componente al căror comportament poate fi specificat, testat și reprodus; un cititor determinist cu o legendă declarată, pe care oamenii o pot citi și aproba, se potrivește acelei logici într-un fel în care un model mare, prin natura lui, nu se poate potrivi.

Unde intră modelele mari, și unde nu

Nu ascundem că modelele mari sunt necesare. Ele scriu primele versiuni ale cadrelor de antrenare, parafrazează, rezumă text deschis, tratează coada grea a cererilor și propun formalizări ale documentelor complexe. Rolul nostru e cel definit în tot acest document: stratul verificabil dintre limbaj și acțiune, cel care spune ce s-a înțeles, ce nu, și unde, și prin care trece tot ce ajunge la execuție sau în baza de date. Împărțirea e curată: modelul mare

pentru acoperire, pilotul pentru verificare. Într-o întreprindere, asta înseamnă costuri de inferență mai mici, latență sub milisecundă pe calea rapidă, și un jurnal pe care îl poate citi un auditor. Într-o industrie reglementată înseamnă posibilitatea de a folosi limbaj natural acolo unde până acum nu era permis, pentru că nu putea fi verificat.

Ce decide ritmul

Pentru fiecare lume, legenda și carnetul: inginerie pe lume, cu conducta de adnotare deja desenată. Măsurile care decid ritmul sunt verificatorul cu erori injectate pe text real, calibrarea raportată pe fiecare tabel și proporția de cereri care rămân pe calea rapidă într-o lume reală. Direcția e clară și pașii sunt incrementali; ce se câștigă la fiecare pas se măsoară în cereri servite fără model mare și în erori prinse înainte de execuție.

12. Mai departe: documente, noutate, părtinire, netezire

O propoziție e un traseu scurt. Un document e unul lung: multe hărți, legate prin chei („contractul menționat mai sus” e o cheie înapoi spre o hartă anterioară). Odată ce documentele sunt hărți cu legende, trei lucruri care îi interesează pe oameni se dovedesc geometrice.

Noutatea. Citește un articol științific față de legenda lumii lui, unde stelele cu nume sunt rolurile proprii acelei lumi: problema abordată, metoda folosită, mărimea măsurată, rezultatul obținut. Cea mai mare parte a constelației articolului cade în domenii deja ocupate de stele cunoscute: o metodă cunoscută, o problemă cunoscută. Noutatea e regiunea în care nu cade așa: o stea într-un domeniu în care câmpul avea alt ocupant, sau stele în puncte pe care legenda nu le numește deloc. Nu „sună nou”, ci „diferă de legendă aici, în domeniul ăsta”. Locul acelei regiuni și direcția în care cad stelele neexplicate sunt măsura; mărimea restului singură nu e (capitolul 8, a treia precizare).

Părtinirea. Două relatări ale aceluiași eveniment ar trebui să deseneze aceeași constelație. Când o sursă desenează constant aceleași stele în domenii deplasate, cel care a acționat devine cel afectat, cauza devine o împrejurare, acea deplasare e o cheie în plus, aceeași peste multe perechi de relatări. Părtinirea unei surse e direcția medie în care constelațiile ei deviază de la ale celorlalte, lizibilă domeniu cu domeniu. Nu cere să știi ce s-a întâmplat de fapt; cere doar mai multe relatări de comparat, și legenda.

Netezirea (smoothing). E un concept din propria noastră cercetare: o părtinire care nu deformează, ci omite. Consecința care nu convine pur și simplu nu e menționată. Răspunsul e lustruit, diplomatic, și nimic din el nu e fals. Metodele care examinează ce se spune nu o pot vedea, pentru că nimic din ce se spune nu e greșit. În povestea noastră e o părtinire a absenței. Legenda unui eveniment are un domeniu pentru consecința lui și unul pentru cine o suportă; în relatările neutre ale acelui fel de eveniment, domeniile acelea sunt pline de cele mai multe ori. La o sursă care netezește, ele sunt sistematic goale. Traseul ocolește o regiune a cerului. Propunem să-i spunem *evitare de regiune a cadrului*, și poate fi măsurată fără să înțelegi conținutul consecinței: e de ajuns să știi că acel domeniu există în legendă, că de obicei e plin, și că aici nu e.

Toate trei sunt *indicatori candidați*, nu măsuri validate: un rest mare poate fi o citire greșită, nu noutate; o abatere de la celelalte surse poate fi o minoritate care are dreptate, nu părtinire; un domeniu gol poate fi concizie sau lipsă de informație, nu netezire. Fiecare cere controale (întrebarea pusă, informația disponibilă, lungimea, relevanța consecinței omise) înainte să primească numele fenomenului. Și toate sunt direcții. Au nevoie de legende la nivelul evenimentelor, nu al propozițiilor, de rate de bază pe lume pentru cât de des se umple fiecare domeniu, și de legarea referințelor între propoziții. Nu are nevoie ca pilotul să cunoască lumea. Are nevoie doar de forma evenimentelor, iar forma e ce învață.

13. Pentru ingineri: algoritmul, în zece pași

Tot ce e mai sus a fost spus în imagini. Capitolul acesta spune aceleași lucruri fără imagini, în zece pași, cu o singură notație și fără cuvinte neexplicate. Nu presupune decât matematică de liceu: vectori, unghiuri, cosinus. Fiecare pas se sprijină doar pe cei dinainte.

Notația, o dată pentru tot capitolul. Un vector e o listă de D numere; D e dimensiunea (noi folosim 1024). Scriem:

- f[cuvânt] — vectorul de identitate al unui cuvânt: f[Paul], f[carte];
- K[CATEGORIE] — vectorul-cheie al unei categorii de cuvinte: K[NUME]; un cuvânt gramatical sau un verb e singur în categoria lui (pasul 3), și atunci categoria poartă numele cuvântului, cu majuscule: K[ÎI], K[DĂ], K[CĂ]; K[ÎI→NUME] — cheia unei *perechi* de categorii (pașii 8–9); adresa(cuvânt) — adresa la care a fost pus un cuvânt (pasul 3);
- R[întrebare] — vectorul unei întrebări a lumii: R[cine dă], R[cui];
- S — starea construită din propoziție; S* — starea pe care lumea o declară corectă pentru aceeași propoziție; S*(P) — ținta propoziției P, când P e parte dintr-o propoziție mai mare (pasul 7); Ŝ — starea reconstruită din răspunsurile recunoscute (pasul 6); m — marcajul de ocupare, un vector fix (pasul 4);
- ⊗ — înmulțirea de la pasul 2; + și − — adunarea și scăderea obișnuite de vectori; conj(x) — inversul lui x; x^k — x ⊗ x ⊗ … ⊗ x, de k ori; cos(x, y) — cosinusul între doi vectori; ≈ — aproape egal (cosinus aproape 1); ‖x‖ — lungimea vectorului x (rădăcina sumei pătratelor celor 2·D numere reale). Parantezele pătrate indexează mereu: f[…] cu un cuvânt, K[…] cu o categorie sau o pereche de categorii, R[…] cu o întrebare. Nicio altă notație nu apare.

Pasul 1. Trei feluri de vectori, și cine îi folosește

Un vector are D componente, iar fiecare componentă e un **unghi**, între 0° și 360°. Atât. Un vector e o listă de 1024 de unghiuri.

nume	ce este	câți sunt	cine îl alege	cine îl folosește
identitate f[w]	vectorul unui cuvânt	unul pe cuvânt	aleator, o dată, apoi fix	codorul (îl pune în stare) și decodorul (îl recunoaște)

nume	ce este	câți sunt	cine îl alege	cine îl folosește
întrebare R[r]	vectorul unei întrebări a lumii: „cine dă?”, „ce?”, „cui?”, „care e acțiunea?”	unul pe întrebare	aleator, o dată, fix — dat de cel care definește lumea	lumea (scrie răspunsul corect) și decodorul (pune întrebarea). **Codorul nu-l vede niciodată**
cheie K[c]	vectorul unei *categorii* de cuvinte: nume, verb, prepoziție…	una pe categorie și câte una pe pereche de categorii (pașii 8–9); câteva mii în total, niciuna pe un nume sau pe un obiect	**învățat**	numai codorul

Identitățile și cheile sunt ale codorului; întrebările sunt ale lumii și ale decodorului. Singurul lucru care se învață vreodată sunt cheile: câteva mii de vectori, toți pe categorii de cuvinte, niciunul pe un nume sau pe un obiect anume (ce e o categorie, și de unde vine, la pasul 3). Că codorul nu vede întrebările e esențial: el nu „caută subiectul”; el înmulțește chei și adună identități. Dacă răspunsul ajunge la locul întrebării, e pentru că produsul cheilor categoriilor dinaintea cuvântului a *învățat* să fie egal cu vectorul întrebării. Asta verificăm, nu impunem.

Pasul 2. Două operații, o comparație și patru proprietăți

Amândouă operațiile se fac componentă cu componentă (unghiul 1 cu unghiul 1, unghiul 2 cu unghiul 2 și așa mai departe).

Înmulțirea a ⊗ b: aduni unghiurile. Rezultatul are pe fiecare poziție unghi_a + unghi_b (dacă trece de 360°, o ia de la capăt). Rezultatul e tot o listă de unghiuri. Îi spunem înmulțire pentru că, dacă scrii fiecare unghi ca un număr complex de lungime 1, adunarea unghiurilor e exact înmulțirea numerelor; dar poți gândi mereu „adun unghiurile”.

Adunarea a + b: adunarea obișnuită de vectori, componentă cu componentă, ca numere complexe (fiecare unghi e o săgeată de lungime 1 pe un cerc; adunarea pune săgețile cap la cap). Rezultatul *nu* mai e o listă de unghiuri pure: fiecare componentă are acum și o lungime — mare dacă unghiurile adunate erau apropiate, aproape zero dacă erau opuse. Starea S e o asemenea sumă.

Înmulțirea se aplică și unui vector cu lungimi, cum e S: pe fiecare componentă, săgeata se rotește cu unghiul cheii și își păstrează lungimea. De aceea înmulțirea *distribuie* peste adunare, a ⊗ (x + y) = a ⊗ x + a ⊗ y: a roti o sumă de săgeți e același lucru cu a aduna săgețile rotite. Folosim asta la pașii 6 și 7.

Compararea cos(a, b) (a *compara* din 2.1): cosinusul unghiului dintre cei doi vectori, luați ca vectori obișnuiți de 2·D numere reale: 1 dacă sunt identici, 0 dacă n-au nicio legătură, −1 dacă sunt opuși.

Cele patru proprietăți de care depinde tot ce urmează:

1. **Doi vectori aleatori n-au legătură.** Trage 1024 de unghiuri la întâmplare, de două ori; cosinusul între cele două liste e aproape exact 0 (abaterea tipică e cam 0,02). De aceea identitățile și întrebările pot fi trase la sorți: sunt automat distincte, cu certitudine practică.
2. **Înmulțirea are inversă exactă.** conj(a) e vectorul cu toate unghiurile negate. a ⊗ conj(a) are toate unghiurile 0 — elementul neutru. Deci din a ⊗ x recuperezi x exact: conj(a) ⊗ (a ⊗ x) = x.
3. **Înmulțirea păstrează distanțele.** cos(a ⊗ x, a ⊗ y) = cos(x, y): două lucruri înmulțite cu același vector rămân la fel de apropiate. Consecința: a ⊗ x n-are legătură nici cu a, nici cu x; pentru oricine nu știe a, e un vector nou, aleator.
4. **Înmulțirea comută.** a ⊗ b = b ⊗ a, pentru că adunarea unghiurilor comută. Comod, și o limitare reală (pasul 10).

Și consecința lor, pe care o folosim la fiecare citire. Dacă S = a ⊗ x + (alți termeni), atunci

conj(a) ⊗ S = x + conj(a) ⊗ (alți termeni)

Primul termen e exact x. Ceilalți sunt vectori aleatori înmulțiți cu alt vector aleator — zgomot, fără legătură cu niciun cuvânt din catalog. Deci x se recunoaște clar, iar restul nu deranjează. Zgomotul crește cu numărul de termeni din S, cam cu rădăcina pătrată a lui: la D = 1024, zeci de termeni într-o stare se citesc curat, sutele încep să se amestece.

Pasul 3. Codarea unei propoziții

Înaintea codorului lucrează două unelte clasice, care nu fac parte din ce se învață. Un *segmentator* împarte textul în propoziții și cuvinte (semnele de punctuație sunt cuvinte, cu categoria lor). Un *dicționar* dă fiecărui cuvânt două lucruri. Întâi **forma de bază**: „Annei” e forma lui „Anna”, „dă” a lui „a da”. Ținta lumii folosește mereu forma de bază; codorul o folosește pentru nume și obiecte, care împart câte o categorie (f[Anna], aceeași identitate pentru codor și pentru lume), iar pentru verb poate păstra forma din propoziție și lăsa cheia verbului să absoarbă diferența, cum face sistemul măsurat (în formule: codorul are f[dă], lumea f[a da]). Apoi **categoria**. Categoriile sunt un inventar fix și mic, de ordinul zecilor: una pentru toate numele de persoane, una pentru toate obiectele, câte una pentru fiecare cuvânt gramatical („că”, prepozițiile, articolele; sunt puține, deci fiecare poate fi propria categorie) și, în lumile măsurate, câte una pe verb al lumii (tot puține; de aceea cheia lui „dă” se scrie K[DĂ], a lui „îi” K[ÎI]: categoria poartă numele cuvântului). Lumile mai bogate declară mai multe feluri deschise (instituții, locuri, cantități…) în lista lor de feluri, cum face capitolul 5 cu „bancă”. Pentru un cuvânt pe care dicționarul nu-l are, un etichetator automat obișnuit propune categoria din context. Asta e tot ce înseamnă „cuvinte noi fără reantrenare”: un nume nou primește categoria NUME și o identitate nouă pe loc,

iar cheile, care nu depind de cuvânt, îl duc unde trebuie; un cuvânt a cărui categorie nu poate fi dată rămâne fără cheie, și capitolul 8 („Cheia ruptă”) spune ce se vede atunci pe hartă.

Exemplu: „Paul îi dă Annei o carte.” Categoriile, din dicționar:

cuvânt	Paul	îi	dă	Annei	o	carte
categorie	NUME	ÎI	DĂ	NUME	O	SUBST

Codorul ține două lucruri: o **adresă curentă** a (pornește de la vectorul neutru, toate unghiurile 0) și **starea** S (pornește de la zero). Pentru fiecare cuvânt, în ordine, două linii:

```
# adresa, înmulțită cu cheia categoriei
a = a ⊗ K[categoria cuvântului]
# identitatea, înmulțită cu adresa, adunată în S
S = S + a ⊗ f[cuvânt]
```

Adresa la care e pus fiecare cuvânt:

cuvânt	adresa la care e adunat în S
Paul	K[NUME]
îi	K[NUME] ⊗ K[ÎI]
dă	K[NUME] ⊗ K[ÎI] ⊗ K[DĂ]
Annei	K[NUME] ⊗ K[ÎI] ⊗ K[DĂ] ⊗ K[NUME]
o	… ⊗ K[O]
carte	… ⊗ K[O] ⊗ K[SUBST]

Starea finală:

```
S = K[NUME] ⊗ f[Paul]
  + K[NUME] ⊗ K[ÎI] ⊗ f[îi]
  + K[NUME] ⊗ K[ÎI] ⊗ K[DĂ] ⊗ f[dă]
  + K[NUME] ⊗ K[ÎI] ⊗ K[DĂ] ⊗ K[NUME] ⊗ f[Anna]
  + … ⊗ f[o]
  + … ⊗ f[carte]
```

Trei observații:

- **Paul și Annei au aceeași categorie, dar adrese diferite**, pentru că adresa lui Annei conține în plus cheile cuvintelor dintre ei. Asta e tot ce deosebește „cine dă” de „cui”: *ce a venit înainte*.

- **Ordinea cheilor dintr-un produs nu contează** (proprietatea 4). Adresa lui Annei spune „un NUME, un ÎI, un DĂ și încă un NUME au apărut până aici”, nu „în ordinea asta”. Pentru propoziția de aici ajunge; unde nu ajunge, vedem la pașii 8 și 10.
- **Nicăieri nu apare o întrebare, un subiect, o regulă.** Codorul nu știe că Paul e cel care dă; știe doar că Paul e la adresa K[NUME]. Dacă acea adresă *ajunge să fie* R[cine dă], e meritul învățării (pasul 5).

Toate cuvintele intră în S, inclusiv „îi” și „o”. Ele nu răspund la nicio întrebare a legendei; rămân în S ca termeni pe care decodorul nu-i caută (contează la pasul 6, la „ce n-a fost explicat”), dar nu sunt inutile: pasul 4 arată ce rol ajung să joace. Și un lucru pe care codorul *nu* îl face: nu adaugă nimic în afară de identități, niciun semn de „aici am desenat ceva”. Marcajul m de la pasul următor e numai în ținta lumii; cum ajunge totuși codorul să-l reproducă, tot acolo.

Pasul 4. Ținta: ce spune lumea că ar trebui să fie S

Lumea (evidența de împrumuturi, aici) are întrebările ei fixe, fiecare cu vectorul ei: R[cine dă], R[ce], R[cui], R[acțiunea]. Pentru propoziția noastră, lumea declară starea corectă ca sumă de perechi (întrebare, răspuns), fiecare pereche legată prin înmulțire:

```
# acțiunea: verbul la forma de bază, apoi cele trei întrebări
S* = R[acțiunea] ⊗ f[a da]
   + R[cine dă] ⊗ (f[Paul]  + m)
   + R[ce]      ⊗ (f[carte] + m)
   + R[cui]     ⊗ (f[Anna]  + m)
```

m e un vector fix, același peste tot, adunat de lume la fiecare răspuns: **marcajul de ocupare** (*markerul* din capitolul 8). Rostul lui: la citire (pasul 6) să poți spune „la întrebarea asta s-a răspuns ceva, chiar dacă nu recunosc cuvântul” separat de „la întrebarea asta nu s-a răspuns”. Fără el, un cuvânt necunoscut și o întrebare goală ar arăta la fel. Codorul nu-l desenează (pasul 3); dar S trebuie să se apropie de S*, deci cheile învață să pună termenul R[cui] ⊗ m din altceva: din cuvintele gramaticale. „Îi” are mereu aceeași identitate și apare ori de câte ori există un destinatar; învățarea îi așază steaua exact acolo unde ținta are R[cui] ⊗ m, și la fel pentru celelalte întrebări. De aceea o propoziție fără destinatar („Paul dă o carte”) n-are nimic la acea adresă și se citește „nedat”, iar una cu un destinatar necunoscut are marcajul fără un cuvânt recunoscut. R[acțiunea] n-are + m pentru că acțiunea e mereu dată (orice propoziție are un verb), deci n-are ce deosebi. Cele două piese, R[acțiunea] și m, sunt în algoritm și sunt măsurate (predicatul în lumea de referință, marcajul în verificatorul de cadre din 9.2); starea lor la zi, ca a oricărei măsurători, e în STATUS.md.

Ce *nu* conține S*: „îi”, „o”, ordinea cuvintelor, forma verbului. Două formulări ale aceluiași eveniment („Paul îi dă Annei o carte”, „Anna primește o carte de la Paul”) au aceeași S*, pentru că răspund la fel la aceleași întrebări — cu o excepție, verbul: „dă” și „primește” sunt cuvinte diferite, deci R[acțiunea] primește identități diferite, iar un tabel al lumii spune la

citire că amândouă numesc același fel de eveniment. Aceasta e definiția sensului din 1.1, scrisă ca vector.

Pasul 5. Învățarea: un singur obiectiv

Pentru fiecare propoziție de antrenare avem S (ce a construit codorul, pasul 3) și S* (ce declară lumea, pasul 4). Obiectivul e unul singur: cheile K[·] se ajustează ca cos(S, S*) să fie cât mai mare, în medie, pe zeci de mii de propoziții generate de lume. Nimic altceva nu se învață: identitățile, întrebările și marcajul rămân fixe.

Ce înseamnă, concret, că învățarea a reușit: produsul cheilor dinaintea unui cuvânt a devenit egal cu vectorul întrebării la care răspunde cuvântul. Pe exemplu:

K[NUME] ≈ R[cine dă]
pentru verb, cheia absoarbe și trecerea la forma de bază
K[NUME] ⊗ K[ÎI] ⊗ K[DĂ] ⊗ f[dă] ≈ R[acțiunea] ⊗ f[a da]
K[NUME] ⊗ K[ÎI] ⊗ K[DĂ] ⊗ K[NUME] ≈ R[cui]

De aceea parafrazele converg fără să le fi spus cineva că sunt parafraze: în „Anna primește o carte de la Paul”, produsul cheilor dinaintea lui Paul e altul (… ⊗ K[DE] ⊗ K[LA] ⊗ K[NUME]), dar trebuie să ajungă tot la R[cine dă], pentru că acolo îl pune S*. Aceleași câteva mii de chei trebuie să împace toate formulările deodată; dacă reușesc, reușesc pentru orice nume și orice obiect, pentru că numele și obiectele nu intră în chei. Antrenarea e descreștere de gradient obișnuită (Adam) pe unghiurile cheilor, câteva minute pe un accelerator mic; o repetăm cu cinci alegeri aleatoare diferite ale identităților și întrebărilor, ca să vedem că rezultatul nu depinde de sorți.

Un avertisment, înainte de a merge mai departe. Cu convenția de la pasul 3 singură, exemplul nu se poate rezolva până la capăt: „Anna” din a doua propoziție e primul cuvânt, categoria NUME, deci ar primi tot adresa K[NUME] și ar ajunge la R[cine dă], unde lumea o pune pe Paul. Pașii 3–7 descriu schema de bază; cuvântul care vine înaintea verbului e rezolvat la pasul 9, iar adresa „de la început” rămâne una din cele trei adrese adunate acolo.

Pasul 6. Decodarea: de la stare la răspunsuri

Pentru fiecare întrebare r a lumii:

scot adresa întrebării (proprietatea 2), apoi caut în catalog
x = conj(R[r]) ⊗ S
răspuns = cuvântul w cu cel mai mare cos(x, f[w])
marjă = cel mai mare cos − al doilea cel mai mare

Pe exemplu, conj(R[cui]) ⊗ S e aproape f[Anna] + m + zgomot; cos cu f[Anna] e cam 0,3–0,4, cu orice alt cuvânt cam 0,02 (nu 1, pentru că S conține și ceilalți termeni, șase cuvinte la câte trei adrese, pasul 9, care împart lungimea; contează distanța față de 0,02, nu față de 1). Marja e mare: răspuns sigur. Când două cuvinte ies cam la fel, marja e mică și decodorul **se abține** în loc să ghicească — pragul e fixat înainte, iar rata de „sigur și greșit” e raportată la fiecare măsurătoare.

Trei întrebări în plus, din aceeași stare:

- **E ocupat?** cos(x, m) peste un prag: s-a răspuns ceva la întrebarea asta, chiar dacă nu recunosc cuvântul. Sub prag: nu s-a răspuns.
- **Ce n-a fost explicat?** Reconstruiesc Ŝ din perechile recunoscute (R[r] ⊗ f[răspuns]), o aduc la scara lui S (o înmulțesc cu ‖S‖ / ‖Ŝ‖) și calculez ‖S − Ŝ‖ / ‖S‖. Cuvintele gramaticale („îi", „o") rămân mereu în această diferență, deci o comparăm cu valoarea ei obișnuită pe propoziții corecte ale aceleiași lumi; o diferență mult peste acea valoare înseamnă că propoziția conține ceva ce lumea nu explică deloc (cuvinte necunoscute, o construcție neparcursă). O întrebare care *lipsește* din legendă nu se vede în mărimea diferenței, ci în direcția ei: resturile propozițiilor cu același răspuns neîntrebat se grupează (măsurat; capitolul 8, a treia precizare).
- **Care eveniment?** Răspunsul la R[acțiunea] e un verb; tabelul lumii îl traduce în felul de eveniment (a da, a primi → transfer).

Rezultatul tuturor acestor citiri — răspunsurile, marjele, ocuparea, restul — e *cadrul* propoziției, un obiect pe care îl poți scrie ca JSON, Prolog sau formulă logică: e exact ce primește restul sistemului.

Pasul 7. Propoziții în propoziții

„Maria spune că Paul îi dă Annei o carte." Lumea definește ținta prin aceeași regulă, aplicată o dată în plus: propoziția interioară e tratată ca răspuns la întrebarea R[relatat], iar cine vorbește, la R[cine spune]:

S*(Maria spune că P) = R[cine spune] ⊗ (f[Maria] + m)
+ R[relatat] ⊗ S*(P)

Pentru că înmulțirea distribuie peste adunare, R[relatat] ⊗ S*(P) înseamnă că fiecare pereche din P e înmulțită cu încă un vector: răspunsul la „cui" din propoziția interioară stă acum la adresa R[relatat] ⊗ R[cui]. Două relatări una în alta: R[relatat] ⊗ R[relatat] ⊗ R[cui]. Adâncimea e numărul de factori R[relatat].

Decodarea la adâncimea k: x = conj(R[relatat])^k ⊗ conj(R[r]) ⊗ S — aceeași linie de la pasul 6, cu k inversări în plus. Adâncimea însăși se estimează încercând k = 0, 1, 2, … și păstrând valoarea la care răspunsurile ies cel mai clar.

De ce compune la o adâncime pe care n-a văzut-o: codorul nu are nimic special pentru relatare. Cuvântul „că" are categoria lui și, cum arată pasul 8, e un *deschizător*: adresa fiecărui cuvânt de după el se calculează pornind de la adresa lui „că", înmulțită cu cheia perechii K[CĂ→categoria cuvântului]. Al doilea „că" dintr-o relatare în relatare e el însuși un cuvânt de după primul, deci adresa lui e adresa primului înmulțită cu cheia perechii K[CĂ→CĂ]. Învățarea (pasul 5), pe propoziții cu una și două relatări, împinge această cheie să fie egală cu R[relatat]: fiecare „că" în plus înmulțește adresa cu încă un R[relatat]. Atunci, într-o propoziție cu trei relatări, al treilea „că" aplică aceeași cheie încă o dată, iar cuvintele interioare ajung la R[relatat]^3 ⊗ …, exact adresa pe care o cere lumea. Nimic nou de învățat, cu o condiție: perechea K[CĂ→CĂ] trebuie să fi apărut la antrenare, adică două

relatări una în alta trebuie să fi fost văzute; „adâncime nevăzută” înseamnă mereu încă o repetare a unui salt văzut, nu un salt nou (capitolul 8, „Nivelul nemaivăzut”). Asta măsurăm când spunem „compune la o adâncime nevăzută”.

Pasul 8. Punctul de referință: deschizătorii

Adresa de la pasul 3 e produsul *tuturor* cheilor de la începutul propoziției. E suficient pentru propoziții simple, dar într-o propoziție lungă adresa unui cuvânt ajunge să depindă de tot ce a fost înainte, inclusiv de lucruri fără legătură cu el. Așa că adresa se calculează și **de la un punct de referință**: ultimul cuvânt „deschizător” de dinaintea lui.

Fiecare categorie are, pe lângă cheie, un **număr între 0 și 1, învățat**, care spune dacă acea categorie *deschide*: dacă un cuvânt din acea categorie devine noul punct de referință pentru cuvintele de după el. Învățat, nu dat: iese din același obiectiv de la pasul 5. Pe lumile noastre a ieșit aproape mereu același tipar: „că”, prepozițiile și cuvintele de legătură deschid; numele, obiectele și verbele nu.

Cu deschizători, adresa unui cuvânt e: adresa deschizătorului de dinaintea lui, înmulțită cu o cheie care depinde de perechea (categoria deschizătorului, categoria cuvântului). Cheia e tot pe categorii — nimic pe cuvânt. În „Paul îi dă Annei o carte”, „îi” deschide, deci Annei are adresa adresa(îi) ⊗ K[ÎI→NUME], iar „dă”, „o” și „carte” la fel, fiecare cu cheia perechii lui; dacă niciun cuvânt dinainte n-ar deschide, adresa s-ar lua de la începutul propoziției. Un deschizător la nivel de propoziție, „că”, face exact ce descrie pasul 7: tot ce urmează pornește de la adresa lui.

Un detaliu de implementare care contează: ca numerele „deschide / nu deschide” să poată fi învățate prin gradient, adresa unui cuvânt e, în timpul antrenării, o medie ponderată peste toți deschizătorii posibili de dinaintea lui, cu ponderi date de acele numere. După antrenare numerele ajung aproape de 0 sau de 1 și media devine o alegere. Pentru trei categorii (prepozițiile „to”, „by”, „from” din lumea de referință; lumile măsurate sunt în engleză, 1.3) am fixat noi deschiderea la 1, după ce o versiune învățată a ales altfel și o analiză a arătat de ce greșea; e singura alegere pe care am făcut-o de mână, e declarată, și coincide cu ce învățase singur modelul pe o lume mai simplă.

Pasul 9. Trei convenții de adresă, și cuvântul care vine prea devreme

Codorul nu alege o singură convenție de adresă; folosește trei deodată și le adună:

1. **produsul tuturor cheilor de la început** (pasul 3) — robust, dar nu știe de niveluri;
2. **cheia perechii (categoria dinainte, categoria curentă)** — locală, simplă, nu știe nimic de departe;
3. **de la ultimul deschizător** (pasul 8) — cea care compune pe niveluri.

Fiecare cuvânt e adunat în S de trei ori, la trei adrese. La citire, cele trei contribuții se întăresc acolo unde sunt de acord și se anulează acolo unde una greșește; asta e „votul” de care vorbește capitolul 15. Costul e de trei ori mai mulți termeni în S, acoperit de dimensiune.

Rămâne o problemă pe care niciuna din convenții n-o rezolvă singură: **subiectul vine înaintea verbului**. Când codorul citește „Paul”, nu știe încă dacă Paul dă sau primește; cheia care l-ar duce la R[cine dă] sau la R[cui] depinde de verbul care *urmează*. Soluția: un cuvânt care n-are încă deschizător înaintea lui primește adresa primului deschizător de *după* el, înmulțită cu o cheie pe perechea (categoria acelui deschizător, categoria cuvântului). Paul e astfel așezat de verb, sau de prepoziția de după verb, oricare deschide primul. Cheia folosită aici vine dintr-un tabel separat de cel al convenției 3: așezarea *înapoi*, prin deschizătorul de după, are cheile ei pe perechi de categorii, diferite de cele pentru cuvintele de după un deschizător; altfel Paul (așezat înapoi de „îi”) și Annei (așezată înainte de același „îi”) ar primi aceeași adresă. Acesta e mecanismul numit în poveste „ținut deoparte”.

Și aici e o limită pe care a găsit-o verificarea adreselor (regula: ținte diferite cer adrese diferite, înainte de orice antrenare): se folosește doar *primul* deschizător de după cuvânt, deci cu porți tari subiectul din „Paul **trebuie să nu** dea cartea” ar avea aceeași adresă ca în „Paul trebuie să dea cartea”. Am măsurat-o pe propoziții redate: porțile operatorilor învață să se deschidă parțial, adresa subiectului devine un amestec, iar cadrul iese corect, inclusiv la o adâncime nevăzută, pentru că structura (ce operatori sunt și ce acoperă) o poartă cuvintele de după operatori, iar recunoașterea identității are nevoie doar de un cosinus pozitiv cu cheia corectă. Costul e în primul rând pe adresă: luat singur, termenul subiectului ar vota uneori pentru cealaltă propoziție din pereche; în cadru, costul măsurat e o singură formă („poate să nu”, la adâncimea nevăzută), care iese sub criteriu. Remediul natural, așezarea prin toți deschizătorii care urmează, oglinda a ceea ce convenția 3 face spre stânga, rămâne o ipoteză: o schimbare a codorului, de construit doar dacă o lume are nevoie de subiect ca martor al structurii.

Pasul 10. Operatorii, și ce nu poate face comutativitatea

Negația, modalitatea („trebuie”, „poate”), condiționalul și relatarea sunt tratate la fel în ținta lumii: o întrebare înmulțită cu starea pe care o acoperă.

S*(nu P) = R[negație] ⊗ S*(P)
S*(trebuie P) = R[obligatoriu] ⊗ S*(P)
S*(dacă P atunci Q) = R[condiție] ⊗ S*(P)
+ R[consecință] ⊗ S*(Q)
S*(Maria spune că P) = R[cine spune] ⊗ (f[Maria] + m)
+ R[relatat] ⊗ S*(P)

Negația unui singur cuvânt e aceeași regulă aplicată cuvântului: „fișierele nesalvate” pune în răspunsul la R[ce] suma f[fișiere] + R[negație] ⊗ f[salvate]. La citire, decodorul încearcă ipotezele posibile (fără negație / negație pe toată propoziția / pe un cuvânt / amândouă), construiește starea pe care fiecare ar da-o și o alege pe cea mai apropiată de S (e același lucru cu „a muta harta cu negația și a vedea ce reapare” din capitolul 8, făcut pentru fiecare ipoteză deodată); raportează separat *dacă* există negație și *ce* acoperă. *Cuantificatorii*, cuvintele care spun la câte lucruri se referă un substantiv („fiecare”, „câteva”, „un anumit”, „niciun”), sunt tratați local: lumea cere ca substantivul lor să fie desenat a doua oară, la adresa lui înmulțită cu încă o întrebare, R[deasupra] pentru cuvintele care impun o citire

(„fiecare în parte”, „un anumit”) și R[cuantificat] pentru cele care o lasă deschisă („fiecare”, „câteva”); articolele („o”, „un”) nu adaugă nimic. Codorul produce acest al doilea termen cu o cheie relativă învățată a categoriei cuantificatorului, aplicată adresei substantivului (același mecanism ca balizele de la pasul 8). Codorul nu calculează care din două fraze cuantificate e „deasupra”; lumea deduce asta din cei doi termeni, cu o regulă: un termen R[deasupra] prezent → acea frază e deasupra, cealaltă dedesubt; un singur R[cuantificat] → singurul cuantificator; doi R[cuantificat] și niciun R[deasupra] → *ambele citiri*, raportate ca atare (capitolul 8, „Ambiguitatea ținută deschisă”). Codorul desenează, lumea raționează.

Ce nu poate reprezenta această algebră, prin construcție: **ordinea a doi operatori aplicați aceleiași stări**. Pentru că înmulțirea comută, R[obligatoriu] ⊗ R[negație] ⊗ S*(P) și R[negație] ⊗ R[obligatoriu] ⊗ S*(P) sunt același vector — „trebuie să nu” și „nu trebuie să” ar da aceeași stare dacă nimic altceva n-ar diferi. În practică aproape mereu diferă altceva: în „Maria nu spune că P” față de „Maria spune că nu P”, răspunsul la R[cine spune] poartă negația doar în prima; acel termen-martor deosebește cele două stări, și le-am măsurat deosebite. Unde nu există martor, ordinea e pierdută, și declarăm asta ca limită: o adresă ține minte *ce* chei au fost folosite, nu *în ce ordine*. Remediile cunoscute — permutări fixe ale componentelor sau câte o întrebare separată pe nivel — cumpără ordinea cu o parte din libertatea de la pasul 7, și nu le folosim; remediul pe care îl urmărim e tot aditiv: fiecare operator să-și lase și identitatea la nivelul lui, ca relatarea, astfel încât doi operatori în ordini diferite să lase martori diferiți.

Dicționar: povestea, tradusă

În poveste	În acest capitol
un punct din cer; cele mii de rotițe	un vector de D unghiuri
„aproape” / „fără legătură”	cos mare / aproape zero
steaua unui cuvânt; catalogul	identitatea f[w]; mulțimea tuturor identităților
a muta un punct cu un cod; cheia inversă	înmulțirea ⊗; conj(·)
sonda; poziția raportată	adresa curentă a
carnetul de rute	cheile K[c], pe categorie
a desena steaua mutată cu poziția sondei	a ⊗ f[w]
harta; a topi; constelația	starea S, suma termenilor (centrul de greutate al stelelor desenate, pe fiecare componentă, înmulțit cu numărul lor)
legenda; steaua cu nume a unei întrebări	R[r]
domeniul unei întrebări	mulțimea {R[r] ⊗ f[w]} pentru toate cuvintele w
scanerul; marja	căutarea celui mai apropiat f[w]; diferența primilor doi cosinusuri
„relatat”; originea mutată; nivelul	R[relatat]; înmulțirea cu R[relatat] de k ori
poarta; deschizătorul	numărul învățat 0…1 pe categorie (pasul 8)

În poveste	În acest capitol
baliza; ținut deoparte	adresa de la deschizător; așezarea prin deschizătorul de după (pasul 9)
cele trei desene; votul	cele trei convenții de adresă, adunate
constelația lumii	S*
markerul (de ocupare)	m
restul	$\|S - \hat{S}\| / \|S\|$, față de valoarea obișnuită a lumii
drumurile fac o grilă, nu un arbore	comutativitatea lui ⊗ (pasul 10)

Ce e standard, ce e al nostru

Standard, cu zeci de ani de literatură în spate: vectori de unghiuri cu înmulțirea ca adunare de unghiuri (reprezentările holografice ale lui Plate, 1995; calculul hiperdimensional al lui Kanerva; codarea cu puteri fracționare a lui Frady, Kleyko și Sommer), codarea unei structuri ca sumă de perechi întrebare ț răspuns, rea cerea prin înmul mai apropiat vecin, limitele de capacitate ale unei sume de termeni aleatori. Nu am inventat algebra.

Al nostru, și măsurat pe lumi construite și pe un set real de întrebări: că adresele pot fi *învățate* ca produse de chei pe categorie, dintr-un singur obiectiv (pasul 5), astfel încât formulările diferite ale aceluiași eveniment ajung la aceleași întrebări fără ca cineva să le fi etichetat ca parafraze; că adunarea a trei convenții de adresă face asta robust la formulări nevăzute; că adresa de la deschizător cu așezarea prin deschizătorul următor compune cu un nivel mai departe decât a fost antrenată (pasul 7); că deschizătorii pot fi învățați și ies la fel pe lumi diferite; că operatorii tratați ca întrebări înmulțite cu starea sunt citiți cu ce acoperă, fără reguli; și că aceeași stare dă, prin marje, ocupare și rest, o relatare utilizabilă a ceea ce cititorul n-a înțeles. Ce rămâne în afara acestei liste e în STATUS.md, cu criteriul și logul fiecărei măsurători.

14. Din ce se poate face cerul: variante de reprezentare, codul rar și creierul

Povestea a fost spusă într-un singur material: puncte cu coordonate pe mii de rotițe, adică vectori de faze. Nu e singurul material posibil. Familia de reprezentări din care facem parte, arhitecturile vector-simbolice, a construit aceleași trei operații, a muta, a topi, a recunoaște, în mai multe feluri, iar alegerea între ele e o decizie de inginerie cu consecințe pentru cost, hardware și apropierea de ce se știe despre creier. Capitolul acesta spune care sunt variantele, de ce am ales-o pe a noastră, ce ar aduce codul rar, de ce se crede că în creier codul e rar, și, mai ales, ce parte din ce am construit depinde de material și ce parte nu.

Aceleași trei operații, mai multe materiale

Tot ce cere povestea de la un material e: puncte alese la întâmplare să fie fără legătură între ele; mai multe puncte să poată fi topite într-unul singur din care fiecare să rămână

recunoscibil; și o operație de mutare care păstrează forma și se desface exact. Fiecare familie le realizează altfel.

- *Vectori de semne* (+1 și −1 pe mii de poziții): mutarea e înmulțirea poziție cu poziție, care e propriul ei invers; topirea e un vot majoritar pe fiecare poziție. Simplu, binar, ieftin; mutarea e exactă doar aproximativ după topire, pentru că votul rotunjește.
- *Vectori reali cu convoluție circulară* (reprezentările holografice reduse ale lui Plate): mutarea e convoluția, inversul e aproximativ (corelația), topirea e adunarea. Istoric prima variantă completă; inversul aproximativ costă precizie la adâncimi mari.
- *Faze complexe* (varianta noastră, numită și holografică în domeniul frecvenței): fiecare poziție e un unghi; mutarea e adunarea unghiurilor, inversul e scăderea lor, exact; topirea e adunarea săgeților. E convoluția lui Plate scrisă în domeniul Fourier, unde devine exactă și ieftină.
- *Coduri rare pe blocuri*: fiecare poziție e un bloc de L unități din care e activă una; mutarea e deplasarea unității active în bloc; topirea e suprapunere urmată de competiție în bloc. Echivalentul rar al fazelor, pe care îl descriem mai jos.
- *Coduri binare rare cu subțiere*: puține unități active din foarte multe; topirea cere o operație care păstrează raritatea; mutarea e o operație dependentă de context. Cele mai apropiate de reprezentările rare din neuroștiință, cele mai departe de învățarea cu gradient.
- *Legare prin permutări sau matrici*: mutarea e o permutare a pozițiilor sau o înmulțire cu o matrice; nu comută, deci păstrează ordinea, cu prețul discutat în capitolul 8 la „Ordinea”.

Ce au în comun toate: separarea prin aleator, suprapunerea, inversabilitatea. Unde diferă: cât de exact e inversul, câte lucruri încap într-o hartă la același număr de poziții, cât costă o operație, și cât de ușor se poate învăța o mutare cu gradient.

De ce am ales fazele

Patru motive, în ordinea importanței. Inversul e exact, deci citirea unui domeniu la orice adâncime nu acumulează erori de material, ci doar zgomotul suprapunerii. Mutările sunt continue, deci carnetul de rute se învață cu gradient, pas cu pas, ceea ce a făcut posibil tot capitolul 6. Aceeași codare ar servi și mărimi continue: un număr, o poziție, un moment pot fi faze cu unghi proporțional cu mărimea, astfel încât „40” și „41” să fie stele apropiate fără să fie identice; e o extensie neconstruită, azi numerele sunt cuvinte cu stele aleatoare (9.1). Și, al patrulea, fazele sunt exact modelul matematic prin care literatura de calcul hiperdimensional a descris integrarea de drum a celulelor grilă, ceea ce leagă direct povestea de teoria din capitolul următor.

Prețul e că fazele sunt dense: fiecare rotiță are mereu o valoare, și o hartă e un șir de mii de numere reale. Pe un procesor obișnuit nu contează; pe hardware foarte simplu, sau ca model al cortexului, contează.

Codul rar: același cer, scris în neuroni

Reprezentarea densă cu faze are un echivalent rar, și echivalența e exactă, nu o analogie: o rotiță cu L poziții e un bloc de L unități din care e activă una singură, cea de la unghiul rotiței. Mutarea devine deplasarea unității active în interiorul blocului (adunare modulo L); topirea devine suprapunere, mai multe unități active în bloc, urmată de o competiție care păstrează cele mai active k; scanerul rămâne același. Familia e cunoscută (coduri rare pe blocuri), iar echivalența cu fazele trece prin transformata Fourier.

Ce ar aduce. Aritmetică pe indici întregi în loc de numere complexe, deci implementare directă pe microcontrolere sau circuite programabile, exact ce cere interlock-ul din capitolul 10. Memorie de ordinul biților pe rotiță (log L biți) în loc de zeci. O normalizare naturală: competiția în bloc e tocmai ce lipsește hărții dense, care la multe stele alunecă spre „mijloc”; și, interesant, competiția păstrează reuniunile de ipoteze: două unități active într-un bloc înseamnă „ori una, ori alta”, adică exact reuniunea de ipoteze din teoria lui Hawkins, apărută de la sine. Și o raritate de ordinul procentului, ca în cortex.

Ce ar costa. Unghiurile se cuantizează la L poziții, deci mărimile continue au granularitate 360/L; la L de ordinul zecilor sau sutelor probabil nu doare, dar e un parametru în plus. Antrenarea rămâne pe faze continue, cuantizarea făcându-se la sfârșit; cât se pierde se măsoară, și e unul dintre experimentele planificate. Capacitatea e comparabilă ca ordin de mărime, dar detaliile diferă cu L și k.

De ce se crede că în creier codul e rar

Trei linii de dovezi, cu rezervele lor. Întâi, bugetul energetic: un neuron care descarcă consumă mult, iar estimările clasice arată că cortexul nu-și poate permite decât o fracție mică de neuroni activi în același timp, de ordinul procentelor; un cod dens ar fi nesustenabil metabolic. Al doilea, codarea rară ca principiu de calcul: lucrările lui Olshausen și Field au arătat că un sistem care învață să reprezinte imagini naturale cu cât mai puține unități active deodată regăsește de la sine câmpurile receptive observate în cortexul vizual; e unul dintre cele mai reproduse rezultate din neuroștiința computațională. Al treilea, structura: modulele de celule grilă funcționează ca atractori în care e activ un singur „ghem” de celule, a cărui poziție codifică faza, adică exact un bloc rar cu o unitate activă; iar teoria din capitolul următor pune reprezentările rare la baza întregului neocortex.

Rezervele, la fel de clar: există arii și regimuri cu activitate densă; dezbaterea despre cât contează rata de descărcare față de sincronizarea descărcărilor nu e închisă; și „rar” e o constatare despre activitate, nu o dovadă că algebra folosită de creier e una dintre cele de mai sus. Spunem deci „se crede, cu dovezi consistente”, nu „se știe”.

Unde e inovația și unde e ingineria

Algebra, oricare dintre variante, e standard și veche de treizeci de ani; nu revendicăm nimic din ea. Ce am construit stă *deasupra* materialului și nu depinde de el: deplasările învățate pe fel de cuvânt, cele trei desene ale aceluiași cuvânt, porțile care decid cine pune balize, lumea ca țintă de antrenare, operatorii ca roluri în jurul scopului lor, citirea cu marjă și

rest. Fiecare dintre acestea se scrie la fel pe faze, pe blocuri rare sau pe semne; ar fi aceeași poveste, cu alt cer. Pentru cine evaluează proiectul, asta e un argument de robustețe: rezultatele nu atârnă de o alegere de material, iar materialul se poate schimba când ținta de hardware o cere.

Ingineria e alegerea materialului pentru țintă: numărul de rotițe, L și k pentru codul rar, cuantizarea, maparea pe un circuit. Și cercetarea viitoare e tot aici, cu întrebări precise: dacă antrenarea direct pe cod rar (nu doar cuantizarea la sfârșit) păstrează rezultatele; dacă competiția în bloc dă reuniunea de ipoteze cu aceeași precizie ca suprapunerea densă; dacă legarea necomutativă se poate adăuga doar operatorilor, păstrând compoziția liberă pentru rest; și cât din capacitatea unei hărți se recuperează prin structuri de hărți în loc de rotițe în plus. Niciuna nu schimbă povestea; fiecare schimbă cât de ieftin și cât de aproape de creier poate fi spusă.

15. Paralela cu *O mie de creiere*

Povestea de mai sus n-a apărut din nimic. Punctul de plecare a fost teoria lui Jeff Hawkins despre neocortex, expusă în *A Thousand Brains* (2021) și în lucrările grupului Numenta dinaintea ei. Datorăm o punere față în față cinstită: ce am luat, ce am tradus, ce avem în altă formă și ce n-avem deloc. E și instructiv, pentru că fiecare piesă a poveștii are un corespondent în teorie, iar diferențele spun exact ce fel de lucru am construit.

Ce spune teoria, în cinci propoziții

Neocortexul e făcut din zeci de mii de *coloane corticale*, toate cu aceeași structură. Fiecare coloană învață modele complete ale obiectelor din lume, nu fragmente. Un model e un *cadru de referință* ancorat pe obiect, în care trăsăturile sunt stocate la *locații*: a cunoaște o cană înseamnă a ști ce se simte la fiecare loc de pe ea. Locația curentă e ținută de celule asemănătoare celulelor grilă din hipocamp și e actualizată prin *mișcare*: când degetul se mută, locația se mută cu el, înainte ca orice senzație să sosească (integrare de drum). Coloanele, fiecare cu dovezi parțiale, *votează* prin legături lungi și converg rapid pe un singur obiect; cât nu converg, țin deschisă o reuniune de ipoteze. Și, afirmația cea mai îndrăzneață: același mecanism, cadre de referință plus mișcare, servește și pentru concepte și limbaj, unde „mișcarea” e abstractă.

Dicționarul

În teoria lui Hawkins	În poveste	În capitolul 13
cadrul de referință al unui obiect	cerul, cu legenda ca ancore	toți vectorii de D unghiuri; întrebările R[r]
o locație în cadru; celulele grilă	un punct din cer; fiecare rotiță e faza unui modul de grilă	un vector de D unghiuri (pasul 1)
mișcarea; integrarea de drum	saltul sondei; poziția = suma cheilor	înmulțirea cheilor: a = a ⊗ K[c] (pasul 3)
o trăsătură stocată la o locație	steaua unui cuvânt mutată cu poziția sondei	a ⊗ f[w]

În teoria lui Hawkins	În poveste	În capitolul 13
modelul unui obiect	constelația unui eveniment	starea S
recunoașterea obiectului	compararea constelațiilor; cadrul citit prin legendă	cos(S, S*); conj(R[r]) ⊗ S și căutarea celui mai apropiat f[w] (pașii 5–6)
celulele de deplasare; obiect în obiect	steaua „relatat”; legenda mutată la altă origine	înmulțirea cu R[relatat] (pasul 7)
predicția următoarei senzații	balizele: unde va cădea următorul cuvânt de un anumit fel	adresa de la deschizător, cu cheile perechilor de categorii (pasul 8)
votul între coloane	cele trei desene ale aceluiași cuvânt, topite; scanerul	suma celor trei adrese (pasul 9); căutarea cu marjă (pasul 6)
reuniunea de ipoteze	cuvântul ținut deoparte, așezat de ce urmează; ambiguitatea raportată ca „ambele”	așezarea prin deschizătorul de după (pasul 9); doi termeni de cuantificator în S (pasul 10)
surpriza; ce nu se potrivește cu modelul	stelele în afara legendei	restul $\|S - \hat{S}\| / \|S\|$ (pasul 6)
mii de coloane	un singur pilot	un singur model

Ce am luat literal

Două lucruri sunt luate ca atare, nu ca metaforă. Întâi, *locația ca fază*. Celulele grilă codifică o poziție ca un set de faze, câte una pe modul, iar mișcarea adună faze; codarea cu puteri fracționare pe care o folosim e exact acest mecanism scris ca vectori, și a fost propusă în literatura de calcul hiperdimensional tocmai ca model al celulelor grilă și al integrării de drum. Rotițele din poveste corespund, în modelul matematic, modulelor de grilă; e o corespondență formală, nu o afirmație despre biologie. Al doilea, *cunoașterea ca trăsături la locații*. Harta unei propoziții nu stochează cuvintele, ci cuvintele-la-poziții, și două propoziții sunt același obiect când au aceleași trăsături la aceleași locații, indiferent de drumul senzorului. Asta e definiția unui obiect la Hawkins, aplicată la evenimente.

A treia preluare e cea pe care am descoperit-o abia pe drum: *compoziția prin deplasare*. Hawkins propune „celule de deplasare” care țin vectorul dintre două locații, ca să poată pune un obiect în cadrul altuia: sigla pe cană e cadrul siglei, așezat la o deplasare în cadrul cănii. Propoziția relatată din „Maria spune că…” e exact asta: cadrul propoziției interioare, așezat la deplasarea „relatat” în cadrul celei exterioare. Faptul că pilotul citește trei niveluri după ce a învățat două e proprietatea pe care teoria o prezice pentru deplasări: o deplasare se aplică de oricâte ori.

Unde e votul

Întrebarea cea mai bună pe care ne-a pus-o cineva despre poveste a fost „unde e votul?”. În teorie, votul e între coloane: fiecare coloană are o ipoteză despre ce obiect simte, ipotezele

se trimit una alteia, și mulțimea candidaților se îngustează până rămâne unul. Noi n-avem coloane; avem un singur pilot. Dar votul există, în două locuri mai mici.

Primul: *cele trei desene*. Pilotul are trei feluri de a afla unde e un cuvânt, din trei surse de dovezi diferite, cu moduri de a greși diferite: drumul parcurs (suma tuturor cheilor de la începutul propoziției), tranziția locală (ce fel de cuvânt a fost chiar înainte) și așteptarea deschisă de un cuvânt anterior (baliza). Fiecare e o „coloană" care spune „cuvântul e aici". Nu le lăsăm să discute; le topim. Topirea e un vot prin suprapunere: unde cele trei sunt de acord, săgețile se adună și steaua e netă; unde una greșește, celelalte două o acoperă și steaua e doar puțin încețoșată. E un vot fără dezbatere, într-o singură rundă, și tocmai de aceea e ieftin.

Al doilea: *scanerul*. Când harta e mutată înapoi cu o stea cu nume, ce rezultă e comparat cu toate stelele din catalog deodată, și cea mai apropiată câștigă, cu o marjă. E un vot între candidați, nu între coloane, dar are aceeași formă: toți concurează, unul iese, iar diferența față de al doilea spune cât de convins e votul. Iar când nu iese unul, când doi candidați sunt la fel de aproape, pilotul nu forțează; raportează amândoi. Asta e *reuniunea de ipoteze* din teorie, ținută deschisă până sosește o dovadă care o închide: așa citim „fiecare copil citește câteva cărți" cu ambele înțelesuri deodată, și așa stă primul cuvânt pe jumătate în două domenii până sosește verbul.

De ce doar trei

Pentru că trei e numărul surselor de dovezi *diferite* pe care le-am găsit, nu un număr ales. Un vot folosește doar dacă votanții greșesc în feluri diferite; trei copii ale aceluiași calcul nu adaugă nimic. Drumul parcurs greșește când o cheie din șir e greșită și duce tot ce urmează aiurea; tranziția locală nu greșește aproape niciodată, dar e grosolană, pentru că nu știe decât ce a fost chiar înainte; baliza e precisă, dar lipsește când construcția e nouă și nimeni n-a anunțat-o. Cele trei se acoperă reciproc, și măsurătoarea arată cât: construcțiile nemaivăzute se citesc nouă din zece cu toate trei, șase din zece cu una singură.

Am încercat și o a patra sursă: locul cuvântului în propoziție, al treilea, al cincilea. A adus puțin în plus pe construcțiile noi, și totuși am renunțat la ea. Argumentul de principiu e că leagă citirea de locul cuvântului în șir, adică exact de lucrul pe care parafrazele îl schimbă; am remăsurat-o apoi cu ambele variante pe aceleași teste, și urma a confirmat argumentul: câștigul vine din formulările noi care așază un rol unde îl așezau cele vechi, iar pierderile din cele care nu, cu un cost în plus de o treime la termenii unei propoziții; rămâne o opțiune declarată pentru lumi cu ordine rigidă, nu parte a referinței. O a cincea sursă ar trebui să fie din nou un fel diferit de a greși, și n-am găsit-o încă. Dacă apare, se adaugă ca o nouă săgeată la topire, fără să se schimbe altceva: acesta e avantajul votului prin suprapunere.

Există și un răspuns mai larg la „de ce nu mii". În teorie, miile de coloane primesc fiecare altă bucată din lume: alt deget, alt petic de retină. În text, intrarea e un singur șir de cuvinte, și nu există mii de senzori independenți care să-l simtă din unghiuri diferite. Versiunea cu adevărat „cu o mie de creiere" ar avea mulți piloți care citesc același text cu catalogul literelor, cu cel al cuvintelor, cu cel al grupurilor de cuvinte, sau mai multe simțuri deodată,

cum spune 9.4, și ar vota între ei. E un pas de făcut; cele trei desene sunt forma ei cea mai mică, într-un singur pilot.

Unde ne despărțim, și de ce

Mișcarea e ghicită, nu dată. În teorie, locația se actualizează din mișcarea senzorului, pe care creierul o cunoaște pentru că el a comandat-o. În text, nimeni nu ne spune cât „ne mișcăm" la fiecare cuvânt; saltul trebuie dedus din felul cuvântului, și asta e tot ce învață pilotul. Limbajul e cazul greu al teoriei, nu cel ușor; oriunde mișcarea e dată direct, de un senzor, de o comandă motorie, de timp, mașinăria se aplică fără partea învățată.

Învățarea e de la lume, nu din predicție. Coloanele lui Hawkins învață prezicând următoarea senzație, fără profesor. Pilotul nostru învață de la constelația lumii: i se spune cum trebuie să arate harta, și își ajustează cheile ca să ajungă acolo. Am încercat varianta cu profesor zero, prezicerea următorului cuvânt, și domeniile au dispărut: nimic n-a obligat pilotul să separe rolurile. Nu afirmăm că în creier e altfel; afirmăm că în textul fără lume, predicția singură nu ne-a dat cadre de referință cu roluri. E o diferență de regim, nu de mecanism, și o declarăm ca atare.

Cadrul e dat, nu învățat. La Hawkins, coloana învață ea însăși cadrul obiectului. La noi, stelele cu nume ale legendei vin de la lume; ce învață pilotul e drumul spre ele și cine deschide cadre (porțile). Am verificat că felurile de cuvinte nu apar de la sine din obiectivul nostru, și am retras afirmația contrară; porțile, în schimb, apar.

Un singur drum înainte, fără memorie între propoziții. Teoria are coloane care integrează în timp și se întorc asupra ipotezelor; pilotul merge înainte, o propoziție, o hartă, și uită. Structura anunțată după și referința pronumelor sunt exact locurile unde se vede lipsa.

Substratul. Reprezentările din teorie sunt rare și binare; ale noastre sunt dense, faze pe mii de rotițe. Proprietățile care contează pentru poveste (două puncte aleatoare n-au legătură, suprapunerea ține mai multe lucruri, mutarea păstrează forma) sunt comune celor două familii; detaliile de capacitate diferă.

Ce rămâne, după ce scădem diferențele, e afirmația pe care am testat-o și care ține: un cadru de referință cu locații date de mișcare și trăsături depuse la locații, cu compoziție prin deplasare și vot prin suprapunere, e suficient pentru ca propoziții diferite despre același eveniment să ajungă la aceeași hartă, pe lumi construite și pe un set real de întrebări, și pentru ca harta să spună unde n-a ajuns. Restul teoriei, miile de coloane, învățarea fără profesor, cadrele învățate, e ce urmează, și e lista de pe masă.

16. Alinierea și problemele din jurul ei: ce se poate spune inginerește

Prin *aliniere* înțelegem aici ceva modest și verificabil: un sistem care primește cereri în limbaj să facă ce i s-a cerut, numai ce e permis, și să se vadă limpede când n-a înțeles sau când a ieșit din ce e permis. Nu propunem o soluție la alinierea modelelor mari de limbaj; nu le atingem interiorul și nu pretindem că l-am putea atinge. Ce avem e altceva: o *interfață verificabilă* între limbaj și acțiune, și câteva măsurători care devin posibile de îndată ce sensul e definit ca în 1.1, un set de răspunsuri la întrebări știute dinainte. Capitolul spune,

pentru fiecare problemă, ce mecanism avem; ce e construit și ce e în lucru din fiecare se citește în fișierul de stare al proiectului (https://github.com/ml-circuits-research/vsa-semantic-space/blob/main/STATUS.md).

Ideea de bază: legenda ca graniță

Un sistem care acționează în lume primește cereri în limbaj natural și produce acțiuni: o interogare, o comandă, un răspuns, o modificare într-o bază de date. Cele mai multe probleme de aliniere trăiesc la această trecere: cererea e înțeleasă altfel decât a fost spusă, acțiunea e alta decât cea cerută, sau răspunsul sună bine și e fals. Propunerea inginerească e ca trecerea să nu se facă direct. Orice cerere și orice acțiune propusă de modelul mare trec prin pilot, care le transformă în răspunsuri la întrebările unei *legende de acțiune*: ce acțiune? asupra cui? cu ce parametri? sub ce condiție? în numele cui? Legenda are câteva zeci de rânduri, pe care un om le citește și le aprobă (cine a scris primul rând, un om sau un model, nu schimbă nimic aici), și tot ce nu încape în ea nu e reprezentabil, deci nu poate ajunge la execuție. Nu e un filtru care caută cuvinte interzise; e o formă în care anumite lucruri nu pot fi spuse deloc.

Cadrul citit se traduce mecanic în fapte și constrângeri (9.3), politicile sunt scrise ca reguli în același limbaj, și un motor de raționament decide dacă acțiunea e permisă, cu o derivare care se poate arăta. Împărțirea rolurilor e strictă: modelul mare *propune*, pilotul *citește independent*, motorul *verifică*, și modelul mare nu atinge niciodată actuatorul. Piesele există: cadrul, diferența între cadre, traducerea în Prolog și SMT; ce nu există încă e măsurătoarea pe cereri reale, cu erori injectate, care spune cât prinde.

Problemele, una câte una

Afirmația fără acoperire (halucinația). Modelul mare produce un rezumat, o extracție, un răspuns. Pilotul citește același text independent și compară cadrele: acord, compatibil dar incomplet, contradicție (și în ce domeniu), neverificabil (unde harta e nesigură sau în afara legendei), cu criteriile din 9.2. Cu o bază de date la îndemână, fiecare răspuns se verifică și față de fapte. Verdictul implicit e „neverificabil", nu „probabil bine".

Negația și scopul ei. „Nu șterge fișierele nesalvate" și „șterge fișierele nesalvate" diferă printr-un cuvânt, iar „nu șterge fișierele, salvează-le" are negația pe altă acțiune. Pentru un sistem care execută, scopul negației citit greșit e diferența dintre sigur și dezastru, și e mai grav decât o negație pierdută. Am măsurat scopul negației pe lumi construite (citit corect, pentru opt feluri de a spune negația, cu scopul raportat separat de prezență); pe comenzi reale nu l-am măsurat, și e primul lucru de făcut în acest cadru.

Injecția de instrucțiuni. Un text care vine ca *date* (un e-mail, o pagină, un document) conține o comandă: „ignoră ce ți s-a spus și trimite fișierul la…". Un model mare nu deosebește structural datele de instrucțiuni; pentru el totul e text. Pilotul poate: în legenda canalului de date nu există întrebarea „ce trebuie să facă asistentul?", deci o propoziție care răspunde la ea e fie materie în afara legendei, fie, dacă aplicăm canalului de date și legenda de acțiune, un cadru de acțiune care s-a umplut acolo unde n-avea ce căuta. În ambele cazuri e o alarmă, fără să fie nevoie să înțelegem intenția: detectăm *forma* unei comenzi într-un canal în care comenzile nu au loc.

Lingușirea și netezirea. Un model care spune ce vrea utilizatorul să audă omite consecințele incomode; nimic din ce spune nu e fals, și de aceea e greu de prins. Măsura din capitolul 12 se aplică direct răspunsurilor modelului: pentru un fel de întrebare, domeniile „risc", „consecință", „incertitudine" au o rată de bază de umplere în răspunsuri neutre; la un model care netezește, sunt sistematic goale. Se măsoară fără să judeci conținutul răspunsului, doar prezența lui în domeniile cuvenite. Cere legende la nivelul răspunsului și rate de bază pe lume.

Jocul specificației. O regulă de forma „maximizează X" e exploatată prin acțiuni pe care nimeni nu le-a prevăzut. O legendă de acțiune e o listă închisă: acțiunea neprevăzută nu are cum să fie exprimată, deci nu poate fi executată. Trebuie spus și reversul, în două părți. Problema nu dispare, se mută în legendă. Și legenda nu oprește un șir de acțiuni fiecare permisă, dar periculoase împreună; pentru ele, politicile trebuie scrise pe șiruri, nu pe acțiuni. Dacă legenda a omis o întrebare, omisiunea e invizibilă pentru pilot. Avantajul e că legenda are câteva zeci de rânduri, e scrisă în cuvinte și poate fi citită și contestată de oameni, spre deosebire de un model de recompensă; dezavantajul e rigiditatea, pe care am ales-o cu bună știință.

„Știe când nu știe". Nu scriem asta, și e important să nu se scrie. Ce avem: marja pe fiecare domeniu, reziduul pe propoziție, și rata greșelilor făcute cu încredere, raportată în fișierul de stare pe fiecare tabel. Fără acest număr, abținerea nu e o garanție. Cu el, abținerea devine o politică măsurabilă: sub un prag, acțiunea nu se execută și întrebarea merge la om, iar cât de des se întâmplă asta și cât de des pragul lasă să treacă o greșeală sunt două curbe, nu două adjective.

Inspectabilitate și audit. Tot ce știe pilotul e un carnet cu câteva mii de chei pe feluri de cuvinte; fiecare decizie e o hartă care se citește domeniu cu domeniu; nu există memorie între propoziții, deci nu există o stare ascunsă care să derive de-a lungul unei conversații; faptele stau într-o bază de date unde pot fi corectate. Toate acestea sunt despre pilot, nu despre modelul mare, care rămâne la fel de opac. Ce facem transparent e *interfața*: locul unde limbajul devine acțiune.

Atacuri asupra pilotului însuși. Un adversar care cunoaște mecanismul poate încerca parafraze construite ca să se citească într-un cadru inofensiv, cuvinte necunoscute puse anume ca să rupă o cheie, propoziții aglomerate ca să înece un domeniu. Tot ce rupe citirea scade marja sau crește reziduul și cade în „neverificabil", care e implicitul sigur; dar un atacator care știe carnetul poate construi propoziții cu marjă mare și cadru greșit, și rata la care reușește trebuie măsurată, cu un atacator automat (un model mare pus să genereze parafraze adversariale), nu presupusă.

Ce nu rezolvă

Nu spune nimic despre obiectivele interne ale modelelor mari și nu le schimbă. Nu funcționează pe lumi deschise, fără legendă. O legendă greșită sau incompletă dă o verificare greșită sau incompletă, cu încredere. Și tot ce e mai sus se sprijină pe un număr raportat în fișierul de stare, rata greșelilor făcute cu încredere; el decide pragurile, nu arhitectura.

Rezumat

Problema	Mecanismul	Ce se măsoară
afirmații fără acoperire	citire independentă, diferență între cadre, verdict „neverificabil” implicit	precizie și acoperire pe erori injectate
negația și scopul ei	operatorii ca roluri; scopul raportat separat de prezență	scopul pe comenzi reale; greșeli cu încredere
injecția de instrucțiuni	cadru de acțiune apărut într-un canal de date; materie în afara legendei	alarme adevărate și false pe un canal de date
lingușire, netezire	domenii de risc/consecință sistematic goale față de rata de bază	rata de umplere față de rata de bază
jocul specificației	legendă de acțiune închisă; politici ca reguli; motor care decide	acoperirea legendei; mută problema în legendă
abținere	marjă, reziduu, rata greșelilor cu încredere, prag	curba acurateței–acoperire
audit	carnet mic, hartă lizibilă, fără stare între propoziții, fapte externe	lizibilitatea jurnalului, pe eșantion
atacuri asupra pilotului	implicit „neverificabil”; rată de reușită cu atacator automat	rata de reușită a atacatorului

17. O speculație disciplinată: un sistem care descoperă

Tot ce e în capitolele de până aici e fie măsurat, fie spus ca direcție cu un mecanism precis în spate. Capitolul acesta merge mai departe decât orice direcție de până acum și o spune de la început: nimic din el nu e construit, nimic nu e măsurat, și nu va fi până când nu există o lume construită pentru asta. E un exercițiu de inginerie pe hârtie: dacă ai vrea un sistem care *descoperă*, care propune ipoteze, le încearcă, își dă seama ce e nou, își revizuiește concluziile și învață din ele, ce ar avea de câștigat dacă ar fi construit în jurul hărții, și nu în jurul unui model mare singur? Unde o piesă există deja, spunem în ce capitol; unde nu există, spunem că nu există.

De ce în jurul hărții

Un sistem care descoperă are nevoie de patru lucruri pe care un model de limbaj nu le dă direct. O reprezentare în care *noutatea e măsurabilă*, nu simțită: a noastră are restul (capitolul 8) și regiunea de noutate (capitolul 12). O formă în care *o ipoteză și o observație pot fi comparate* bucată cu bucată: a noastră compară constelații domeniu cu domeniu

(9.2). O *teorie care încape într-un text scurt și editabil*: a noastră e legenda, câteva zeci de rânduri, plus lista felurilor de cuvinte. Și *un număr pentru ce nu știe*, pe fiecare întrebare: marja, ocuparea și restul. Modelul mare are, în schimb, ce lipsește hărții: limbă multă, cultură multă, capacitatea de a propune. Imaginea din 9.2 se aplică întreagă: partea fluentă propune, partea verificabilă validează, iar validările se întorc ca date.

Ipotezele sunt constelații

O *ipoteză* e, aici, o propoziție într-o lume cu legendă: „Enzima X încetinește reacția Y la temperatura Z." Citită de pilot, devine o constelație: X în domeniul agentului, Y în al procesului, „încetinește" în al acțiunii, Z în al condiției. O *observație* e tot o propoziție, venită din date (6.4, a doua sursă: structuri existente lângă text), și devine tot o constelație. Primul nivel al „testării" unei ipoteze e deci exact compararea din 9.2, cu cele patru verdicte: *acord*, *compatibil dar incomplet*, *contradicție* (în ce domeniu), *neverificabil*. Nimic din asta nu înțelege enzimele; totul e ieftin, repetabil și poate fi urmărit până la propoziție și la rândul din legendă. Ce nu dă acest nivel e cauzalitatea sau statistica unei confirmări; dă doar o contabilitate corectă a ce susține ce, și asta e exact ce lipsește de obicei în jurul unui model mare.

De unde vin ipotezele: trei surse

Din *modele mari*: ele propun, în limbaj; pilotul formalizează, iar ce nu încape în legendă iese marcat *neverificabil*. Marcajul acela e el însuși informație: o ipoteză pe care legenda n-o poate ține e fie prost formulată, fie cere o întrebare nouă, și a doua posibilitate e locul de unde pornesc conceptele noi, mai jos.

De la *oameni*: aceeași cale, cu propoziții tastate; sistemul nu face deosebire între o ipoteză venită de la un om și una venită de la un model, decât în proveniență, pe care o păstrează.

Din *observații*, dedusă: citirea unui corpus întreg de observații produce lista de cadre din capitolul 5, un graf de cunoștințe relativ la legendă. Regularitățile din acel graf sunt ipoteze candidate: „ori de câte ori X e în domeniul agentului, Y e în domeniul rezultatului". Aici harta ajută direct, pentru că constelațiile sunt puncte: centrul de greutate al tuturor constelațiilor de același fel (2.5) e constelația *tipică* a acelui fel de eveniment; o observație departe de el e o abatere, cu restul ei; o direcție în care constelațiile se abat *împreună* e o regularitate. Nu e nevoie să înțelegi X și Y ca să vezi că merg mereu împreună; e nevoie doar de legendă și de multe observații, exact ca la părtinire în capitolul 12.

Noutatea, și de unde vin conceptele noi

Noutatea e restul, și capitolul 12 a spus de ce: ce nu încape în legendă se vede ca stele în puncte fără nume. Un sistem care descoperă nu se uită la restul unei propoziții, ci la restul *unui corpus*: dacă aceeași stea necunoscută reapare la aceeași poziție relativă, în multe propoziții, restul nu e zgomot, e *structurat*. Asta e deja măsurat pe o lume construită: cu un rol scos din legendă, mărimea restului n-a trădat nimic, dar resturile s-au grupat după răspunsul ascuns, fără niciun vector și fără etichete (capitolul 8). O structură care se repetă în rest e un candidat de *concept nou*, iar „concept" are aici un înțeles operațional: o întrebare nouă în legendă, o stea cu nume, împreună cu felul cuvintelor care îi răspund. A

propune un concept înseamnă a propune un rând în legendă. A-l accepta e măsurabil: cu rândul adăugat și carnetul reantrenat (minute, capitolul 6), restul corpusului scade, cadrele se citesc cu marjă, iar răspunsurile la întrebarea nouă pot fi regăsite. Două piese există: unealta descrisă în depozitul proiectului are deja bucla în care un model propune rânduri de legendă și le păstrează doar pe cele care trec un criteriu măsurat (9.3), iar experimentul care a verificat dacă restul *arată spre* întrebarea lipsă a găsit că o face prin direcție, nu prin mărime (capitolul 8). Restul e speculație: că structura din rest poate fi grupată automat în candidați, și că modelul mare, văzând stelele necunoscute și contextele lor, poate propune *numele* întrebării.

Ce e valoros într-o ipoteză: un pariu, nu un oracol

Un sistem care propune mai mult decât poate încerca trebuie să aleagă. În termenii hărții, valoarea unei ipoteze se poate estima înainte de a o testa prin câteva mărimi, toate calculabile din constelații. *Acoperirea*: cât din restul corpusului ar explica, dacă ar fi adevărată și adăugată legendei. *Legătura*: câte domenii, până acum necorelate, ar lega între ele; o ipoteză care unește două părți ale grafului valorează mai mult decât una care adaugă un fapt la o parte deja densă. *Contradicția*: câte observații o contrazic, și în ce domeniu; o ipoteză contrazisă de date ferme e probabil greșită, una contrazisă de puține observații izolate indică date interesante. *Noutatea față de ce se știe*: distanța constelației ei față de centrul de greutate al ipotezelor deja confirmate; aproape înseamnă redundant, departe înseamnă nou sau absurd, și contradicția spune care. Valoarea estimată e, grosier, scăderea așteptată a restului neexplicat împărțită la costul încercării. E o *ordonare*, nu o predicție; cât de bună e se poate afla doar încercând ipoteze ordonate așa și numărând câte din primele s-au confirmat. Nu avem acel număr.

Cum își face review

Review-ul nu e un pas separat, ci verificatorul din 9.2 întors asupra propriilor concluzii. Fiecare afirmație pe care sistemul vrea s-o rețină e o propoziție, deci o constelație; e recitită și comparată cu constelațiile observațiilor, cu cele patru verdicte, cu marjele și cu restul ei. O concluzie *neverificabilă* nu e publicată, e pusă la coadă, cu motivul: domeniu nerecunoscut, domeniu aglomerat, în afara legendei. O concluzie *contrazisă* rămâne cu contradicția lângă ea, în domeniul numit. Și al doilea ochi din 6.4 se aplică: unde pilotul și modelul mare nu sunt de acord cu marjă mare, cazul e revăzut, de un alt model sau de un om. Regula din tot acest proiect, nimic dat fără declarație, se aplică sistemului însuși: fiecare concluzie poartă proveniența (din ce observații, din ce ipoteză, cu ce versiune a legendei), și o concluzie fără proveniență nu există.

Cum își gestionează înțelegerile

O *înțelegere câștigată* (un insight) e, operațional, un cadru confirmat care a schimbat ceva: un rând nou în legendă, o regularitate în graf, o contradicție rezolvată. Se păstrează ca ce este: constelația, cadrul, proveniența și versiunea legendei la care a fost citită. Pentru că e un punct, se regăsește prin asemănare: o întrebare nouă e citită, iar înțelegerile apropiate de constelația ei ies la suprafață fără index și fără cuvinte-cheie; centrul de greutate al unui grup de înțelegeri e o *temă*, și distanța până la el spune cât de departe e o întrebare de ce s-

a înțeles deja. Înțelegerile confirmate iar și iar sunt candidate să devină rânduri de legendă, adică să treacă din observat în declarat; cele contrazise ulterior coboară, cu contradicția înregistrată. Memoria aceasta nu e în ponderile niciunui model: e un catalog de constelații, inspectabil, pe care un om îl poate citi rând cu rând. Diferența față de memoria unui model mare nu e de capacitate, e de *adresabilitate*: știi ce e în ea și de unde vine fiecare lucru.

Cum și-ar îmbunătăți propriul model mare

Aici bucla din 9.2 se închide în sens invers, și e partea cea mai speculativă. Tot ce a fost verificat e material de antrenare curat: perechi (propoziție, cadru) pentru a formaliza mai bine; perechi (ipoteză, verdict) pentru a propune ipoteze mai bune; perechi (rest structurat, întrebare acceptată) pentru a propune concepte. Modelul mare se reantrenează sau se ajustează fin *numai* pe ce a trecut prin verificator, niciodată pe propriile ieșiri neverificate; harta e filtrul. Pilotul, la rândul lui, se reantrenează ori de câte ori legenda se schimbă, în minute, pentru că tot ce învață e carnetul. Oamenii rămân unde i-a pus capitolul 16: la aprobarea rândurilor de legendă, pentru că acolo se decide ce poate fi spus deloc. Nu e învățare de sine în sensul unui sistem care se îmbunătățește singur fără graniță; e o pereche în care partea fluentă nu are voie să învețe decât din ce partea verificabilă a confirmat, și în care fiecare pas lasă o urmă pe care o poți citi.

Ce e construit, ce e planificat, ce e speculație

Construit și măsurat: cititorul cu marjele și restul lui, verificatorul cu cele patru verdicte, profesorul pe un eșantion, bucla de legendă a uneltei cu criteriul ei de acceptare. Măsurat pe o lume construită: că o întrebare lipsă din legendă se vede în direcția restului, nu în mărimea lui (capitolul 8). Speculație, fără nicio măsurătoare: gruparea automată a restului în candidați de concept, ordonarea ipotezelor după valoare, memoria de înțelegeri ca catalog de constelații, și reantrenarea modelului mare pe cadre verificate. Capitolul acesta nu cere să fie crezut; cere să fie citit ca o listă de experimente care nu au fost făcute, scrise suficient de precis ca să se vadă, pentru fiecare, ce ar însemna să eșueze.

18. Ce este, și ce nu este

Povestea cu cerul, sonda și pilotul e o imagine; capitolul acesta o lasă deoparte și spune ce fel de sistem e, față de ce se compară, și unde îi sunt limitele, în termenii în care s-ar discuta într-un articol.

Ce este

Este un *cititor vector-simbolic* de propoziții: o schemă de adresare învățată, pusă peste o algebră fixă. Algebra e cea a arhitecturilor vector-simbolice cu faze (binding prin înmulțire element cu element, bundling prin adunare, cleanup prin cel mai apropiat vecin); identitățile cuvintelor și vectorii de rol sunt aleatori și ficși; ce se învață sunt vectorii de deplasare pe categorie de cuvânt, vectorii relativi ai așteptărilor și o poartă pe categorie, în total zece–cincisprezece megaocteți. Ieșirea e o stare care codifică un cadru rol-umplutură, imbricat prin binding repetat, și care se citește înapoi prin unbinding și cleanup, cu o marjă pe rol și un reziduu de reconstrucție pe propoziție. Obiectivul de antrenare e potrivirea,

prin cosinus, cu o stare-țintă construită de o lume cu roluri explicite; supervizarea e structurală și pe propoziție, cu roluri date dinainte, și nu conține etichete de parafrază.

Ca tip de sistem, e un analizor semantic în cadre (un *frame parser*) cu o particularitate geometrică: starea nu e produsă de un decodor, ci compusă algebric, astfel încât compoziția la adâncimi nevăzute, tratamentul vocabularului nou și diagnosticele de incertitudine decurg din algebră, nu din învățare. Nu deține fapte: faptele stau într-o bază de date sau un motor de raționament, iar cadrul se traduce mecanic în fapte Prolog sau constrângeri SMT.

Față de ce se compară

Față de transformere și modelele de limbaj. Nu există obiectiv de predicție a cuvântului următor și nu există generare. Comparația cinstită e cu un transformer mic antrenat pe aceeași intrare și aceleași ținte: pe lumi construite, la o adâncime de imbricare neîntâlnită la antrenare, cititorul decodează rolurile interioare complet, iar transformerul e la nivelul hazardului; pe WikiSQL, într-o configurație modificată față de protocolul oficial (întrebarea singură, fără schema tabelului; se prezice coloana selectată, nu interogarea executată), coloana selectată e 0,82 față de 0,54 pentru transformerul cu aceeași intrare (0,65 cu un buget de zece ori mai mare) și 0,50 pentru o regresie logistică pe sac de cuvinte, la toate mărimile setului de antrenare începând de la o mie de exemple (tabelul pe mărimi, cu semințe și intervale, e în depozit). În sens invers, acolo unde decizia e lexicală (agregatorul din aceleași interogări), clasificatorii neuronali câștigă (0,89), iar acuratețea pe interogarea întreagă (0,37) nu e comparabilă direct cu cifrele sistemelor specializate, care văd schema tabelului și raportează alte metrici; e departe de ele. Afirmația susținută e deci precisă: pe aceeași intrare, fără cunoaștere externă, cu date puține, cititorul învață structura de roluri mai bine și generalizează compozițional; nu e un concurent pe text deschis și nu pretinde să fie.

Față de analizoarele semantice cu aceeași supervizare (etichetare de roluri semantice, parsare în cadre). Supervizarea e de același fel: ținte structurate pe propoziție. Diferențele sunt reprezentarea (stare vectorială compozabilă, nu arbore produs de un decodor), vocabularul (identități aleatoare plus categorie, deci cuvinte noi fără reantrenare) și citirea incertitudinii (marjă, reziduu, abținere). Comparația directă cu un analizor clasic antrenat pe exact aceleași ținte, clasificator pe rol, e în planul de măsurători; până atunci, comparăm pe proprietăți, nu pe cifre.

Față de reprezentările tensoriale și holografice (Smolensky, Plate, Kanerva, codarea cu puteri fracționare a lui Frady, Kleyko și Sommer). Codarea cadrelor ca sumă de perechi întrebare răspuns e standard și veche de treizeci de ani. Ce e nou aici e că adresele sunt *învățate* din obiectivul generativ, că sunt redundante (trei coduri de adresă topite), că adresarea prin așteptări cu reașezarea cuvintelor timpurii dă adrese nete și compune cu un nivel mai departe decât a fost antrenată, și că porțile (ce categorii deschid cadre) emerg prin gradient și coincid între lumi și cu textul real (cu trei porți fixate de mână și declarate, pasul 8 din capitolul 13). Am verificat și ce *nu* emerge: categoriile de cuvinte coincid cu cele date de un skip-gram simplu pe aceleași propoziții și nu sunt revendicate.

Față de teoria celor o mie de creiere, capitolul 15: cadre de referință cu locații date de mișcare, trăsături la locații, compoziție prin deplasare, vot prin suprapunere; fără mii de coloane, fără învățare nesupervizată, cu cadrul dat de lume.

Față de modelele de lume latente (Dreamer, JEPA): aceeași sarcină formală, stare plus eveniment dă starea următoare, cu o stare lizibilă în loc de una latentă; direcție, nu rezultat.

Ce nu este

Nu e un model de limbaj; nu e o bază de cunoștințe; nu e nesupervizat; nu e un înlocuitor al modelelor mari pe text deschis. Locul lui realist e lângă un model mare, ca partea care produce un cadru verificabil și spune ce n-a putut așeza.

Ce ar putea schimba mesajul

E cinstit să spunem noi care măsurători viitoare pot muta mesajul științific al acestei cărți, și în ce direcție. Trei o pot face. *Calibrarea*: dacă rata greșelilor făcute cu încredere iese mare și nu scade cu îmbunătățirile cunoscute, „spune unde n-a înțeles" devine „raportează incertitudine, dar prost calibrată", și capitolele despre verificare, robotică și aliniere trebuie rescrise în consecință; e măsurătoarea cu cea mai mare miză. *Textul real cu parafraze libere, față de o bază de comparație cu aceeași supervizare*: dacă un clasificator clasic pe rol, antrenat pe exact aceleași ținte, egalează cititorul pe text real, mesajul se mută de la „învață structura mai bine" la „comparabil ca acuratețe, cu compoziție la adâncimi nevăzute și cu diagnostice"; unicitatea rămâne la compoziție și la citirea înapoi, nu la cifre. *Operatorii de după subiect*: subiectul urmat de doi operatori e citit corect în cadru (capitolul 8, „Ordinea"), dar adresa lui nu deosebește cele două ordini; dacă o lume are nevoie de subiect ca martor al structurii, „operatorii sunt roluri, citiți fără reguli" capătă acolo o graniță declarată.

Ce nu se poate schimba: faptele matematice (izometria, comutativitatea, lipsa de legătură a punctelor aleatoare, scalarea capacității) și rezultatele deja măsurate pe lumi construite. Iar câteva măsurători pot doar îmbunătăți: pronumele, codul rar. Teza, sensul ca răspunsuri la întrebările unei legende, atinse printr-un drum învățat, stă pe ce nu se poate schimba.

Limite

Limitele structurale: capacitatea unei stări (zeci până la sute de perechi la o mie și ceva de dimensiuni; textele lungi sunt structuri de stări); comutativitatea binding-ului (ordinea a doi operatori e reținută doar când altceva diferă; remediile necomutative cedează compoziția liberă și sunt de testat); citirea doar înainte (un cuvânt dinaintea a doi operatori suprapuși e așezat doar de primul, cu un singur cost măsurat în cadru, o formă la adâncimea nevăzută; capitolul 8, „Ordinea"); un cuvânt nou a cărui categorie dicționarul și etichetatorul n-o pot da rămâne fără cheie (8, „Cheia ruptă").

Rezultatele de la momentul scrierii vin din lumi construite, cu cinci semințe, și dintr-un singur set real. Ce e măsurat, ce e în lucru și cu ce criteriu se găsește, la zi, în fișierul de stare al proiectului:

https://github.com/ml-circuits-research/vsa-semantic-space/blob/main/STATUS.md. Documentul acesta nu se schimbă la fiecare experiment; fișierul da.